Azelmodsx677 · Documentación técnica

NexusChat 8.4.1

Un mensajero hecho para el anonimato. Qué es, cómo está construido, por qué se decidió cada cosa y qué hace cada parte de su código.

Documento completo en tres partes

I. El producto y sus decisiones técnicas
II. Guía técnica para desarrolladores
III. Referencia de cada archivo y función

Contenido

Parte I · El producto y sus decisiones técnicas

Capítulo 1	Por qué existe NexusChat
Capítulo 2	El producto en cifras
Capítulo 3	Arquitectura
Capítulo 4	Cómo se protege un mensaje
Capítulo 5	Anonimato: Tor y modo anónimo
Capítulo 6	Seguridad del servidor
Capítulo 7	Funciones en la nube
Capítulo 8	Actualizaciones sin tienda
Capítulo 9	La lista de chats en tiempo real
Capítulo 10	Llamadas y videollamadas
Capítulo 11	El estudio de diseño
Capítulo 12	Inteligencia artificial
Capítulo 13	Herramientas avanzadas
Capítulo 14	Funciones de mensajería
Capítulo 15	Calidad y pruebas
Capítulo 16	Hoja de ruta
Capítulo 17	Tecnologías

Parte II · Guía técnica para desarrolladores

Cómo estudiar esta guía · Visión general y tecnologías · Mapa de carpetas · Arquitectura por capas · Arranque y navegación · Cifrado · Sistema de diseño · Compilar y probar · Ejercicios y glosario

Parte III · Referencia del código

Cada archivo Kotlin de la app, para qué sirve y qué hace cada una de sus funciones, ordenado en 32 áreas: Chat, Lista de chats, Navegación principal, Historias, Llamadas, Grupos, Chats secretos, Canales, Perfil, Ajustes y estudio de diseño, Seguridad, Registro e inicio de sesión, Inteligencia artificial, Editor de código, Terminal, Descargas, Otras pantallas, Componentes, Tema, Rutas, Interfaz, varios, Repositorios, Seguridad y cifrado, Motor de IA, Base de datos local, Copias de seguridad, Datos, varios, Casos de uso, Servicios, WebRTC, Inyección de dependencias, Aplicación.

276
archivos Kotlin

2163
funciones

811
con explicación del autor

Parte I

El producto y sus decisiones técnicas

Para quien quiere entender qué es NexusChat y por qué está construido así: la misión, la arquitectura, la seguridad, el servidor y la hoja de ruta, contado por su desarrollador.

Capítulo 1

Por qué existe NexusChat

NexusChat nació con un objetivo: un mensajero en el que puedas hablar sin dejar rastro. Todo lo demás, desde el cifrado hasta el diseño, es consecuencia de esa decisión.

Los mensajeros masivos resolvieron hace años el problema de llegar a todo el mundo. No resolvieron el de no saber nada de quien los usa. Incluso los que cifran el contenido conservan metadatos: quién habla con quién, cuándo, desde qué red y con qué frecuencia. Para mucha gente esa información es más sensible que el propio mensaje.

Por eso construí NexusChat alrededor de tres compromisos que no se negocian:

- **Lo que escribes solo lo leen las personas a las que se lo envías.** El servidor guarda texto cifrado y una etiqueta genérica, nunca el mensaje. Esto incluye la vista previa de la lista de chats, que en otras apps suele quedar en claro.
- **Puedes ocultar desde dónde te conectas.** La app integra Tor, la misma red que usan periodistas y activistas, sin instalar nada aparte. Hay un modo anónimo que además deja de publicar tu "en línea" y tu "última vez".
- **No hay modelo de negocio basado en tus datos.** No hay anuncios, no hay rastreadores publicitarios y no se venden ni se comparten datos.

Sobre esa base añadí lo que hace que una app se use a diario: llamadas, historias, canales, grupos, stickers, inteligencia artificial con tu propia clave y un estudio de diseño que permite que la app de cada persona se vea distinta. La privacidad no sirve si nadie quiere usar la app; la personalización es lo que hace que la gente se quede.

A quién va dirigida

A quien quiere un mensajero completo sin entregar su vida digital a cambio: usuarios conscientes de su privacidad, comunidades que necesitan hablar con seguridad y personas que quieren una app que se sienta suya.

Qué la hace distinta

	NexusChat	Mensajeros masivos
Vista previa del chat en el servidor	Cifrada	Normalmente en claro o derivada
Ocultar la red desde la que te conectas	Tor integrado	Proxy externo, como mucho
Modo anónimo de actividad	Sí, un interruptor	Ajustes dispersos
Chats secretos con clave nueva en cada mensaje	Sí	Según la app
Diseño de la interfaz	Estudio completo	Temas de color
Inteligencia artificial	Con tu clave y tu proveedor	Del fabricante, sin elección

Capítulo 2

El producto en cifras

73.162 líneas de Kotlin	**275** archivos de código	**2.154** funciones	**13** funciones en la nube
83 librerías integradas	**120** pruebas automáticas	**12** campos de escribir	**10** diseños de fila
8 barras inferiores	**21** tipos de letra	**36** colores de acento	**2** idiomas

Funciona en Android 12 o superior y está compilada para las versiones más recientes del sistema. Se distribuye como APK firmado desde GitHub, con un sistema de actualización propio que avisa a cada teléfono cuando sale una versión nueva.

Capítulo 3

Arquitectura

La app sigue una arquitectura en capas: la interfaz no habla con el servidor, los repositorios no saben cómo se dibuja nada y la lógica de negocio vive en un sitio donde se puede probar.

```
Pantallas (Jetpack Compose)
      │  observan StateFlow
ViewModels  ──────────────  Casos de uso (domain)
      │  llaman
Repositorios  ── Firebase (Auth, Realtime Database, Storage, FCM)
      │          Room (caché local) · DataStore · Keystore
Servicios     ── Cifrado · Tor · WebRTC · Notificaciones · Trabajos en segundo plano
```

Por qué Jetpack Compose

El estudio de diseño cambia filas, burbujas, campos y barras en vivo. Con vistas XML cada variante sería un layout distinto y cambiar de estilo obligaría a recrear pantallas. Con Compose el estilo es un valor: cambia el valor y la interfaz se redibuja. Eso es lo que hace posible ofrecer decenas de variantes sin multiplicar el código.

Por qué MVVM con StateFlow

Cada pantalla tiene un ViewModel que expone un único estado inmutable. La pantalla solo lo dibuja. Así una rotación, un cambio de tema o volver de segundo plano nunca pierden datos, y el comportamiento se puede probar sin abrir la app.

Por qué Hilt

La inyección de dependencias permite que un ViewModel reciba un repositorio sin construirlo. En producción recibe el real; en una prueba, uno falso. También garantiza que hay una sola instancia del servicio de cifrado, de la base de datos o del cliente de red.

Por qué Firebase

Para un desarrollador independiente, Firebase ofrece autenticación, base de datos en tiempo real, almacenamiento y notificaciones con escalado automático y sin servidores que mantener. La clave es que **no confío en el servidor**: los mensajes llegan cifrados, y las reglas de seguridad impiden que un usuario lea lo que no le corresponde aunque modifique la app.

Organización del código

Carpeta	Qué contiene
ui/screens	Cada pantalla con su ViewModel: chat, inicio, historias, llamadas, grupos, canales, ajustes, IA, terminal, editor, descargas.
ui/components	Piezas reutilizables: avatares, burbujas, filas, barras, el aro animado y el objeto ChatLook con toda la personalización.
data/repository	Acceso a Firebase: mensajes, chats, usuarios, archivos, llamadas.
data/security	Cifrado de extremo a extremo, chats secretos, cifrado de grupos, Tor y modo anónimo.
data/ai	Proveedores de IA, agentes y sus herramientas.
data/local	Caché con Room para abrir la app sin conexión.
domain	Casos de uso: iniciar sesión, cifrar, descifrar.
service, webrtc	Notificaciones push, servicio de llamada y conexión WebRTC.
functions	Las 13 funciones en la nube en Node.js.

Capítulo 4

Cómo se protege un mensaje

El recorrido de un mensaje explica la mitad de la app. Lo describo paso a paso, desde que pulsas enviar hasta que aparece en el otro teléfono.

1. **Identificador del chat.** Un chat de dos se llama `uidA_uidB`, con los identificadores ordenados. Es determinista: los dos teléfonos llegan al mismo chat sin preguntarle a nadie, y las reglas de Storage pueden comprobar que eres participante leyendo solo el nombre.
2. **Claves.** Cada teléfono genera su par de claves y guarda la privada en el almacén seguro del sistema. Solo la pública sube al servidor.
3. **Secreto compartido.** Con tu clave privada y la pública del otro se calcula un secreto que solo conocéis los dos (ECDH). Es simétrico: el mismo secreto sirve para lo que envías y para lo que recibes, así que también puedes leer tus propios mensajes.
4. **Cifrado.** El texto se cifra con ese secreto. Al servidor llegan el texto cifrado (`encryptedPayload`) y una etiqueta fija, "Mensaje cifrado de extremo a extremo".
5. **Vista previa.** La lista de chats necesita mostrar el último mensaje. En lugar de guardarlo en claro, se guarda cifrado en `lastMessagePayload` y cada teléfono lo descifra al dibujar la lista.
6. **Aviso.** Una función en la nube detecta el mensaje nuevo y envía la notificación push, que no contiene el texto.
7. **Recepción.** El otro teléfono descifra con el mismo secreto y lo muestra.

Decisión: el cifrado nunca bloquea el envío

Si el destinatario todavía no ha generado sus claves, por ejemplo porque nunca abrió la app nueva, el mensaje no se puede cifrar para él. Bloquear el envío dejaría la conversación rota, así que en ese caso se envía sin cifrar y queda marcado. En cuanto ambos tienen claves, todo se cifra.

Decisión: qué pasa al cambiar de teléfono

Si alguien reinstala la app, genera claves nuevas. Los mensajes antiguos se cifraron con las anteriores y ya no se pueden abrir en ese dispositivo. Es la garantía del cifrado de extremo a extremo funcionando, no un fallo. La app lo explica con un aviso claro en lugar de un error.

Chats secretos: la cadena que olvida

Los chats secretos van más allá: cada mensaje usa una clave derivada de la anterior y la anterior se borra (trinquete criptográfico). Si alguien robara el teléfono mañana, no podría descifrar lo que se dijo ayer. Además viven solo en los dos teléfonos.

Grupos cifrados

En un grupo no se cifra cada mensaje para cada miembro, porque sería lento. El grupo tiene una clave que se reparte cifrada a cada miembro con su clave pública. Cuando alguien sale o es expulsado, la clave se rota y quien salió no puede leer lo siguiente.

Capítulo 5

Anonimato: Tor y modo anónimo

El cifrado protege el contenido; el anonimato protege el contexto. NexusChat integra Tor con `tor-android` y controla el proceso con `jtorctl`. Cuando se activa, el tráfico sale por la red Tor y el servidor ve una dirección de salida de Tor, no la tuya.

El modo anónimo, controlado por `AnonymityPolicy`, cambia el comportamiento de toda la app de una vez:

- Publica "desconectado" y deja de actualizar tu última conexión.
- Desactiva las vistas previas de enlaces, porque descargar una vista previa revelaría tu dirección al sitio enlazado.
- Los mensajes temporales no dejan vista previa en ningún servidor externo.

La decisión de diseño es que el anonimato sea un único interruptor y no veinte ajustes repartidos. Si proteger tu identidad exige ser experto, la mayoría no lo hará.

Capítulo 6

Seguridad del servidor

Asumo que alguien modificará la app para intentar leer lo que no debe. Por eso la seguridad real está en las reglas del servidor, que se ejecutan en Firebase y no se pueden saltar desde el teléfono.

Dato	Quién puede leerlo	Quién puede escribirlo
Chat y mensajes	Sus miembros	Sus miembros, salvo bloqueados
Índice de chats de un usuario	Solo ese usuario	Ese usuario o un miembro del chat que lo añade
Ajustes de chat (fijar, silenciar, archivar)	Solo su dueño	Solo su dueño
Llamadas	Sus participantes, mediante índice propio	Sus participantes
Reportes	Nadie desde la app	Cualquier usuario, firmando con su identidad
Archivos de un chat de dos	Sus dos participantes	Sus dos participantes
Copias de seguridad	Solo su dueño	Solo su dueño, hasta 500 MB

Decisiones que corrigen errores reales

- **Ajustes personales separados.** Fijar, silenciar y archivar vivían dentro del chat compartido, así que silenciar una conversación también la silenciaba a la otra persona. Ahora viven en `chat_settings/{usuario}`.
- **Índice de llamadas.** Consultar el historial filtrando todas las llamadas exigía permiso de lectura sobre las de todo el mundo. Con un índice por usuario, cada uno lee solo las suyas.
- **Chats huérfanos.** Un chat que perdió su lista de miembros quedaba bloqueado para siempre. Ahora puede reclamarlo quien aparece en su identificador, y nadie más.
- **Grupos de los que te expulsaron.** La app los retira de tu índice en lugar de reintentar sin fin.
- **Enlaces internos.** La app nunca muestra rutas de Storage ni de la base de datos en mensajes, stickers o vistas previas.
- **Registros.** La versión publicada elimina los registros de depuración al compilar y nunca registra el contenido de un mensaje.

Capítulo 7

Funciones en la nube

Son 13 funciones en Node.js 22. Ninguna lee el contenido de los mensajes: trabajan con metadatos mínimos.

Función	Qué hace
onMessageCreate	Envía la notificación push de un mensaje nuevo a los teléfonos del destinatario.
onCallCreate · onCallEnded	Hacen sonar el teléfono que recibe una llamada y avisan de las perdidas.
onStoryCreate · onStoryReaction	Avisan de historias nuevas y de reacciones a las tuyas.
indexEphemeralMessage · cleanupExpiredMessages	Registran cuándo caduca un mensaje temporal y lo borran del servidor a su hora. Se ejecuta cada minuto.
cleanupExpiredStories	Borra las historias pasadas 24 horas.
onSecretMessage	Avisa de mensajes en chats secretos sin conocer su contenido.
onChannelPost · onChannelSubscriber	Notifican publicaciones de canales y mantienen el recuento de suscriptores.
premiumApi	Gestiona la suscripción premium.
checkAppRelease	Cada 15 minutos consulta la última versión publicada en GitHub. Si es nueva, la anuncia a todos los teléfonos.

Los registros de producción muestran tiempos de ejecución de entre 5 y 300 milisegundos y ningún error.

Capítulo 8

Actualizaciones sin tienda

Mientras la app se distribuye fuera de Play Store, necesitaba un sistema de actualización propio y fiable:

1. Publico una versión en GitHub con su APK.
2. `checkAppRelease` la detecta en un máximo de 15 minutos y escribe la versión en `config/appUpdate`.
3. Envía una notificación push a todos los teléfonos.
4. La app compara su versión y muestra la pantalla de actualización. Es obligatoria salvo que las notas digan "[opcional]", para que nadie siga con un fallo de seguridad corregido.
5. Elige el APK adecuado para el procesador del teléfono o el universal.

Cada versión se compila con R8, que reduce y ofusca el código, y se prueba instalada en un teléfono real antes de publicarse. Esa regla nació de un fallo real: la 8.0.0 funcionaba en depuración pero se cerraba al abrir la versión publicada, porque R8 había renombrado una librería. Desde entonces ninguna versión sale sin probarse tal y como la recibirá el usuario.

Capítulo 9

La lista de chats en tiempo real

Cada chat guarda todo su historial bajo el mismo nodo. Escuchar el nodo entero para mostrar la lista descargaría todos los mensajes de todos los chats. Por eso la app escucha solo los campos de la vista previa de cada chat: último mensaje, hora, remitente, no leídos y estado de escritura.

Hay un detalle que costó encontrar: la lista solo se publica cuando todos los chats han respondido. Si un chat desaparecía mientras cargaba, quedaba un campo pendiente para siempre y la lista nunca se mostraba. Ahora los pendientes se cuentan por chat y se descartan con él.

Para abrir la app sin conexión, Room guarda una copia local de chats y perfiles. La pantalla aparece al instante con lo último conocido y se actualiza cuando llega la red.

Capítulo 10

Llamadas y videollamadas

Las llamadas usan WebRTC: el audio y el vídeo viajan directamente entre los dos teléfonos y cifrados por el propio protocolo (DTLS-SRTP). El servidor solo hace de buzón para que los teléfonos se encuentren: la oferta, la respuesta y los candidatos de conexión pasan por la base de datos.

Los servidores que ayudan a conectar se leen de `config/webrtc/iceServers`, así que se pueden cambiar sin publicar otra versión.

Limitación conocida

Ahora mismo solo hay servidores STUN. Cuando los dos teléfonos están detrás de redes estrictas, como muchas redes móviles, hace falta un servidor TURN que retransmita. Sin él, algunas llamadas se quedan en "Conectando". Es el siguiente paso de infraestructura y no requiere cambiar la app: basta con añadir el servidor a la configuración.

Capítulo 11

El estudio de diseño

Todo el aspecto vive en un objeto, `ChatLook`. Cada opción es un flujo de estado que se guarda en las preferencias del teléfono. Cuando cambias una opción, cada pantalla que la usa se redibuja sola. No hay que reiniciar nada.

Qué se personaliza	Variantes
Filas de la lista	10: tarjeta, barra lateral, línea, relieve, borde, tarjeta 3D, neón, cristal, foto de fondo y más
Campo de escribir	12, incluido el aro animado y el texto propio en lugar de "Mensaje"
Burbujas	8
Inicio	7
Ajustes	7
Barras inferiores	8, con 7 juegos de iconos
Tipografía y color	21 tipos de letra y 36 colores, o cualquier color en hexadecimal

La vista previa del estudio usa los mismos componentes que la app real: lo que ves al configurar es exactamente lo que verás en tus chats.

El aro animado

Es una réplica nativa de un componente de la comunidad de mods: tu foto dentro de un aro que gira en 10 segundos, crece y se recoge en 2 con desaceleración, con arcos en degradado y una pequeña reducción al tocarlo. Se dibuja directamente sobre el lienzo de Compose, sin imágenes, así que se adapta a cualquier tamaño y color.

Capítulo 12

Inteligencia artificial

La IA de NexusChat no es un servicio del fabricante. El usuario elige el proveedor compatible y pone su propia clave, así que lo que envía va directamente a ese proveedor y nunca pasa por mis servidores. Es coherente con el objetivo de anonimato: yo no veo esas conversaciones.

- **Chat con la IA** con respuesta en tiempo real mediante eventos del servidor (SSE).
- **Agentes** que encadenan pasos y usan herramientas. Antes de guardar un archivo o hacer algo con efecto, piden permiso.
- **Azel Assistant**, el asistente de bienvenida, funciona sin conexión y sin clave. Desde la 8.4 lee el historial de cambios incluido en la app, así que conoce las novedades de cada versión sin reescribirlo a mano.

Capítulo 13

Herramientas avanzadas

- **Terminal** con emulador real y entorno Kali Linux, para usuarios técnicos.
- **Editor de código** con pestañas y resaltado de sintaxis.
- **Descargador de vídeos** basado en yt-dlp y FFmpeg. Las webs cambian a menudo, así que se actualiza solo y, si un intento falla, trae la versión más reciente y reintenta.
- **Navegador privado** integrado.
- **Copias de seguridad cifradas** que solo puede leer su dueño.

Capítulo 14

Funciones de mensajería

Área	Funciones
Mensajes	Responder citando con el nombre de quien escribió, reenviar con etiqueta, editar durante 15 minutos, borrar para todos, fijar, guardar, programar, traducir, menciones, texto oculto, reacciones con la lista de quién reaccionó
Privacidad	Mensajes temporales, de una sola vista, chats ocultos tras tu huella, bloquear y reportar
Multimedia	Fotos y vídeos con texto, notas de voz distintas de los audios, documentos, ubicación, contactos, GIF, stickers con favoritos
Organización	Carpetas, archivar, silenciar por tiempo, marcar como no leído, exportar un chat a texto, búsqueda global
Comunidad	Grupos cifrados con foto, descripción, administradores e invitación por enlace; canales públicos; historias de 24 horas con respuestas y stickers
Cuenta	Registro con correo, Google o teléfono; perfil con foto o vídeo; eliminar la cuenta desde la app; política de privacidad incluida

Capítulo 15

Calidad y pruebas

- **120 pruebas automáticas** que se ejecutan antes de cada versión.
- **Prueba en teléfono real** de la versión compilada con R8, con registro del sistema para confirmar cero cierres y cero errores de permisos.
- **Crashlytics** para conocer cualquier cierre en los teléfonos de los usuarios, con el mapa de ofuscación subido en cada compilación.
- **Revisión del servidor:** las reglas desplegadas se comparan con las del repositorio, y los registros de las funciones se revisan buscando errores y tiempos.

Un ejemplo del método: un usuario reportó "no tengo permisos" al usar la app. En lugar de suponer, leí el registro del teléfono y encontré la causa exacta: un grupo del que había sido expulsado seguía en su índice y la app abría 15 escuchas que el servidor rechazaba. La corrección retira esos chats automáticamente y se verificó en el teléfono con cero errores tras reabrir.

Capítulo 16

Hoja de ruta

Comparé NexusChat con WhatsApp, Telegram y Signal función por función. Esto es lo que falta, por orden de impacto:

1. **Servidor TURN** para que las llamadas conecten en cualquier red.
2. **Llamadas de grupo.**
3. **Grupos:** que solo escriban los administradores, aprobación de nuevos miembros y modo lento.
4. **Enlaces de llamada** y compartir pantalla.
5. **Dispositivos vinculados**, que exigen sincronizar claves entre dispositivos sin romper el cifrado.
6. **Publicación en Play Store**, con una variante que respete sus políticas: sin instalación de APK propia y sin descargador de vídeos.

Capítulo 17

Tecnologías

Área	Tecnología	Por qué
Lenguaje	Kotlin, corrutinas	Lenguaje oficial de Android; las corrutinas evitan bloquear la interfaz.
Interfaz	Jetpack Compose, Material 3	Interfaz declarativa, la base del estudio de diseño.
Dependencias	Hilt	Inyección de dependencias comprobada al compilar.
Servidor	Firebase Auth, Realtime Database, Storage, Cloud Messaging, Crashlytics	Tiempo real y escalado sin mantener servidores.
Funciones	Cloud Functions en Node.js 22	Notificaciones, limpieza y actualizaciones en el servidor.
Local	Room, DataStore, Android Keystore, Security Crypto	Caché sin conexión, preferencias y claves protegidas por el sistema.
Cifrado	libsignal, ECDH, AES	Primitivas criptográficas probadas, sin inventar criptografía propia.
Anonimato	tor-android, jtorctl	Red Tor dentro de la app.
Llamadas	WebRTC	Audio y vídeo directos y cifrados entre teléfonos.
Multimedia	Media3 (ExoPlayer y Transformer), Coil 3, Lottie, CameraX	Vídeo, imágenes, animaciones y cámara.
Visión	ML Kit: códigos QR y recorte de sujetos	Añadir contactos por QR y crear stickers.
Red	OkHttp con SSE	IA con respuesta en tiempo real.
Herramientas	Emulador de terminal, editor con resaltado, yt-dlp, FFmpeg	Terminal, editor de código y descargas.
Trabajo en segundo plano	WorkManager	Tareas que sobreviven al cierre de la app.

La segunda parte de este documento es la referencia completa del código: cada archivo, para qué sirve y cada una de sus funciones.

Parte II

Guía técnica para desarrolladores

Cómo estudiar y modificar el código: recorridos por los flujos reales, recetas para añadir funciones y ejemplos comentados.

Cómo estudiar esta guía

Cada capítulo explica una parte y te indica qué archivos abrir. Las rutas son relativas a `app/src/main/java/com/Azelmods/App/`, salvo que se indique otra cosa. Las cifras se contaron en el propio repositorio y los fragmentos son de tu código, a veces abreviados.

1	Visión general y tecnologías	9	Tor y anonimato
2	Mapa de carpetas	10	Agentes de IA
3	Arquitectura por capas	11	Sistema de diseño
4	Arranque y navegación	12	Actualizaciones
5	Vida de un mensaje	13	Compilar y probar
6	Cifrado	14	Puntos a mejorar
7	Servidor: reglas y funciones	15	Ejercicios y glosario
8	Llamadas		

Orden recomendado

1. **Base:** capítulos 1 a 4. Sin esto, lo demás parece magia.
2. **El corazón:** capítulo 5, siguiendo un mensaje con el depurador.
3. **Tu sello:** capítulo 11. Es lo más fácil de modificar y lo que más se ve.
4. **Lo difícil:** capítulos 6, 8, 9 y 10, en el orden que te interese.
5. **Practica:** los ejercicios del capítulo 15.

> **Método:** lee la explicación, abre el archivo, pon un punto de ruptura en Android Studio y provoca la acción en el móvil. Ver los valores cambiar enseña más que leer.

Capítulo 1

Visión general y tecnologías

NexusChat es una app Android nativa cuyo servidor es Firebase: la app habla directamente con la base de datos, protegida por reglas, y unas pocas funciones en la nube hacen lo que el teléfono no puede, como enviar notificaciones o borrar lo caducado.

273
archivos Kotlin

71.648
líneas Kotlin

52
rutas de navegación

35
archivos con ViewModel

30
nodos de base de datos

13
funciones en la nube

120
pruebas unitarias

1.424
textos traducidos al inglés

Área	Tecnología	Para qué
Lenguaje e interfaz	Kotlin, Jetpack Compose, Material 3	Toda la interfaz es declarativa
Inyección	Hilt	Entrega repositorios y servicios a los ViewModel
Asincronía	Coroutines, StateFlow, callbackFlow	Trabajo en segundo plano y datos en vivo
Servidor	Firebase Auth, Realtime Database, Storage, FCM, Functions	Cuentas, datos, archivos, avisos y lógica de servidor
Local	Room, DataStore, EncryptedSharedPreferences	Caché, preferencias y secretos
Multimedia	Media3, Coil, Lottie	Vídeo, edición, imágenes y stickers animados
Llamadas	WebRTC (stream-webrtc-android)	Voz y vídeo entre teléfonos
Anonimato	tor-android, jtorctl	Tor integrado
Red e IA	OkHttp con SSE	Proveedores de IA con respuesta en directo

Requisitos: `minSdk 31` (Android 12) y `targetSdk 37`. El servidor usa Node.js 22.

Capítulo 2

Mapa de carpetas

Carpeta	Archivos	Líneas	Qué contiene
`ui/screens`	95	39.751	Pantallas y sus ViewModel, por función: chat, home, settings, stories...
`ui/components`	32	7.814	Piezas reutilizables: avatar, burbuja, stickers, `ChatLook`
`data/security`	25	3.668	Cifrado, Tor, modo fantasma, bloqueo, TOTP
`data/ai`	15	3.552	Proveedores de IA y motor de agentes
`data/repository`	9	3.264	Acceso a Firebase y Storage
`data/backup`	4	1.180	Copias de seguridad cifradas
`service` , `services`	7	1.273	Notificaciones (FCM) y servicio de llamada
`webrtc`	4	824	Conexión de llamadas, audio y servidores ICE
`ui/theme`	7	819	Colores, tipografía y tokens de diseño
`data/local`	11	699	Room: 4 tablas, 4 DAO y `CacheManager`
`ui/navigation`	3	636	Rutas y grafo de navegación
`di`	4	271	Módulos de Hilt

Fuera de la app: `functions/index.js` (1.029 líneas de servidor), `database.rules.json` y `storage.rules` (permisos), y `app/src/test` (14 archivos de prueba).

Los archivos más grandes

Archivo	Líneas	Por qué importa
`RealtimeDatabaseRepository.kt`	1.764	Casi toda la comunicación con la base de datos
`ChatViewModel.kt`	1.711	Toda la lógica de una conversación
`AgentsPanel.kt`	1.209	Interfaz de los agentes
`ChatScreen.kt`	1.174	Pantalla de chat
`HomeScreenRedesigned.kt`	1.080	Lista de chats
`ChatComposer.kt`	1.077	Campo de escribir y sus 8 estilos

Capítulo 3

Arquitectura por capas

La app sigue MVVM: la pantalla solo dibuja, el ViewModel decide y el repositorio trae los datos. La regla es que cada capa solo conoce a la de abajo.



Las órdenes bajan; los datos suben como flujos que Compose observa.

Ejemplo real: la lista de chats sin conexión

`HomeViewModel` enseña primero lo guardado y luego escucha a Firebase. Por eso la lista aparece al instante incluso sin red.

```
// ui/screens/home/HomeViewModel.kt (abreviado)
val cachedEntities = cacheManager.getCachedChats()      // 1. Room: inmediato
if (cachedEntities.isNotEmpty()) _state.value = ...       // 2. se muestra ya
databaseRepository.getUserChats(userId)                   // 3. Firebase en vivo
    .debounce(120L)                                       // espera a que lleguen todos los campos
    .collectLatest { rawChats ->
        val enriched = rawChats.mapNotNull { buildChatFromMap(it, userId) }
        cacheManager.replaceChats(enriched)               // 4. actualiza la caché
        _state.value = _state.value.copy(chats = enriched)
    }
```

Inyección con Hilt

Los ViewModel no crean sus dependencias; las piden en el constructor. `di/AppModule.kt`, `FirebaseModule.kt`, `RepositoryModule.kt` y `SecurityModule.kt` saben fabricarlas. En una prueba puedes entregar una versión falsa, como hacen las pruebas de `ChatViewModel`.

Capítulo 4

Arranque y navegación

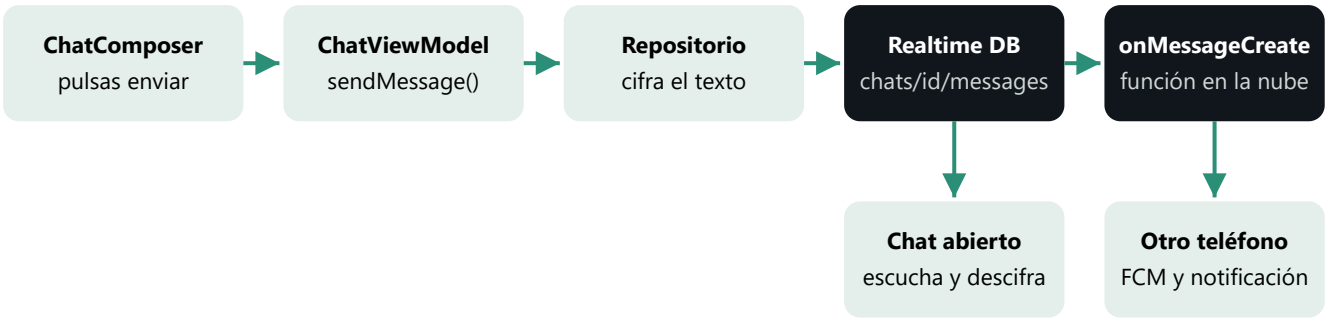
1. `MainActivity` arranca (es `FragmentActivity` por el diálogo de huella) e inicia `ChatLook` y `ChatOrganizer`, que leen tus preferencias.
2. `NavGraph.kt` define las 52 rutas con `composable(...)`. `Screen.kt` les da nombre para no escribir textos sueltos.
3. Tras la sesión se llega a `MainScreen`: un `HorizontalPager` con Chats, Historias, Llamadas y Perfil, más la barra inferior.

```
// ui/screens/main/MainScreen.kt: cada página del pager y su pantalla
when (tabOrder.getOrNull(page) ?: page) {
    0 -> HomeScreenRedesigned(navController = navController, onOpenTab = { ... })
    1 -> StoriesScreen(navController = navController)
    2 -> CallsScreen(navController = navController)
    3 -> ProfileScreen(navController = navController, userId = userId, showBack = false)
}
```

Hay dos niveles: el **pager** cambia de pestaña sin perder el estado, y el **NavController** abre pantallas encima, como un chat o Ajustes. Para añadir una pantalla basta registrar la ruta en `NavGraph.kt` y llamar a `navController.navigate("ruta")`.

Capítulo 5

Vida de un mensaje



Abre `sendMessageInternal` en `RealtimeDatabaseRepository.kt`. Si la lectura del chat falla, no asume que existe: sigue al bloque de creación, que fusiona datos en vez de reemplazarlos. Es un buen ejemplo de código defensivo.

Cómo se escucha la lista de chats

`getUserChats` es la mejor lección sobre Realtime Database. El índice `userChats/{uid}` dice en qué chats estás. Por cada uno escucha solo los campos de vista previa, nunca los mensajes, porque descargar `chats/{id}` entero traería toda la conversación.

```
fun getUserChats(userId: String) = callbackFlow {
    fun watch(chatId: String) {
        pendingFirstLoads += CHAT_PREVIEW_FIELDS.size
        watchers[chatId] = CHAT_PREVIEW_FIELDS.map { field ->
            val ref = database.child("chats").child(chatId).child(field)
            ref.addValueEventListener(listener); ref to listener
        }
    }
    val indexRef = database.child("userChats").child(userId)
    indexRef.addValueEventListener(indexListener)  // añade y quita chats
    awaitClose { /* retira todos los listeners: evita fugas */ }
}
```

Fíjate en tres ideas: `pendingFirstLoads` impide publicar una lista a medias, `callbackFlow` convierte callbacks en un flujo, y `awaitClose` libera todo al salir de la pantalla.

Datos en Realtime Database

Nodo	Contenido
`users`, `fcmTokens`	Perfiles, claves públicas y tokens de notificación
`userChats`, `chats`	Índice por usuario, y chats con sus miembros y mensajes
`groups`, `groupKeys`, `groupInvites`	Grupos, sus claves cifradas para cada miembro e invitaciones
`secretChats`, `userSecretChats`	Buzón temporal de chats secretos
`stories`, `calls`, `userCalls`	Historias, llamadas e historial
`channels`, `channelPosts`	Canales y publicaciones
`expiringMessages`	Lista de lo que debe borrarse y cuándo
`config`	Configuración remota, como la actualización

Capítulo 6

Cifrado

Tienes tres sistemas, cada uno con un equilibrio distinto entre comodidad y secreto.

	Chat normal	Grupo	Chat secreto
Clave	ECDH entre claves fijas de identidad	Una AES-256 aleatoria por grupo	ECDH efímero; una clave por mensaje
Cifrado	AES-256-GCM	AES-256-GCM	AES-256-GCM con datos asociados
Archivo	`E2EECryptoService.kt`	`GroupCrypto.kt`	`SecretRatchet.kt`, `SecretChatRepository.kt`
Secreto hacia adelante	No	Solo al cambiar la clave	Sí, avanza y olvida
Copia en servidor	Sí, cifrada	Sí, cifrada	Hasta entregarse

Chat de dos personas

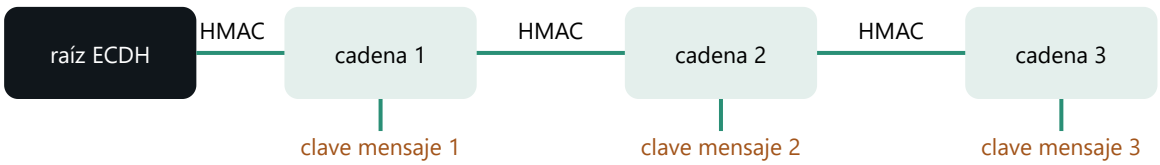
```
// data/security/encryption/E2EECryptoService.kt
val agreement = KeyAgreement.getInstance("ECDH")
agreement.init(local.private)              // mi clave privada
agreement.doPhase(remotePublic, true)       // la pública del otro
val key = MessageDigest.getInstance("SHA-256").digest(agreement.generateSecret())

val cipher = Cipher.getInstance("AES/GCM/NoPadding")
cipher.init(Cipher.ENCRYPT_MODE, SecretKeySpec(aesKey, "AES"))
val encrypted = cipher.doFinal(plaintext.toByteArray())   // se envía iv + encrypted
```

La idea de ECDH: tu privada con su pública da el mismo secreto que su privada con tu pública, sin que ese secreto viaje nunca. GCM añade una etiqueta que delata cualquier alteración. Las privadas se guardan por uid en `EncryptedSharedPreferences` y las públicas se publican en `users/{uid}/keys`. `safetyNumber()` permite comparar claves entre dos personas para detectar una sustitución.

> El comentario de esta clase explica un fallo real: antes las claves eran únicas por dispositivo, y con dos cuentas el segundo usuario reutilizaba la privada del primero, así que nada se descifraba. Por eso ahora son por uid.

Chat secreto: la cadena que olvida



HMAC no se puede invertir: quien robe la cadena 3 no obtiene las claves 1 y 2.

```
fun advance(chain: ByteArray) = Step(hmac(chain, MESSAGE_KEY_LABEL), hmac(chain, NEXT_CHAIN_LABEL))
```

Cada participante tiene su cadena (`chainFor`), de modo que nunca se reutiliza una clave. `seal` liga además cada mensaje a su chat, remitente y número mediante datos asociados.

Grupos y copias

`GroupCrypto` crea una clave aleatoria y la cifra por separado para cada miembro en `groupKeys/{chatId}/{keyId}/{uid}`. Si cambian los miembros se crea otra, y quien salió no puede leer lo siguiente. `BackupEncryptor` deriva la clave de tu contraseña con PBKDF2 (100.000 iteraciones) y usa AES-256-GCM con HMAC-SHA256.

Capítulo 7

Servidor: reglas y funciones

Como la app habla directamente con la base de datos, **las reglas son tu verdadera seguridad**. Cualquiera puede modificar una app; las reglas se ejecutan en el servidor.

```
// database.rules.json
"chats": { "$chatId": {
  ".read":  "auth != null && (!data.exists() || data.child('members').child(auth.uid).exists())",
  ".write": "auth != null && (!data.exists() || (... miembro y sin bloqueo ...))"
}}
```

> En Realtime Database **un permiso concedido por el padre no se puede retirar en un hijo**. Por eso el bloqueo se comprueba en `chats/$chatId` y no en cada mensaje. Es el error de seguridad más común en Firebase.

Funciones en la nube (`functions/index.js`)

Función	Se activa	Hace
`onMessageCreate`	Mensaje nuevo	Avisa a los otros miembros por FCM
`onSecretMessage`	Mensaje secreto	Avisa sin revelar contenido
`onStoryCreate` , `onStoryReaction`	Historia o reacción	Avisa a los contactos o al autor
`onCallCreate` , `onCallEnded`	Cambios en `calls`	Timbre y llamada perdida
`onChannelPost` , `onChannelSubscriber`	Canales	Publicaciones y suscriptores
`indexEphemeralMessage`	Mensaje temporal	Lo apunta en `expiringMessages`
`cleanupExpiredMessages`	Cada minuto	Borra lo caducado, hasta 500 cada vez
`cleanupExpiredStories`	Programada	Borra historias vencidas
`checkAppRelease`	Cada 15 minutos	Detecta releases nuevos de GitHub
`premiumApi`	Petición HTTP	Pagos de Premium; pendiente de desplegar

Capítulo 8

Llamadas

WebRTC conecta los dos teléfonos directamente; Firebase solo presenta a las partes (**señalización**).

1. Quien llama crea una oferta SDP (`createOffer` en `webrtc/WebRTCManager.kt`) y la escribe en `calls` .
2. `onCallCreate` envía el aviso y el otro teléfono muestra la llamada (`showIncomingCallNotification`).
3. Al aceptar se responde, y ambos intercambian candidatos ICE: las rutas de red posibles.
4. La media fluye directa; `CallService` mantiene viva la llamada en segundo plano.

`IceServerProvider.kt` entrega los servidores. Un STUN solo descubre tu dirección pública; en redes restrictivas hace falta un TURN que retransmita.

Capítulo 9

Tor y anonimato

Vive en `data/security/tor/`, con 16 archivos.

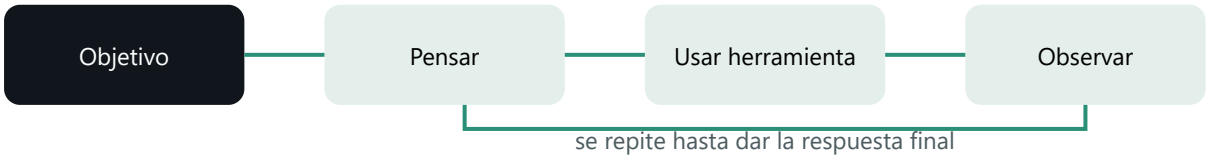
Archivo	Función
`EmbeddedTor.kt`	Arranca el Tor incluido, sin instalar nada
`TorService.kt`, `TorState.kt`	Ciclo de vida y estado de la conexión
`OrbotDetector.kt`, `OrbotIntents.kt`	Integración con Orbot si lo tienes
`FirebaseProxyConfigurator.kt`	Lleva el tráfico de Firebase por Tor
`TorDnsResolver.kt`	Resuelve DNS por Tor para evitar fugas
`TorConnectivityChecker.kt`	Comprueba que la salida es de verdad por Tor
`AnonymityPolicy.kt`	Decide qué se permite en modo anónimo

> La lección es **fallar cerrado**: si el modo anónimo está activo y Tor cae, es mejor no conectar que filtrar tu IP. Estudia `AnonymityPolicy` y `TorSocketRouter` con esa pregunta en mente.

Capítulo 10

Agentes de IA

El motor (`data/ai/agents/AgentOrchestrator.kt`, 512 líneas) sigue el patrón **ReAct**: pensar, usar una herramienta, observar y repetir hasta tener la respuesta.



Archivo	Papel
`AgentCatalog.kt`	Agentes y equipos, con sus instrucciones
`AgentProtocol.kt`	Protocolo JSON propio, en vez del tool calling de cada proveedor
`AgentToolbox.kt`	Las herramientas reales (919 líneas)
`JsSandbox.kt`	Ejecuta JavaScript en un WebView aislado
`SafeCalculator.kt`	Aritmética exacta: los modelos se equivocan
`WebSearch.kt`	Búsqueda en internet

> **Inyección de instrucciones:** un agente lee páginas de internet, y esas páginas pueden intentar darle órdenes. Por eso el JavaScript va aislado y guardar archivos pide tu permiso. Todo lo que venga de fuera es dato, no instrucción.

Capítulo 11

Sistema de diseño

Toda la personalización se apoya en `ui/components/ChatLook.kt`: un objeto con un `StateFlow` por opción, guardado en preferencias. Al cambiar el valor, cada pantalla que lo observa se redibuja sola.

Receta: añadir una opción

Así se añadió el juego de iconos de la barra:

```
// 1. Las opciones
enum class NavIconSet(val title: String) { OUTLINED("Contorno"), FILLED("Relleno") }
// 2. Estado y guardado en ChatLook
private val _navIconSet = MutableStateFlow(NavIconSet.OUTLINED)
val navIconSet: StateFlow<NavIconSet> = _navIconSet.asStateFlow()
fun setNavIconSet(set: NavIconSet) {
    _navIconSet.value = set
    prefs?.edit()?.putString("nav_icons", set.name)?.apply()
}
// 3. Leerlo al arrancar, en init()
// 4. Control en ChatStudioScreen.kt
ChoiceChips(NavIconSet.entries, navIcons, { it.title }, ChatLook::setNavIconSet)
// 5. Usarlo en la pantalla
val navIconSet by ChatLook.navIconSet.collectAsState()
```

Y añade el texto a `assets/i18n/en.json`, porque `tr("...")` traduce buscando la frase en español.

Dibujar a mano: el aro central

El aro de la barra se dibuja con `drawBehind`. Sus proporciones y su ritmo se tomaron del análisis de `ImgView` en V2: seis arcos de 48° y un giro cada 2 segundos.

```
val angle by spin.animateFloat(0f, 360f, infiniteRepeatable(tween(2_000, easing = LinearEasing)))
Modifier.drawBehind {
    val paint = Brush.linearGradient(listOf(accent, Color.White))
    rotate(angle) {
        repeat(6) { segment -> drawArc(paint, segment * 60f, 48f, false, topLeft, arcSize, style = stroke) }
    }
}
```

El color: `AccentState` es estado de Compose, y `NexusTokens.Color.Primary` lo lee, así que cambiar el acento repinta toda la app. Usa siempre tokens de `ui/theme/`, no colores fijos.

Capítulo 12

Actualizaciones

1. `checkAppRelease` consulta GitHub cada 15 minutos.
2. Con una versión nueva escribe `config/appUpdate` y avisa por FCM. `mandatory` es verdadero salvo que las notas lleven `[opcional]`.
3. `AppUpdateManager` escucha ese nodo, y `isNewerVersion` compara versiones.
4. `apkUrlFor` elige el APK por arquitectura o el universal, y solo acepta enlaces de confianza (`TRUSTED_DOWNLOADS`).
5. `AppUpdateDialog` no se puede cerrar si es obligatoria.

Capítulo 13

Compilar y probar

```
# APK de prueba
./gradlew :app:assembleDebug
# Pruebas unitarias (120)
./gradlew :app:testDebugUnitTest
# Análisis estático
./gradlew :app:lintDebug
# APK firmado, con R8; necesita keystore.properties, que nunca se sube
./gradlew :app:assembleRelease
# Servidor, con Node 22
npx -p node@22 -p firebase-tools firebase deploy --only database,storage,functions
```

Las pruebas usan JUnit y Kotest. El motor `junit-vintage-engine` es necesario para ejecutar las de JUnit 4: sin él pasaban por alto sin avisar. El release activa `isMinifyEnabled` (R8) y genera un APK por arquitectura más el universal.

Capítulo 14

Puntos a mejorar

Toda app de este tamaño tiene deuda técnica. Saber verla es parte de estudiarla.

Punto	Por qué importa	Qué haría
Chats normales sin secreto hacia adelante	Si roban la clave privada, se pueden leer los mensajes pasados	Llevar el ratchet de los chats secretos a todos
Cifrado sin auditoría externa	Los errores de criptografía son invisibles para las pruebas normales	Publicar el diseño y pedir una revisión
SHA-256 sobre el secreto ECDH en chats normales	HKDF, como en los secretos, es la práctica estándar	Unificar con HKDF; requiere migración
Logs con parte del contenido	`sendMessage` registra el inicio del texto antes de cifrar, y no hay regla R8 que retire los `Log`	Quitarlo y usar `-assumenosideeffects`
Archivos gigantes	`RealtimeDatabaseRepository` y `ChatViewModel` son difíciles de probar	Dividir por dominio: grupos, temporales, bloqueo
Carpetas duplicadas	`service` y `services`; `util` y `utils`	Unificarlas
Licencia	La pantalla dice derechos reservados; el repositorio, MIT; y hay librerías GPL	Alinear el texto y revisar esas dependencias

Capítulo 15

Ejercicios y glosario

Ejercicios, de fácil a difícil

1. **Una opción propia.** Añade un interruptor al Estudio siguiendo la receta del capítulo 11.
2. **Otra barra.** Crea una novena barra en `MainScreen.kt` con un icono que crezca al seleccionarlo.
3. **Sigue un mensaje.** Pon puntos de ruptura en `sendMessage`, `sendMessageInternal` y `encryptFor`. Mira el valor cifrado en la consola de Firebase.
4. **Limpia los logs.** Busca los `Log.d` que muestran contenido y elimínalos.
5. **Divide el repositorio.** Saca el bloqueo de `RealtimeDatabaseRepository` a su propia clase sin romper las pruebas.
6. **Escribe una regla.** Prueba en el simulador de Firebase si un no miembro puede leer un chat.
7. **Mejora la cripto.** Escribe pruebas del ratchet: la misma cadena debe dar siempre las mismas claves, y un mensaje alterado debe fallar.
8. **Una herramienta para agentes.** Añade una herramienta simple en `AgentToolbox.kt` y descríbela en `AgentModels.kt`.

Glosario

Composable	Función que describe interfaz y se redibuja cuando cambia su estado
StateFlow	Valor observable que siempre tiene un estado actual
callbackFlow	Convierte una API de callbacks en un flujo de corrutinas
ViewModel	Guarda el estado de una pantalla y sobrevive a girarla
Hilt	Fabrica y entrega las dependencias de cada clase
Room	Base de datos SQLite local con comprobación al compilar
ECDH	Acordar un secreto común intercambiando solo claves públicas
AES-GCM	Cifrado rápido que además detecta alteraciones
HMAC / HKDF	Resumen con clave / derivación de claves a partir de un secreto
Secreto hacia adelante	Robar la clave de hoy no descifra los mensajes de ayer
SDP / ICE	Descripción de una llamada / búsqueda de la ruta de red
STUN / TURN	Descubre tu dirección pública / retransmite si no hay ruta directa
FCM	Servicio de Google para despertar el teléfono con un aviso
R8	Reduce y ofusca el APK de release
ReAct	Patrón de agente: razonar, actuar y observar en bucle

Parte III

Referencia del código

Cada archivo y cada función de la app. La explicación sale de los comentarios del propio código; donde no hay, se indica y se describe por su nombre.

Parte 1

Chat

La conversación: pantalla, campo de escribir, burbujas y herramientas.

ChatComposer.kt

`ui/screens/chat/ChatComposer.kt · 1192 líneas`

Opciones del temporizador común del chat. */ private val TimerOptions = listOf(0L to "Desactivado", 30L to "30 segundos", 300L to "5 minutos", 3_600L to "1 hora", 28_800L to "8 horas", 86_400L to "1 día", 604_800L to "1 semana") private fun timerShortLabel(seconds: Long): String = TimerOptions.firstOrNull { it.first == seconds }?.second ?: "$seconds s" @Composable fun TypingIndicator() { val transition = rememberInfiniteTransition(label = "typing") Row(modifier = Modifier .clip(RoundedCornerShape(14.dp)) .background(NexusTokens.Color.BgElevated) .padding(horizontal = 14.dp, vertical = 11.dp), horizontalArrangement = Arrangement.spacedBy(4.dp)) { repeat(3) { index -> val alpha by transiti

`timerShortLabel(seconds)` interna	Sin comentario en el código. Por su nombre: timer short label.
`TypingIndicator()` pantalla	Dibuja en pantalla: typing indicator.
`ReplyPreviewBar(message, label, onCancel)` pantalla	Dibuja en pantalla: reply preview bar.
`EphemeralTimerSheet(currentSeconds, viewOnceNext, onSelectTimer)` pantalla	Temporizador del chat y envío de una sola vista.
`ChatInputArea(messageText, onMessageChange)` pantalla	Dibuja en pantalla: chat input area.
`startRecording()`	Sin comentario en el código. Por su nombre: start recording.
`launchCamera()`	Sin comentario en el código. Por su nombre: launch camera.
`sendLastLocation()`	Sin comentario en el código. Por su nombre: send last location.
`AttachmentTile(icon, label, modifier, onClick)` pantalla interna	Dibuja en pantalla: attachment tile.
`GifPickerDialog(onDismiss)` pantalla interna	Selector de GIF: búsqueda con GIPHY si hay clave y, siempre, elegir uno del teléfono.
`gifSearchService()`	Sin comentario en el código. Por su nombre: gif search service.
`RecordingBar(onCancel)` pantalla interna	Barra de grabación de audio: el punto y el contador indican que está grabando.
`MediaCaptionDialog(uri, isVideo, onDismiss)` pantalla interna	Vista previa antes de enviar una foto o un vídeo, con un texto opcional debajo.

ChatExtras.kt

`ui/screens/chat/ChatExtras.kt · 506 líneas`

Una encuesta viaja como un mensaje de texto normal, así que va cifrada como cualquier otro. Este carácter invisible al principio distingue el contenido de un texto que parezca JSON. / private const val POLL_MARK = "poll:" data class Poll(val question: String, val options: List<String>) fun encodePoll(poll: Poll): String = POLL_MARK + JSONObject().put("q", poll.question).put("o", JSONArray(poll.options)).toString() fun decodePoll(text: String): Poll? { if (!text.startsWith(POLL_MARK)) return null return runCatching { val json = JSONObject(text.removePrefix(POLL_MARK)) val options = json.getJSONArray("o") Poll(json.getString("q"), List(options.length()) { options.getString(it) }) }.getOrNull(

`encodePoll(poll)`	Sin comentario en el código. Por su nombre: encode poll.
`decodePoll(text)`	Sin comentario en el código. Por su nombre: decode poll.
`pollPreview(text)`	Texto para la lista de chats y los avisos, en vez del contenido interno de la encuesta.
`PollCard(poll, chatId, messageId)` pantalla	Encuesta dentro de un mensaje. Los votos son públicos para los del chat y se ven en vivo.

`PollCreateDialog(onDismiss)` pantalla	Dibuja en pantalla: poll create dialog.
`databaseRepository()`	Sin comentario en el código. Por su nombre: database repository.
`doWork()` asíncrona	Sin comentario en el código. Por su nombre: do work.
`schedule(context, chatId, text, atMillis)`	Sin comentario en el código. Por su nombre: schedule.
`ScheduleMessageDialog(text, chatId, onDismiss)` pantalla	Dibuja en pantalla: schedule message dialog.
`at(hour, nextDay)`	Sin comentario en el código. Por su nombre: at.
`rememberPinnedMessageId(chatId)` pantalla	Devuelve el id del mensaje fijado de un chat y lo mantiene al día.
`setPinnedMessage(chatId, messageId)`	Sin comentario en el código. Por su nombre: set pinned message.
`PinnedMessageBanner(preview, onOpen)` pantalla	Dibuja en pantalla: pinned message banner.
`SafetyNumberDialog(peerId, onDismiss)` pantalla	Comparar este número en persona o por otra vía confirma que nadie se ha colado en medio. Es el mismo en los dos teléfonos: se calcula sobre las dos claves públicas, en orden fijo.
`load(peerId)`	Sin comentario en el código. Por su nombre: load.
`observe()` interna	Sin comentario en el código. Por su nombre: observe.
`save(text)`	Se cifra para uno mismo: el servidor guarda la nota sin poder leerla.
`delete(id)`	Sin comentario en el código. Por su nombre: delete.
`SavedMessagesScreen(navController, viewModel)` pantalla	Dibuja en pantalla: saved messages screen.

ChatScreen.kt

`ui/screens/chat/ChatScreen.kt · 1336 líneas`

Frase completa de última conexión para la barra del chat. Un reloj adelantado en el teléfono daría diferencias negativas ("hace -3 min"), por eso ese caso va aparte.

`ChatScreen(contactId, navController, viewModel)` pantalla	Dibuja en pantalla: chat screen.
`senderNameOf(uid)`	Sin comentario en el código. Por su nombre: sender name of.
`formatLastSeen(timestamp)` interna	Frase completa de última conexión para la barra del chat. Un reloj adelantado en el teléfono daría diferencias negativas ("hace -3 min"), por eso ese caso va aparte.
`ChatTopBar(contact, groupSubtitle, isTyping, onBackClick)` pantalla	Dibuja en pantalla: chat top bar.
`exportChat(context, messages, myUid, nameOf)` interna	Guarda la conversación como texto y abre el menú de compartir. El archivo sale del teléfono solo si tú lo envías.
`ReportUserDialog(onDismiss)` pantalla interna	Motivo del reporte. No envía mensajes del chat: solo quién, a quién y por qué.

ChatTools.kt

`ui/screens/chat/ChatTools.kt · 575 líneas`

Separador invisible: lo que va detrás no se muestra como texto del mensaje. */ private const val LINK_MARK = "link:" data class LinkPreview(val url: String, val title: String, val description: String) { val domain: String get() = runCatching { Uri.parse(url).host.orEmpty().removePrefix("www.") }.getOrDefault("") } private val URL_PATTERN = Regex("""https://^\s<>"'+""", RegexOption.IGNORE_CASE) fun firstUrl(text: String): String? = URL_PATTERN.find(text)?.value?.trimEnd('.', ',', ')', ']', '!', '?', ';', ':') fun appendLinkPreview(text: String, preview: LinkPreview): String = text + LINK_MARK + JSONObject() .put("u", preview.url) .put("t", preview.title) .put("d", preview.description) .toStr

`firstUrl(text)`	Sin comentario en el código. Por su nombre: first url.
`appendLinkPreview(text, preview)`	Sin comentario en el código. Por su nombre: append link preview.
`withoutLinkPreview(text)`	El texto del mensaje sin la vista previa.
`decodeLinkPreview(text)`	Sin comentario en el código. Por su nombre: decode link preview.
`parseLinkPreview(url, html)`	Saca título y descripción del HTML de la página.
`meta(property)`	Sin comentario en el código. Por su nombre: meta.
`decodeEntities(text)` interna	Sin comentario en el código. Por su nombre: decode entities.
`attachTo(text)` asíncrona	Sin comentario en el código. Por su nombre: attach to.
`fetch(url)` interna	Sin comentario en el código. Por su nombre: fetch.
`LinkPreviewCard(preview)` pantalla	Dibuja en pantalla: link preview card.
`encodeContact(contact)`	Sin comentario en el código. Por su nombre: encode contact.
`decodeContact(text)`	Sin comentario en el código. Por su nombre: decode contact.
`ContactCard(contact)` pantalla	Dibuja en pantalla: contact card.
`load()`	Sin comentario en el código. Por su nombre: load.
`forward(messages, target, onDone)`	Se cifra de nuevo para cada destino: es un envío normal, no una copia de lo que había en el servidor.
`TargetList(title, targets, onDismiss)` pantalla interna	Dibuja en pantalla: target list.
`ForwardDialog(messages, onDismiss)` pantalla	Dibuja en pantalla: forward dialog.
`ContactPickerDialog(onDismiss)` pantalla	Dibuja en pantalla: contact picker dialog.
`SelectionTopBar(count, onClose)` pantalla	Dibuja en pantalla: selection top bar.
`prefs(context)` interna	Sin comentario en el código. Por su nombre: prefs.
`read(context)` interna	Sin comentario en el código. Por su nombre: read.
`write(context, list)` interna	Sin comentario en el código. Por su nombre: write.
`add(context, entry)`	Sin comentario en el código. Por su nombre: add.
`remove(context, workId)`	Sin comentario en el código. Por su nombre: remove.
`forChat(context, chatId)`	Sin comentario en el código. Por su nombre: for chat.
`cancel(context, entry)`	Sin comentario en el código. Por su nombre: cancel.
`ScheduledMessagesDialog(chatId, onDismiss)` pantalla	Dibuja en pantalla: scheduled messages dialog.
`highlightMentions(text, color, onMentionClick)`	Colorea las menciones; el resto del texto queda igual.
`messagePreviewText(message)`	Texto corto para citar un mensaje: nunca muestra enlaces internos ni marcas.
`currentMentionQuery(text)`	Lo que va escrito tras la última @, si se está escribiendo una mención; si no, null.
`applyMention(text, name)`	Cambia la mención a medias por el nombre elegido.
`hideInternalLinks(content, mediaType)`	Quita del texto visible los enlaces internos de Storage y de la base de datos. Los stickers y algunos mensajes antiguos guardaban la URL del archivo como texto, y la conversación la mostraba tal cual: dejaba a la vista la ruta del servidor y su token. El archivo se sigue abriendo desde `mediaUrl`; solo se oculta el texto.
`hasSpoiler(text)`	Si el texto tiene algún `||spoiler||`.
`styledMessageText(raw, mentionColor, spoilerColor, revealed, onReveal)`	Texto con menciones y spoilers. `||así||` queda tapado por una barra hasta que se toca; al tocarlo se llama a onReveal. Las marcas `||` nunca se muestran.

ChatViewModel.kt

`ui/screens/chat/ChatViewModel.kt · 1719 líneas`

Identidad de un mensaje dentro de la lista: es a la vez la clave de la `LazyColumn` y lo que se usa para quitar repetidos. Al ser una sola definición, una lista pasada por mergeMessages no puede provocar el crash "Key was already used".

`listKey()`	Identidad de un mensaje dentro de la lista: es a la vez la clave de la `LazyColumn` y lo que se usa para quitar repetidos. Al ser una sola definición, una lista pasada por mergeMessages no puede provocar el crash "Key was already used".
`mergeMessages(live, older)`	Une lo que llega en vivo con las páginas antiguas, sin repetidos y en orden. live va primero porque su copia es la actual (estado de lectura, reacciones) y es la que debe ganar.
`toggleEphemeralPicker()`	Abre o cierra la hoja del temporizador.
`dismissEphemeralPicker()`	Sin comentario en el código. Por su nombre: dismiss ephemeral picker.
`setViewOnceNext(enabled)`	Marca el siguiente envío como "una sola vista".
`setChatTimer(seconds)`	Cambia el temporizador del chat. Es compartido, como en Signal: lo que elige uno vale para los dos y queda un aviso en la conversación para que nadie lo ignore.
`startEphemeralCleanup(chatId)` interna	Vigila el chat abierto: refleja el temporizador común y, cada segundo, retira los mensajes cuyo tiempo se ha acabado, en pantalla y en el servidor. La versión anterior leía el nodo `chats` completo, que las reglas de seguridad niegan a cualquier cliente, así que no borraba nada nunca.
`removeFromList(ids)` interna	Sin comentario en el código. Por su nombre: remove from list.
`deleteEphemeral(chatId, message)` interna asíncrona	Sin comentario en el código. Por su nombre: delete ephemeral.
`markMessageViewed(message)`	El destinatario ha visto un mensaje temporal: empieza su cuenta atrás.
`consumeViewOnce(message)`	Se cerró un mensaje de una sola vista: desaparece para todos.
`publishMedia(chatId, url, type, caption, extra)` interna asíncrona	Único camino de los archivos al chat: aplica el temporizador o la "una sola vista" a fotos, vídeos, GIF y audio igual que al texto. Antes estos envíos ignoraban el modo temporal, de modo que justo lo más delicado era lo que no desaparecía.
`sendGifMessage(uri, chatId)`	GIF desde el teléfono, tal cual para conservar la animación.
`sendAnimationMessage(uri, chatId)`	Vídeo corto que se repite sin sonido dentro del chat, como los GIF de Telegram.
`sendRemoteGif(gifUrl, chatId)`	GIF elegido en la búsqueda. Se descarga y se sube a nuestro almacenamiento: si el mensaje apuntara a GIPHY, el teléfono del destinatario le revelaría su IP.
`uploadAndSend(chatId, what, block)` interna	Sin comentario en el código. Por su nombre: upload and send.
`timerLabel(seconds)` interna	Número → texto del temporizador, p. ej. 3600 → "1 hora".
`loadChatBackground(chatId)`	Sin comentario en el código. Por su nombre: load chat background.
`isCanonicalForUser(rawId, currentUserId)` interna	Returns true when rawId already is a canonical chatId that includes currentUserId as one of its underscore-separated participants. Firebase UIDs never contain underscores, so boundary checks are safe.
`resolveCanonicalChatId(rawId, currentUserId)` interna	Normalizes a raw navigation argument into the canonical chatId used for storage. Group chats keep their id; an already-canonical id is returned as-is; a bare peer UID is combined with the current user (sorted) so both participants always resolve to the same chat node.
`derivePeerId(canonicalId, currentUserId)` interna	Derives the peer (other participant) UID from a canonical 1:1 chatId.
`loadGroupInfo(chatId)` interna asíncrona	Sin comentario en el código. Por su nombre: load group info.
`resolveContact(canonicalChatId, rawChatId, currentUserId)` interna asíncrona	Resolves the contact for a chat. Prefers the chat's `members` node (covers legacy / group structures) and falls back to deriving the peer from the canonical chatId, then fetches the real user profile (name + photo).
`defaultContactFor(uid, isDemoContact)` interna	Construye un User por defecto cuando Firebase no devuelve datos del contacto. Para el contacto demo usa los valores de Azel Assistant para que el Demo Chat nunca crashee por datos ausentes (Requisito 5.5).
`chatIdFor(passed)`	El identificador real del chat abierto.
`effectiveChatId(passed)` interna	Sin comentario en el código. Por su nombre: effective chat id.

`loadChat(rawChatId)`	Sin comentario en el código. Por su nombre: load chat.
`refreshChat()`	Sin comentario en el código. Por su nombre: refresh chat.
`mapMessage(data, chatId, peerId)` interna asíncrona	Convierte un mensaje de la base de datos en el modelo de la app. Es el único sitio donde se hace. Antes había dos copias y ninguna leía los campos de los mensajes temporales: el servidor los guardaba pero la app nunca se enteraba, y por eso no había cuenta atrás ni "una sola vista", ni desaparecía nada.
`serverNow()`	Sin comentario en el código. Por su nombre: server now.
`peerIdOf(chatId, currentUserId)` interna	UID del otro extremo de un chat privado, o cadena vacía si es un grupo. Los chats 1:1 se nombran uniendo los dos uids ordenados con `_` (ver com.Azelmods.App.data.chat.ChatId); los grupos usan un id push de Firebase, que no contiene `_`, así que devuelven vacío y quedan fuera del cifrado —igual que antes, porque el E2EE por pares no cubre grupos—.
`decryptedContent(rawContent, isEncrypted, payload, peerId)` interna asíncrona	Descifra el contenido de un mensaje E2EE. Se descifra con el uid del **otro extremo del chat**, no con el del remitente. El secreto ECDH es simétrico: la misma clave sirve para lo que llega y para lo que uno mismo escribió. Antes se saltaba el descifrado cuando el remitente era el usuario actual, así que el emisor veía " 🔒 Mensaje cifrado de extremo a extremo" —el marcador que se guarda en el servidor en lugar del tex
`parseMessageStatus(raw)` interna	Sin comentario en el código. Por su nombre: parse message status.
`setTypingStatus(chatId, isTyping)`	Write the current user's typing status to typing/{chatId}/{userId}. Debounced: only writes if the value actually changes. If isTyping is true, auto-clears after 5 s. Cancels previous debounce job on each call.
`observeTypingStatus(chatId)` interna	Observe typing/{chatId} and update ChatState.isTyping and ChatState.typingUserName whenever ANY other user is currently typing.
`sendMessage(rawContent, chatId)`	Sin comentario en el código. Por su nombre: send message.
`isDemoChat(chatId)` interna	`true` si el chat es el de bienvenida con Azel Assistant.
`replyAsDemoAssistant(userText, chatId)` interna	Hace que "Azel Assistant" responda al usuario **mensaje a mensaje** (no todo en un bloque). Escribe cada respuesta como el bot con una breve pausa entre ellas; la lista de mensajes se actualiza en tiempo real vía el listener de Firebase, así que el usuario ve llegar las respuestas de una en una, como en un chat real.
`setReplyingTo(message)`	Sin comentario en el código. Por su nombre: set replying to.
`addReaction(messageId, emoji)`	Add a reaction to a message. Uses currentChatId which is set by loadChat.
`sendImageMessage(imageUri, chatId, caption)`	Send an image message.
`sendAudioMessage(audioUri, chatId, voice)`	Send an audio message.
`sendVideoMessage(videoUri, chatId, caption)`	Send a video message.
`sendDocumentMessage(documentUri, chatId, fileName)`	Send a document message uploaded to Firebase Storage.
`sendLocationMessage(latitude, longitude, address, chatId)`	Send a location message.
`sendStickerMessage(sticker, chatId)`	Send a sticker message.
`clearError()`	Sin comentario en el código. Por su nombre: clear error.
`translateDraft(text, onTranslated)`	Traduce lo que estás escribiendo antes de enviarlo, para responder en otro idioma.
`translateMessage(messageId, text)`	Sin comentario en el código. Por su nombre: translate message.
`clearTranslationError()`	Sin comentario en el código. Por su nombre: clear translation error.
`clearTranslationNotice()`	Sin comentario en el código. Por su nombre: clear translation notice.
`observeAiFeatureFlags()` interna	Lee los interruptores y publica qué funciones debe mostrar la pantalla.
`currentTranscript(maxMessages)` interna	Guion de la conversación para los prompts. Sólo se etiqueta quién habla ("YO"/"OTRO"), sin nombres reales: el modelo no los necesita y no hay motivo para enviar identidades a un proveedor externo.
`refreshSmartReplies()`	Recalcula las tres respuestas sugeridas para el estado actual del chat.

`dismissSmartReplies()`	Sin comentario en el código. Por su nombre: dismiss smart replies.
`summarizeConversation()`	Resume la conversación y deja el texto en el estado para mostrarlo.
`dismissSummary()`	Sin comentario en el código. Por su nombre: dismiss summary.
`rewriteDraftTone(draft, tone, onResult)`	Reescribe draft en otro tono y devuelve el resultado por onResult para que la pantalla lo ponga en el campo de texto. El texto NO se envía: reescribir es una sugerencia, y decidir si se manda sigue siendo del usuario.
`autoTranslateIncoming()` interna	Traduce automáticamente los mensajes entrantes que aún no estén traducidos. Se limita a los últimos AUTO_TRANSLATE_WINDOW para no disparar una ráfaga de peticiones al abrir un chat con años de historial.
`openSearch()`	Sin comentario en el código. Por su nombre: open search.
`closeSearch()`	Sin comentario en el código. Por su nombre: close search.
`onSearchQueryChange(query)`	Sin comentario en el código. Por su nombre: on search query change.
`nextSearchResult()`	Salta al siguiente resultado, dando la vuelta al llegar al final.
`previousSearchResult()`	Sin comentario en el código. Por su nombre: previous search result.
`currentSearchMessageId()`	Id del mensaje que hay que resaltar y hacia el que desplazarse.
`loadMuteState(chatId)` interna	Lee el ajuste de silencio del chat abierto.
`reportarEnMain(mensaje, onResult)` interna asíncrona	Entrega el aviso al llamador SIEMPRE en el hilo principal. Mismo motivo que en `HomeViewModel`: estas funciones trabajan en Dispatchers.IO y la pantalla responde al callback con un `Toast`. Mostrar un Toast fuera del hilo principal lanza `RuntimeException: Can't toast on a thread that has not called Looper.prepare()`. En Home ya se corrigió, pero aquí se quedó sin alinear y por eso "Vaciar chat" cerraba la app desde
`toggleMute(durationMs, onResult)`	Silencia o reactiva la conversación abierta.
`clearChat(onResult)`	Vacía la conversación abierta: borra los mensajes en el servidor y en el caché local. La entrada "Clear Chat" del menú del chat existía desde hacía versiones pero su `onClick` sólo cerraba el menú, así que pulsarla no borraba nada.
`deleteMessage(message, forEveryone)`	Sin comentario en el código. Por su nombre: delete message.
`editMessage(messageId, newContent)`	Edit a message (REAL implementation)
`setEditingMessage(message)`	Set message to edit
`loadMoreMessages()`	🆕 Load MORE older messages (pagination — scroll-up). Fetches 30 messages before the earliest known timestamp and prepends them.

MediaGalleryScreen.kt

`ui/screens/chat/MediaGalleryScreen.kt · 452 líneas`

Media gallery screen showing all photos/videos in a chat. Features: - Grid layout of media items - Tap to open full screen viewer - Pinch-to-zoom support - Swipe between media items

`MediaGalleryScreen(chatId, navController, messages)` pantalla	Media gallery screen showing all photos/videos in a chat. Features: - Grid layout of media items - Tap to open full screen viewer - Pinch-to-zoom support - Swipe between media items
`MediaGridItem(message, onClick)` pantalla interna	Dibuja en pantalla: media grid item.
`FullScreenMediaViewer(messages, initialIndex, onDismiss)` pantalla interna	Dibuja en pantalla: full screen media viewer.
`ZoomableImage(imageUrl, onDismiss)` pantalla interna	Dibuja en pantalla: zoomable image.
`FullscreenVideoPlayer(videoUrl, onDismiss)` pantalla interna	Fullscreen video playback with pinch-to-zoom, using Media3 ExoPlayer. Mirrors the inline player in MessageBubble but adds transformable zoom/pan.

QuickFunctions.kt

`ui/screens/chat/QuickFunctions.kt · 218 líneas`

Cuenta los mensajes por tipo. Solo los cargados en este chat.

`countMessages(messages)`	Cuenta los mensajes por tipo. Solo los cargados en este chat.
`QuickFunctionsButton(onClick)` pantalla	Botón flotante del chat: abre el panel de funciones rápidas.
`QuickFunctionsSheet(chatId, contact, messages, navController, onSelectAll)` pantalla	Dibuja en pantalla: quick functions sheet.
`openWeb(url)`	Sin comentario en el código. Por su nombre: open web.
`go(route)`	Sin comentario en el código. Por su nombre: go.
`ShortcutTile(icon, label, onClick)` pantalla interna	Dibuja en pantalla: shortcut tile.
`QuickRow(icon, label, onClick)` pantalla interna	Dibuja en pantalla: quick row.

Parte 2

Lista de chats

Pantalla principal con la lista, filtros, carpetas y búsqueda.

HomeHeaders.kt

`ui/screens/home/HomeHeaders.kt · 335 líneas`

El título de la barra superior según el diseño de inicio elegido.

`HomeTitle(style, me, showAvatar, onOpenProfile)` pantalla	El título de la barra superior según el diseño de inicio elegido.
`HomeHeader(style, me, ghost, anonymous, onOpenProfile)` pantalla	Lo que el diseño elegido coloca entre la barra superior y la lista.
`ProfileCard(me, ghost, anonymous, onClick)` pantalla interna	Dibuja en pantalla: profile card.
`CoverBanner(me, onClick)` pantalla interna	Dibuja en pantalla: cover banner.
`NamePanel(me)` pantalla interna	Panel con las esquinas de arriba redondeadas y una píldora con tu nombre.
`ArcHeader(me, onClick)` pantalla interna	Una banda con el borde de arriba curvado hacia abajo y tu foto en el centro, sobre la curva.
`TabStrip(me, onOpenContacts)` pantalla interna	Barra de 50 dp sin textos: los iconos a los lados y tu foto en el centro.
`StripAction(icon, label, onClick)` pantalla interna	Dibuja en pantalla: strip action.

HomeScreenRedesigned.kt

`ui/screens/home/HomeScreenRedesigned.kt · 1114 líneas`

Fila de conversación: plana, a todo el ancho, con el acento solo en lo que está sin leer. */ @OptIn(ExperimentalFoundationApi::class) /** Una conversación de la lista, con el estilo de fila que haya elegido el usuario.

`HomeScreenRedesigned(navController, viewModel)` pantalla	Dibuja en pantalla: home screen redesigned.
`toast(message)`	Sin comentario en el código. Por su nombre: toast.
`HomeMenuItem(label, icon, onClick)` pantalla interna	Dibuja en pantalla: home menu item.
`ChatRow(chat, onClick)` pantalla	Una conversación de la lista, con el estilo de fila que haya elegido el usuario.
`ChatRowBody(chat, onClick)` pantalla interna	Dibuja en pantalla: chat row body.
`TypingDotsSmall()` pantalla	Indicador de escritura: animación con sentido, comunica un estado en curso.
`EmptyChatsState(filter, onStartConversation)` pantalla	Dibuja en pantalla: empty chats state.
`ChatActionItem(icon, text, textColor, onClick)` pantalla	Dibuja en pantalla: chat action item.
`formatTimestamp(timestamp)` interna	Hora en la lista de chats: hoy la hora, ayer "Ayer", esta semana el día, y si no la fecha. Es lo que el usuario espera de una app de mensajería, en vez de "3h" o "12d".
`confirmDeviceOwner(context, title, onNoLock)` interna	Pide la huella o el bloqueo del teléfono. Sin bloqueo puesto no hay nada que proteja el chat.
`onAuthenticationSucceeded(result)`	Sin comentario en el código. Por su nombre: on authentication succeeded.
`ChatFoldersDialog(chatId, folders, onDismiss)` pantalla interna	Dibuja en pantalla: chat folders dialog.

`HomeFilter(asText, label, selected, count, onClick)` pantalla interna	Un filtro de la lista: como cápsula o como pestaña de texto con un punto de color.
`ArchivedRow(count, onClick)` pantalla interna	Fila de Archivados: icono dentro de un aro de 1,5 dp y el estilo de fila elegido.

HomeViewModel.kt

`ui/screens/home/HomeViewModel.kt · 696 líneas`

Global uid → displayName lookup aggregated across all loaded chats. */ val participantNames: Map<String, String> = emptyMap(), /** Global uid → photoUrl lookup aggregated across all loaded chats. */ val participantPhotos: Map<String, String> = emptyMap()) enum class ChatFilter { ALL, UNREAD, GROUPS, ARCHIVED } // ── ViewModel ── /** Rótulo que deja en el servidor un mensaje cifrado (ver sendMessageInternal). */ private const val ENCRYPTED_PREVIEW_LABEL = "Mensaje cifrado" private const val USER_CACHE_MS = 60_000L @HiltViewModel class HomeViewModel @Inject constructor(private val databaseRepository: RealtimeDatabaseRepository, p

`toggleGhostMode()`	Sin comentario en el código. Por su nombre: toggle ghost mode.
`lastMessagePreview(data, peerUid)` interna asíncrona	Texto de la vista previa. Si el último mensaje va cifrado, en el servidor solo hay un rótulo y el texto cifrado: se descifra aquí, en el teléfono, como el propio chat.
`cachedUser(uid)` interna asíncrona	Sin comentario en el código. Por su nombre: cached user.
`refreshChats()`	Pull-to-refresh or manual retry entry point.
`onSearchQueryChange(query)`	Sin comentario en el código. Por su nombre: on search query change.
`onFilterChange(filter)`	Sin comentario en el código. Por su nombre: on filter change.
`markUnread(chatId)`	Como en WhatsApp: el chat vuelve a aparecer pendiente hasta que se abra otra vez.
`togglePin(chatId)`	Sin comentario en el código. Por su nombre: toggle pin.
`toggleMute(chatId, durationMs, onResult)`	Silencia el chat durante durationMs, o lo reactiva si ya estaba silenciado. para que no caduque.
`reportarEnMain(mensaje, onResult)` interna asíncrona	Entrega el aviso al llamador SIEMPRE en el hilo principal. Estas funciones trabajan en Dispatchers.IO y la pantalla responde al callback con un `Toast`. Mostrar un Toast fuera del hilo principal lanza `RuntimeException: Can't toast on a thread that has not called Looper.prepare()`, así que llamar al callback directamente desde IO hacía que fijar, archivar, silenciar, vaciar y borrar cerraran la app. toggleBlock ya lo
`toggleBlock(chatId, onResult)`	Bloquea o desbloquea al otro miembro de un chat individual. Quien queda bloqueado deja de poder escribir en la conversacion, y eso lo impone la regla de Firebase, no la interfaz. En grupos no aplica: no hay un "otro" unico a quien bloquear.
`toggleArchive(chatId, onResult)`	Archiva o desarchiva. Antes solo sabía archivar (`isArchived = true` fijo), así que una conversación archivada por error solo se recuperaba editando la base de datos a mano, pese a que el filtro "Archivados" ya existía en esta misma pantalla.
`observeChatSettings()` interna	Escucha los ajustes personales y los reaplica sobre los chats cargados.
`deleteChat(chatId, onResult)`	Borra la conversación en el servidor y en el caché local. El `runCatching` de antes se tragaba el fallo sin dejar rastro: si las reglas denegaban el borrado, el usuario veía el chat intacto y no había forma de saber por qué. Ahora un fallo se informa por onResult y queda en el log.
`clearChat(chatId, onResult)`	Vacía los mensajes conservando la conversación.
`loadChats()` interna	Sin comentario en el código. Por su nombre: load chats.
`buildChatFromMap(data, currentUserId)` interna asíncrona	Constructs an enriched Chat from a raw Firebase map. The "other" participant (the one that is NOT currentUserId) is fetched via RealtimeDatabaseRepository.getUserById and their display-name / photo URL are injected into the Chat object's participant maps. For group chats the first non-self member is used as the preview contact.
`filterChats(chats, query, filter)` interna	Sin comentario en el código. Por su nombre: filter chats.
`onCleared()`	Sin comentario en el código. Por su nombre: on cleared.

SearchScreen.kt

`ui/screens/home/SearchScreen.kt · 541 líneas`

`SearchScreen(navController, viewModel)` pantalla	Dibuja en pantalla: search screen.
`ChatSearchResultItem(result, onClick)` pantalla	Dibuja en pantalla: chat search result item.
`ChatSearchResultItemBody(result, onClick)` pantalla interna	Dibuja en pantalla: chat search result item body.
`ContactSearchResultItem(result, onClick)` pantalla	Dibuja en pantalla: contact search result item.
`ContactSearchResultItemBody(result, onClick)` pantalla interna	Dibuja en pantalla: contact search result item body.
`MessageSearchResultItem(result, onClick)` pantalla	Dibuja en pantalla: message search result item.
`users()` interna asíncrona	Sin comentario en el código. Por su nombre: users.
`search(query)`	Sin comentario en el código. Por su nombre: search.

Parte 3

Navegación principal

Pestañas y barras inferiores.

MainScreen.kt

`ui/screens/main/MainScreen.kt · 748 líneas`

La barra inferior elegida. La usan la pantalla principal y la vista previa del estudio.

`MainScreen(navController, appBackgroundManager)` pantalla	Dibuja en pantalla: main screen.
`AppBottomBar(navStyle, tabs, currentPage, missedCallCount, onSelect)` pantalla	La barra inferior elegida. La usan la pantalla principal y la vista previa del estudio.
`CenterNavBar(tabs, currentPage, missedCalls, onSelect)` pantalla interna	Aro central: una cápsula de texto a cada lado (36 dp, esquinas de 5 dp, negrita de 13 sp) y, en una caja central de 96 x 76 dp, el botón de chats dentro de un aro de seis segmentos que gira cada 2 s.
`CenterNavPill(tab, selected, missedCalls, onClick)` pantalla interna	Dibuja en pantalla: center nav pill.
`AccessNavBar(tabs, currentPage, missedCalls, onSelect)` pantalla interna	Accesos: barra de 56 dp con iconos de 23 dp, ajustes incluidos, y un botón redondo de 41 dp al final.
`tabsFor(set)`	Las pestañas con el juego de iconos elegido: icono de la pestaña activa e inactiva.
`NavTabIcon(tab, selected, missedCalls, tint, size)` pantalla interna	Icono de pestaña con el aviso de llamadas perdidas.
`PhotoNavBar(tabs, currentPage, missedCalls, onSelect)` pantalla interna	Barra de 50 dp sobre superficie, con iconos de 22 dp siempre en el color de acento y, en el centro, tu foto de 34 dp dentro de un aro de 41 dp y 2 dp de grosor, 3 dp por encima del borde inferior.
`cell(index, modifier)` pantalla	Dibuja en pantalla: cell.
`LineNavBar(tabs, currentPage, missedCalls, onSelect)` pantalla interna	Pestañas de texto de 48 dp con una línea de 4 dp bajo la elegida.
`PanelNavBar(tabs, currentPage, missedCalls, onSelect)` pantalla interna	Panel con las esquinas superiores de 20 dp, pestañas de texto e indicador de 4 dp.
`AppBackgroundLayer(config)` pantalla interna	Fondo elegido por el usuario. Sin uno propio, el fondo sólido de la app.

MainViewModel.kt

`ui/screens/main/MainViewModel.kt · 11 líneas`

Parte 4

Historias

Crear, ver y responder historias.

CreateStoryScreen.kt

`ui/screens/stories/CreateStoryScreen.kt · 940 líneas`

Cómo entra la foto en el marco antes de que el usuario la mueva.

`parseStoryColor(hex)` interna	Sin comentario en el código. Por su nombre: parse story color.
`CreateStoryScreen(navController, viewModel)` pantalla	Dibuja en pantalla: create story screen.
`resetFraming()`	Sin comentario en el código. Por su nombre: reset framing.
`openMedia(uri)`	Sin comentario en el código. Por su nombre: open media.
`launchCamera()`	Sin comentario en el código. Por su nombre: launch camera.
`discardAndChooseAgain()`	Sin comentario en el código. Por su nombre: discard and choose again.
`publish()`	Sin comentario en el código. Por su nombre: publish.
`recordInto(layer)` interna	Dibuja el contenido normalmente y, además, lo guarda en layer para poder exportarlo.
`StoryVideoPreview(uri)` pantalla interna	Dibuja en pantalla: story video preview.
`StrokesLayer(strokes)` pantalla interna	Dibuja en pantalla: strokes layer.
`DrawingSurface(color, onStrokeFinished)` pantalla interna	Dibuja en pantalla: drawing surface.
`norm(o)`	Sin comentario en el código. Por su nombre: norm.
`OverlayView(item, enabled, onChange)` pantalla interna	Texto o emoji: se arrastra, se pellizca, se gira. Tocar edita; mantener pulsado elimina.
`TextOverlayDialog(initial, onDismiss)` pantalla interna	Dibuja en pantalla: text overlay dialog.
`ColorDot(color, selected, size, onClick)` pantalla interna	Dibuja en pantalla: color dot.
`StoryTool(icon, label, selected, onClick)` pantalla interna	Dibuja en pantalla: story tool.

CreateStoryViewModel.kt

`ui/screens/stories/CreateStoryViewModel.kt · 397 líneas`

Porcentaje de la subida, o null mientras no se sabe (al preparar el vídeo). */ val uploadProgress: Int? = null, val uploadSuccess: Boolean = false, val error: String? = null) @HiltViewModel class CreateStoryViewModel @Inject constructor(@ApplicationContext private val context: Context, private val storageRepository: StorageRepository, private val databaseRepository: RealtimeDatabaseRepository, private val userRepository: com.Azelmods.App.data.repository.UserRepository, private val auth: FirebaseAuth) : ViewModel() { private val _state = MutableStateFlow(CreateStoryState()) val state: StateFlow<CreateStoryState> = _state.asStateFlow() /** Sube la pista de audio elegida, si hay alguna, y de

`uploadMusicOrNull(musicUri, userId)` interna asíncrona	Sube la pista de audio elegida, si hay alguna, y devuelve su URL de descarga. Devuelve null si falla en vez de propagar la excepción: quedarse sin música es un defecto menor, perder la historia entera por eso no es aceptable. Esa es la misma regla que ya sigue el resto de este ViewModel con las composiciones.

`uploadComposedImageStory(composedBitmap, caption, originalUri, musicUri, musicName)`	Upload a COMPOSED IMAGE story: the editor captures its editable area (photo + text / stickers / emojis) into composedBitmap using a Compose GraphicsLayer, so the overlays are already burned into the pixels. We persist that bitmap to a temp file and upload it instead of the original media. If anything goes wrong while saving the composed bitmap we fall back to uploading originalUri so the user never ends up unable to
`uploadImageStory(imageUri, caption, musicUri, musicName)`	Upload IMAGE story to Firebase Storage and save to Realtime Database Pattern: stories/{userId}/{timestamp}.jpg
`uploadAnimatedStory(imageUri, caption, musicUri, musicName)`	Publica una imagen animada (GIF) tal cual, conservando su animación: pasarla por la captura del editor la convertiría en un único fotograma.
`uploadComposedVideoStory(overlayBitmap, caption, originalUri, musicUri, musicName)`	Upload a COMPOSED VIDEO story: overlayBitmap is the editor's editable area captured into a transparent bitmap (text / stickers / emojis already rasterized, transparent where the video shows through). We burn it into every video frame with StoryVideoComposer and upload the resulting MP4 instead of the original clip. If compositing fails for any reason we fall back to uploading originalUri so the user can always publis
`uploadVideoStory(videoUri, caption, musicUri, musicName)`	Upload VIDEO story to Firebase Storage and save to Realtime Database Pattern: stories/{userId}/{timestamp}.mp4
`createTextStory(text, backgroundColor, musicUri, musicName)`	Create TEXT story (no upload needed, direct to database)
`clearError()`	Sin comentario en el código. Por su nombre: clear error.
`clearSuccess()`	Sin comentario en el código. Por su nombre: clear success.

StoriesScreen.kt

`ui/screens/stories/StoriesScreen.kt · 351 líneas`

Historias en forma de lista: tu historia arriba y, debajo, las de tus contactos separadas en recientes y vistas. Cada anillo dice una sola cosa: si queda algo por ver.

`StoriesScreen(navController, viewModel)` pantalla	Historias en forma de lista: tu historia arriba y, debajo, las de tus contactos separadas en recientes y vistas. Cada anillo dice una sola cosa: si queda algo por ver.
`MyStoryRow(photoUrl, hasStory, onClick)` pantalla interna	Dibuja en pantalla: my story row.
`MyStoryRowBody(photoUrl, hasStory, onClick)` pantalla interna	Dibuja en pantalla: my story row body.
`StoryRow(story, onClick)` pantalla interna	Dibuja en pantalla: story row.
`StoryRowBody(story, onClick)` pantalla interna	Dibuja en pantalla: story row body.
`StoryAvatar(name, photoUrl, ringColor)` pantalla interna	Dibuja en pantalla: story avatar.
`formatStoryTime(timestamp)` interna	Sin comentario en el código. Por su nombre: format story time.
`StoryCard(story, onClick)` pantalla interna	Historia como tarjeta vertical: su imagen de fondo, con la foto y el nombre encima.

StoriesViewModel.kt

`ui/screens/stories/StoriesViewModel.kt · 188 líneas`

`loadStories()` interna	Sin comentario en el código. Por su nombre: load stories.
`clearError()`	Sin comentario en el código. Por su nombre: clear error.

StoryViewerScreen.kt

`ui/screens/stories/StoryViewerScreen.kt · 1189 líneas`

`StoryViewerScreen(navController, userId, viewModel)` pantalla	Dibuja en pantalla: story viewer screen.
`onPlaybackStateChanged(playbackState)`	Sin comentario en el código. Por su nombre: on playback state changed.
`onPlayerError(error)`	Sin comentario en el código. Por su nombre: on player error.
`QuickReactionRow(selectedEmoji, onReact)` pantalla interna	Dibuja en pantalla: quick reaction row.
`getTimeAgo(timestamp)`	Sin comentario en el código. Por su nombre: get time ago.

StoryViewerViewModel.kt

`ui/screens/stories/StoryViewerViewModel.kt · 276 líneas`

No se pudieron cargar las historias: es lo único que sustituye al visor. */ val error: String? = null, /** Falló una acción (responder, reaccionar, borrar): se avisa sin cerrar la historia. */ val actionError: String? = null, val replySent: Boolean = false, val viewers: List<Map<String, Any>> = emptyList(), val deleted: Boolean = false, /** Emoji recién enviado (feedback en snackbar); null cuando no hay pendiente. */ val reactionSent: String? = null, /** Reacciones propias de esta sesión: storyId → emoji (resalta el elegido). */ val myReactions: Map<String, String> = emptyMap(), /** Reacciones de la historia propia abierta en el sheet: uid → emoji. */ val reactions: Map<String, String> = emp

`loadStoriesForUser(userId)`	Sin comentario en el código. Por su nombre: load stories for user.
`friendlyMessage(e, fallback)` interna	El texto crudo de Firebase ("Permission denied") no le dice nada a quien usa la app.
`clearActionError()`	Sin comentario en el código. Por su nombre: clear action error.
`nextStory()`	Sin comentario en el código. Por su nombre: next story.
`previousStory()`	Sin comentario en el código. Por su nombre: previous story.
`sendReply(storyOwnerId, storyId, text)`	Sin comentario en el código. Por su nombre: send reply.
`sendStickerReply(storyOwnerId, storyId, sticker)`	Sin comentario en el código. Por su nombre: send sticker reply.
`clearReplySent()`	Sin comentario en el código. Por su nombre: clear reply sent.
`addReaction(storyOwnerId, storyId, emoji)`	Sin comentario en el código. Por su nombre: add reaction.
`clearReactionSent()`	Sin comentario en el código. Por su nombre: clear reaction sent.
`loadViewers(storyId)`	Sin comentario en el código. Por su nombre: load viewers.
`deleteStory(story)`	Elimina la historia indicada (solo si es del usuario actual; el repo lo valida y las reglas también). Actualiza la lista local de inmediato y marca `deleted = true` para que la pantalla cierre el visor.
`clearDeleted()`	Sin comentario en el código. Por su nombre: clear deleted.

Parte 5

Llamadas

Pantalla de llamada e historial.

ActiveCallScreen.kt

`ui/screens/call/ActiveCallScreen.kt · 644 líneas`

Pantalla de llamada en curso (emisor y receptor). de la llamada en Firebase.

`ActiveCallScreen(contactId, callType, navController, isCaller, viewModel)` pantalla	Pantalla de llamada en curso (emisor y receptor). de la llamada en Firebase.
`CallPermissionGate(isVideoCall, onGrant)` pantalla interna	Dibuja en pantalla: call permission gate.
`CallerHero(name, photoUrl, statusText, pulsing, isVideoCall, colors)` pantalla interna	Avatar grande con anillos de pulso mientras suena o conecta.
`SecurityBadge(icon, text)` pantalla interna	Dibuja en pantalla: security badge.
`CallControlsPanel(isVideoCall, isAudioEnabled, isVideoEnabled, isSpeakerOn, enabled, onToggleMic)` pantalla interna	Dibuja en pantalla: call controls panel.
`CallControlButton(icon, label, isActive, enabled, onClick)` pantalla	Dibuja en pantalla: call control button.
`ProximityScreenOff(enabled)` pantalla interna	Bloqueo de proximidad: apaga la pantalla cuando el sensor detecta la oreja, como la app de teléfono. Se suelta en cuanto enabled pasa a false.
`formatDuration(seconds)` interna	Sin comentario en el código. Por su nombre: format duration.

CallViewModel.kt

`ui/screens/call/CallViewModel.kt · 661 líneas`

En qué punto está la llamada, para que la interfaz diga algo honesto.

`myUid()` interna	Sin comentario en el código. Por su nombre: my uid.
`setupWebRTCCallbacks()` interna	Registra los callbacks de señalización en el WebRTCManager. NO se llama desde `init`. El WebRTCManager es un `@Singleton` con un único juego de listeners, y cada pantalla de llamada crea su propio `CallViewModel`: si todos los registraran al construirse, el ViewModel de la pantalla de llamada entrante —que sólo observa— pisaría los del ViewModel que sí conduce la conexión, y la oferta, la respuesta y los candidatos I
`loadContactProfileFromCall(callId)`	Carga el perfil del otro participante a partir del id de la llamada y guarda nombre/foto para la notificación de llamada perdida.
`resolveContactFromCall(callId)` interna asíncrona	Sin comentario en el código. Por su nombre: resolve contact from call.
`toUser(fallbackId)` interna	Sin comentario en el código. Por su nombre: to user.
`displayNameOf(data)` interna	Sin comentario en el código. Por su nombre: display name of.
`startCall(contactId, callType)`	Sin comentario en el código. Por su nombre: start call.
`startRingingTimeout()` interna	Cuelga si nadie contesta. Sin esto, una llamada a alguien con el teléfono apagado dejaba al emisor sonando indefinidamente con el micrófono abierto.
`acceptCall(callId, callType)`	Sin comentario en el código. Por su nombre: accept call.
`observeIncomingCall(callId)`	La pantalla de llamada entrante escucha la llamada ANTES de contestar para cerrarse (y avisar de la perdida) si quien llama cuelga.
`declineCall(callId)`	Sin comentario en el código. Por su nombre: decline call.

`preparePeerConnection(callId)` interna asíncrona	Sin comentario en el código. Por su nombre: prepare peer connection.
`observeConnectionState()` interna	Sin comentario en el código. Por su nombre: observe connection state.
`scheduleConnectionLoss()` interna	Sin comentario en el código. Por su nombre: schedule connection loss.
`listenToCallUpdates(callId)` interna	Escucha el nodo de la llamada (señalización + estado) y sus candidatos ICE.
`handleSignaling(data)` interna	Sin comentario en el código. Por su nombre: handle signaling.
`handleStatus(callId, data)` interna	Sin comentario en el código. Por su nombre: handle status.
`endCall(reason)`	Termina la llamada. Es idempotente: la pueden pedir el botón, el otro extremo, el temporizador o la pérdida de conexión, y solo cuenta la primera.
`releaseMedia()` interna	Sin comentario en el código. Por su nombre: release media.
`toggleAudio()`	Sin comentario en el código. Por su nombre: toggle audio.
`toggleVideo()`	Sin comentario en el código. Por su nombre: toggle video.
`switchCamera()`	Sin comentario en el código. Por su nombre: switch camera.
`toggleSpeaker()`	Sin comentario en el código. Por su nombre: toggle speaker.
`startCallService(callId, callType, contactName)` interna	Sin comentario en el código. Por su nombre: start call service.
`stopCallService()` interna	Sin comentario en el código. Por su nombre: stop call service.
`onCleared()`	Sin comentario en el código. Por su nombre: on cleared.

IncomingCallScreen.kt

`ui/screens/call/IncomingCallScreen.kt · 306 líneas`

Fondo de las llamadas: la foto de la persona, desenfocada y oscurecida, teñida con su color. Se reconoce quién es antes de leer el nombre.

`CallBackdrop(photoUrl, colors)` pantalla	Fondo de las llamadas: la foto de la persona, desenfocada y oscurecida, teñida con su color. Se reconoce quién es antes de leer el nombre.
`IncomingCallScreen(callId, callType, navController, viewModel)` pantalla	Llamada entrante a pantalla completa. Cambios respecto a la versión anterior: - Ya no pide la cámara en una llamada de VOZ (los permisos los pide la pantalla de la llamada al contestar, y solo los necesarios). - Se cierra sola si quien llama cuelga: antes se quedaba "sonando" y vibrando para siempre aunque la llamada ya no existiera. - Al contestar o rechazar se retira la notificación (y su tono). - Se quitó "Recorda
`IncomingMainAction(icon, label, color, enabled, modifier, onClick)` pantalla interna	Dibuja en pantalla: incoming main action.
`IncomingSideAction(icon, label, onClick)` pantalla interna	Dibuja en pantalla: incoming side action.

Parte 6

Grupos

Información, miembros e invitaciones.

GroupScreens.kt

`ui/screens/group/GroupScreens.kt · 729 líneas`

Lo que muestra el enlace antes de unirse.

`newCode()`	Sin comentario en el código. Por su nombre: new code.
`linkFor(code)`	Sin comentario en el código. Por su nombre: link for.
`parseCode(input)`	Acepta el enlace entero o solo el código, como lo haya pegado la persona.
`clearMessage()`	Sin comentario en el código. Por su nombre: clear message.
`load(chatId)`	Sin comentario en el código. Por su nombre: load.
`setPhoto(chatId, uri)`	Foto del grupo: se sube junto a los archivos del chat, donde ya pueden escribir sus miembros.
`setDescription(chatId, text)`	Sin comentario en el código. Por su nombre: set description.
`memberOf(uid, admin)` interna asíncrona	Sin comentario en el código. Por su nombre: member of.
`loadContacts()`	Sin comentario en el código. Por su nombre: load contacts.
`act(chatId, failure, block)` interna	Sin comentario en el código. Por su nombre: act.
`rename(chatId, name)`	Sin comentario en el código. Por su nombre: rename.
`add(chatId, uid)`	Sin comentario en el código. Por su nombre: add.
`remove(chatId, uid)`	Sin comentario en el código. Por su nombre: remove.
`setAdmin(chatId, uid, admin)`	Sin comentario en el código. Por su nombre: set admin.
`ensureAdminsWritten(chatId)` interna asíncrona	Los grupos antiguos solo guardan quién los creó: al primer cambio se deja escrito.
`resetInvite(chatId)`	Crea el enlace, o lo cambia por uno nuevo: el anterior deja de servir.
`leave(chatId, onLeft)`	Sin comentario en el código. Por su nombre: leave.
`findInvite(code)` asíncrona	Sin comentario en el código. Por su nombre: find invite.
`join(invite)` asíncrona	La regla solo deja entrar si el código enviado es el vigente del grupo.
`GroupInfoScreen(chatId, navController, viewModel)` pantalla	Dibuja en pantalla: group info screen.
`InviteCard(code, admin, onCreate)` pantalla interna	Dibuja en pantalla: invite card.
`JoinGroupDialog(initialCode, onDismiss)` pantalla	Pide el código, enseña a qué grupo pertenece y entra. Sirve también para los enlaces abiertos.
`search()`	Sin comentario en el código. Por su nombre: search.
`GroupHeader(info, onPhoto)` pantalla interna	Foto, nombre y descripción del grupo. Solo los administradores pueden cambiarlos.

Parte 7

Chats secretos

Chats que viven solo en los dos teléfonos.

SecretChatScreens.kt

`ui/screens/secret/SecretChatScreens.kt · 403 líneas`

Las personas con las que ya hay una conversación.

`loadContacts()`	Las personas con las que ya hay una conversación.
`start(peer, onReady)`	Sin comentario en el código. Por su nombre: start.
`SecretChatsScreen(navController, viewModel)` pantalla	Dibuja en pantalla: secret chats screen.
`messages(id)`	Sin comentario en el código. Por su nombre: messages.
`open(id, peer)`	Sin comentario en el código. Por su nombre: open.
`send(text)`	Sin comentario en el código. Por su nombre: send.
`setTimer(seconds)`	Sin comentario en el código. Por su nombre: set timer.
`delete(onDone)`	Sin comentario en el código. Por su nombre: delete.
`SecretChatScreen(navController, chatId, peerId, viewModel)` pantalla	Dibuja en pantalla: secret chat screen.
`SecretBubble(message)` pantalla interna	Dibuja en pantalla: secret bubble.

Parte 8

Canales

Canales públicos y sus publicaciones.

ChannelScreens.kt

`ui/screens/channels/ChannelScreens.kt · 415 líneas`

`create(name, handle, description, onCreated)`	Sin comentario en el código. Por su nombre: create.
`find(handle, onFound)`	Sin comentario en el código. Por su nombre: find.
`ChannelsScreen(navController, viewModel)` pantalla	Dibuja en pantalla: channels screen.
`open(channelId)`	Sin comentario en el código. Por su nombre: open.
`toggleSubscription()`	Sin comentario en el código. Por su nombre: toggle subscription.
`publish(text, image)`	Sin comentario en el código. Por su nombre: publish.
`deletePost(postId)`	Sin comentario en el código. Por su nombre: delete post.
`ChannelScreen(navController, channelId, viewModel)` pantalla	Dibuja en pantalla: channel screen.
`PostCard(post, canDelete, onDelete)` pantalla interna	Dibuja en pantalla: post card.

Parte 9

Perfil

Perfil propio y de otros usuarios.

EditProfileScreen.kt

`ui/screens/profile/EditProfileScreen.kt · 351 líneas`

`EditProfileScreen(navController, viewModel)` pantalla	Dibuja en pantalla: edit profile screen.
`ProfileEditField(icon, label, value, onValueChange)` pantalla interna	Dibuja en pantalla: profile edit field.

EditProfileViewModel.kt

`ui/screens/profile/EditProfileViewModel.kt · 220 líneas`

`loadUserProfile()` interna	Sin comentario en el código. Por su nombre: load user profile.
`updateDisplayName(name)`	Sin comentario en el código. Por su nombre: update display name.
`updateUsername(username)`	Sin comentario en el código. Por su nombre: update username.
`updateBio(bio)`	Sin comentario en el código. Por su nombre: update bio.
`updatePhoneNumber(phone)`	Sin comentario en el código. Por su nombre: update phone number.
`updateEmail(email)`	Sin comentario en el código. Por su nombre: update email.
`uploadProfilePhoto(uri)`	Sin comentario en el código. Por su nombre: upload profile photo.
`removeProfilePhoto()`	Sin comentario en el código. Por su nombre: remove profile photo.
`saveProfile()`	Sin comentario en el código. Por su nombre: save profile.
`clearError()`	Sin comentario en el código. Por su nombre: clear error.
`clearSuccessMessage()`	Sin comentario en el código. Por su nombre: clear success message.

ProfileCustomizeScreen.kt

`ui/screens/profile/ProfileCustomizeScreen.kt · 283 líneas`

Todo lo que cambia el aspecto del perfil, en un solo sitio y con vista previa en directo. Antes los colores estaban en medio del perfil, a la vista de quien lo visitaba.

`ProfileCustomizeScreen(navController, viewModel)` pantalla	Todo lo que cambia el aspecto del perfil, en un solo sitio y con vista previa en directo. Antes los colores estaban en medio del perfil, a la vista de quien lo visitaba.
`PaletteGrid(indices, selected, locked, onSelect)` pantalla interna	Dibuja en pantalla: palette grid.
`PaletteCell(index, isSelected, locked, modifier, onClick)` pantalla interna	Dibuja en pantalla: palette cell.

ProfileHero.kt

`ui/screens/profile/ProfileHero.kt · 694 líneas`

Identificador de la foto principal en el visor: no pertenece a la galería. */ const val MAIN_SLIDE_ID = "main" private val COVER_HEIGHT = 196.dp private val AVATAR_SIZE = 112.dp /** Una foto o vídeo de perfil.

`heroSlides(user, media)`	Primero la foto o vídeo principal, luego la galería sin repetirlo.

`toViewerMedia()`	Lo que el visor a pantalla completa necesita de cada foto o vídeo.
`presenceText(isOnline, lastSeen)`	"en línea", o cuándo se le vio por última vez, dicho como lo diría una persona.
`ProfileContent(state, seed, isOwn, actions, onOpenViewer)` pantalla	El perfil, igual para el propio y para el de los demás: portada arriba y foto de perfil encima. Tocar la foto abre todas las fotos y vídeos de la persona a pantalla completa, para pasarlos deslizando, como en Telegram. Al desplazar, la portada se queda atrás más despacio.
`SheetRow(icon, label, tint, onClick)` pantalla interna	Dibuja en pantalla: sheet row.
`ProfileCover(coverUrl, coverVideoUrl, colors, scrollOffset)` pantalla interna	Se desplaza más despacio que el contenido y respira despacio: da sensación de capas.
`ProfileAvatar(name, user, colors, isOwn, isUploading, onClick)` pantalla interna	Dibuja en pantalla: profile avatar.
`MediaCountChip(count, onClick)` pantalla interna	Dice que hay más fotos que la que se ve, y las abre.
`ProfileIdentity(name, username, status, online, verified)` pantalla interna	Dibuja en pantalla: profile identity.
`ProfileTopBar(name, collapse, onBack)` pantalla interna	La barra flota sobre la portada y se vuelve sólida, con el nombre, al desplazar.
`ProfileActionTiles(isOwn, actions)` pantalla interna	Dibuja en pantalla: profile action tiles.
`ActionTile(icon, label, index, modifier, onClick)` pantalla interna	Dibuja en pantalla: action tile.
`ProfileInfoCard(user)` pantalla interna	Dibuja en pantalla: profile info card.
`InfoLine(value, label)` pantalla interna	Dibuja en pantalla: info line.
`PremiumBanner(onClick)` pantalla interna	Dibuja en pantalla: premium banner.

ProfileMediaComponents.kt

`ui/screens/profile/ProfileMediaComponents.kt · 288 líneas`

Cuadrícula de fotos y vídeos. Va dentro de una pantalla desplazable, por eso no es perezosa.

`ProfileMediaGrid(media, canAdd, isUploading, onAdd)` pantalla	Cuadrícula de fotos y vídeos. Va dentro de una pantalla desplazable, por eso no es perezosa.
`AddMediaCell(isUploading, onClick)` pantalla interna	Dibuja en pantalla: add media cell.
`MediaCell(item, onClick)` pantalla interna	Dibuja en pantalla: media cell.
`ProfileMediaViewer(media, startIndex, canDelete, onDelete)` pantalla	Visor a pantalla completa: deslizar para cambiar, pellizcar las fotos para ampliar.
`ViewerVideo(url, active)` pantalla interna	Dibuja en pantalla: viewer video.

ProfileScreen.kt

`ui/screens/profile/ProfileScreen.kt · 157 líneas`

`ProfileScreen(navController, userId, showBack, viewModel)` pantalla	Dibuja en pantalla: profile screen.

ProfileViewModel.kt

`ui/screens/profile/ProfileViewModel.kt · 740 líneas`

ProfileViewModel - Manages profile screen state and operations. Features: - Real-time user profile loading - Real-time online status and last seen tracking - Profile and cover photo uploads with crop parameters - Profile editing (name, bio, status) - Own profile vs other user profile detection

`loadUserProfile(userId)`	Loads user profile and starts observing real-time presence.
`uploadProfilePhoto(uri, scale, offsetX, offsetY)`	Uploads a new profile photo with crop parameters.
`uploadCoverPhoto(uri, scale, offsetX, offsetY)`	Uploads a new cover photo with crop parameters.
`saveProfile(name, bio, status)`	Saves profile changes (name, bio, status).
`toggleEditSheet()`	Toggles the edit profile bottom sheet visibility.
`clearUploadSuccess()`	Clears the upload success flag.
`userRef(userId)` interna	Sin comentario en el código. Por su nombre: user ref.
`observeMedia(userId)` interna	Sin comentario en el código. Por su nombre: observe media.
`ownUid()` interna	Sin comentario en el código. Por su nombre: own uid.
`isVideo(uri)` interna	Sin comentario en el código. Por su nombre: is video.
`videoPoster(uri)` interna	Primer fotograma como imagen: es lo que se ve en cuadrículas y listas.
`videoDurationMs(uri)` interna	Sin comentario en el código. Por su nombre: video duration ms.
`addProfileMedia(uri)`	Añade una foto o un vídeo a la galería del perfil.
`deleteProfileMedia(item)`	Sin comentario en el código. Por su nombre: delete profile media.
`uploadProfileVideo(uri)`	Pone un vídeo corto como avatar. Su primer fotograma pasa a ser la foto en las listas.
`removeProfileVideo()`	Sin comentario en el código. Por su nombre: remove profile video.
`setMainFromMedia(item)`	Pone una foto o vídeo de la galería como principal, sin volver a subir nada.
`uploadCoverVideo(uri)`	La portada también puede ser un vídeo en bucle; su primer fotograma queda como imagen.
`clearCoverVideo()` interna	Sin comentario en el código. Por su nombre: clear cover video.
`removeCover()`	Sin comentario en el código. Por su nombre: remove cover.
`setProfileColor(index)`	Sin comentario en el código. Por su nombre: set profile color.
`clearError()`	Sin comentario en el código. Por su nombre: clear error.
`reloadProfile()`	Reloads the current user profile.
`buildChatId(otherUserId)`	Builds a chat ID from two user IDs (sorted alphabetically).
`startPresenceObservation(userId)` interna	Starts observing real-time presence for the given user.
`onCleared()`	Sin comentario en el código. Por su nombre: on cleared.

ProfileViewerScreen.kt

`ui/screens/profile/ProfileViewerScreen.kt · 63 líneas`

Perfil de otra persona, al tocar su avatar: el mismo diseño que el propio.

`ProfileViewerScreen(navController, userId, viewModel)` pantalla	Perfil de otra persona, al tocar su avatar: el mismo diseño que el propio.

Parte 10

Ajustes y estudio de diseño

Ajustes, personalización, ayuda y privacidad.

AboutScreen.kt

`ui/screens/settings/AboutScreen.kt · 852 líneas`

Acerca de: la app y el dispositivo que la está ejecutando, con datos en vivo. Todo lo que aparece aquí se lee en el momento del sistema: la hora avanza, la batería se actualiza con el aviso del sistema y la memoria se vuelve a medir cada pocos segundos. No hay ningún valor escrito a mano.

`AboutScreen(navController)` pantalla	Acerca de: la app y el dispositivo que la está ejecutando, con datos en vivo. Todo lo que aparece aquí se lee en el momento del sistema: la hora avanza, la batería se actualiza con el aviso del sistema y la memoria se vuelve a medir cada pocos segundos. No hay ningún valor escrito a mano.
`AboutHero()` pantalla interna	La cara de la pantalla: el emblema gira despacio sobre un brillo que respira.
`AboutTile(icon, title, value, index, modifier, onClick)` pantalla interna	Una tarjeta de dato. Con onClick se puede abrir; sin él solo informa.
`LicensesDialog(onDismiss)` pantalla interna	Dibuja en pantalla: licenses dialog.
`HistoryDialog(onDismiss)` pantalla interna	Dibuja en pantalla: history dialog.
`HighlightsGrid()` pantalla interna	Dibuja en pantalla: highlights grid.
`ChangelogCard()` pantalla interna	Dibuja en pantalla: changelog card.
`CommunityRow(onOpen)` pantalla interna	Dibuja en pantalla: community row.
`CommunityTile(icon, label, index, modifier, onClick)` pantalla interna	Dibuja en pantalla: community tile.
`LiveDeviceCard(now, battery, info, locale)` pantalla interna	Dibuja en pantalla: live device card.
`DetailsToggle(expanded, onClick)` pantalla interna	Dibuja en pantalla: details toggle.
`rememberBatteryState(context)` pantalla	Dibuja en pantalla: remember battery state.
`onReceive(ctx, intent)`	Sin comentario en el código. Por su nombre: on receive.
`toBatteryState()` interna	Sin comentario en el código. Por su nombre: to battery state.
`BatteryMeter(battery)` pantalla interna	Dibuja en pantalla: battery meter.
`rememberTicker(periodMs)` pantalla	Dibuja en pantalla: remember ticker.
`rememberMemoryState(context)` pantalla interna	Dibuja en pantalla: remember memory state.
`currentNetworkLabel(context)` interna	Sin comentario en el código. Por su nombre: current network label.
`readStaticDeviceInfo(context)` interna	Sin comentario en el código. Por su nombre: read static device info.
`formatBytes(bytes)` interna	Sin comentario en el código. Por su nombre: format bytes.
`formatUptime(millis)` interna	Sin comentario en el código. Por su nombre: format uptime.

AccountSettingsScreen.kt

`ui/screens/settings/AccountSettingsScreen.kt · 485 líneas`

`AccountSettingsScreen(navController, viewModel)` pantalla	Dibuja en pantalla: account settings screen.
`safeNavigateBack()`	Sin comentario en el código. Por su nombre: safe navigate back.

`AccountInfoItem(label, value, icon, onClick)` pantalla	Dibuja en pantalla: account info item.

AiFeatureToggles.kt

`ui/screens/settings/AiFeatureToggles.kt · 167 líneas`

Los seis interruptores de funciones de IA. Se muestran siempre, pero desactivados mientras no haya proveedor configurado: esconderlos dejaba al usuario sin saber que existen, y permitir activarlos sin clave produciría funciones que fallan en silencio al usarlas. Mejorar fotos y dictado por voz no necesitan proveedor —ocurren en el propio dispositivo— así que están siempre disponibles.

`AiFeatureToggles(viewModel)` pantalla	Los seis interruptores de funciones de IA. Se muestran siempre, pero desactivados mientras no haya proveedor configurado: esconderlos dejaba al usuario sin saber que existen, y permitir activarlos sin clave produciría funciones que fallan en silencio al usarlas. Mejorar fotos y dictado por voz no necesitan proveedor —ocurren en el propio dispositivo— así que están siempre disponibles.
`AiFeatureToggle(icon, title, description, checked, enabled, onCheckedChange)` pantalla interna	Dibuja en pantalla: ai feature toggle.

AiFeaturesScreen.kt

`ui/screens/settings/AiFeaturesScreen.kt · 564 líneas`

🔑 SECCIÓN "API KEY DE GEMINI" Permite al usuario introducir, guardar y borrar su propia API key de Gemini, almacenada de forma cifrada por com.Azelmods.App.data.ai.AiKeyStore. Incluye toggle mostrar/ocultar, validación mínima (no vacía), indicador de estado (activa/inactiva) y mensaje de confirmación.

`AiFeaturesScreen(navController, aiKeyViewModel)` pantalla	Dibuja en pantalla: ai features screen.
`GeminiApiKeySection(viewModel)` pantalla	🔑 SECCIÓN "API KEY DE GEMINI" Permite al usuario introducir, guardar y borrar su propia API key de Gemini, almacenada de forma cifrada por com.Azelmods.App.data.ai.AiKeyStore. Incluye toggle mostrar/ocultar, validación mínima (no vacía), indicador de estado (activa/inactiva) y mensaje de confirmación.
`ModelCatalog(suggested, remote, isLoading, selected, onPick)` pantalla interna	Catálogo de modelos del proveedor activo. Combina dos fuentes deliberadamente: - **Sugeridos**: atajos del preset, disponibles sin red. - **Del proveedor**: lo que responde `GET /models`, es decir el catálogo REAL y actual. Por eso no hace falta publicar una versión de la app cada vez que un proveedor saca un modelo nuevo. El campo "Modelo" sigue siendo texto libre: esto son atajos, no una lista cerrada.
`ModelChips(models, selected, onPick)` pantalla interna	Fila envolvente de chips seleccionables con el id de cada modelo.

AiFeaturesViewModel.kt

`ui/screens/settings/AiFeaturesViewModel.kt · 65 líneas`

`false` si no hay proveedor de IA configurado. */ val providerReady: Boolean = false, val smartReplies: Boolean = false, val autoTranslate: Boolean = false, val conversationSummary: Boolean = false, val toneSuggestions: Boolean = false, val photoEnhance: Boolean = false, val voiceTranscription: Boolean = false) /** Estado de los interruptores de funciones de IA. Cada `set*` escribe en DataStore y el flujo devuelve el valor ya guardado, así que la interfaz refleja lo que hay en disco y no un estado local que se pierde al salir de la pantalla —que es exactamente lo que hacían estos interruptores antes de retirarlos—.

`refreshProviderState()`	Relee si hay proveedor: el usuario puede pegar la clave en esta misma pantalla.
`setSmartReplies(enabled)`	Sin comentario en el código. Por su nombre: set smart replies.
`setAutoTranslate(enabled)`	Sin comentario en el código. Por su nombre: set auto translate.

`setConversationSummary(enabled)`	Sin comentario en el código. Por su nombre: set conversation summary.
`setToneSuggestions(enabled)`	Sin comentario en el código. Por su nombre: set tone suggestions.
`setPhotoEnhance(enabled)`	Sin comentario en el código. Por su nombre: set photo enhance.
`setVoiceTranscription(enabled)`	Sin comentario en el código. Por su nombre: set voice transcription.

AiKeyViewModel.kt

`ui/screens/settings/AiKeyViewModel.kt · 213 líneas`

🔑 AI KEY VIEW MODEL ViewModel mínimo para la sección "API Key de Gemini" de AiFeaturesScreen. Encapsula el acceso a AiKeyStore (almacenamiento cifrado) y expone: - hasKey: estado reactivo que indica si hay una clave activa. - feedback: mensaje efímero de confirmación/validación para la UI. Toda operación está protegida (el AiKeyStore nunca lanza), por lo que la UI jamás crashea al guardar/borrar la clave.

`saveApiKey(rawKey)`	Guarda la API key tras validación mínima (no vacía). Devuelve feedback en feedback.
`clearApiKey()`	Borra la API key almacenada.
`currentBaseUrl()`	URL base efectiva del proveedor activo (la del usuario o la del preset).
`currentModel()`	Modelo efectivo del proveedor activo.
`selectProvider(newProvider)`	Cambia de proveedor. Resetea URL y modelo a los del preset, porque arrastrar la URL del anterior solo produce errores difíciles de entender.
`saveEndpoint(baseUrl, model)`	Guarda la URL base y el modelo del proveedor activo.
`suggestedModels()`	Modelos sugeridos del preset del proveedor activo (atajo sin red).
`loadProviderModels()`	Consulta al proveedor su lista real de modelos. Requiere URL base válida y, salvo en servidores locales, una API key configurada.
`selectModel(model)`	Fija el modelo activo (desde la lista o escrito a mano).
`clearFeedback()`	Limpia el mensaje de feedback tras mostrarlo.

AppearanceScreen.kt

`ui/screens/settings/AppearanceScreen.kt · 255 líneas`

Cualquier color: tono, intensidad y brillo, viendo el resultado mientras se mueve.

`AppearanceScreen(navController, viewModel)` pantalla	Dibuja en pantalla: appearance screen.
`SettingsRadioItem(title, selected, onClick)` pantalla	Dibuja en pantalla: settings radio item.
`CustomAccentDialog(initial, onDismiss)` pantalla interna	Cualquier color: tono, intensidad y brillo, viendo el resultado mientras se mueve.

ChatStudioScreen.kt

`ui/screens/settings/ChatStudioScreen.kt · 1028 líneas`

Todo el aspecto de los chats en un sitio. La vista previa se queda fija arriba y usa las mismas piezas que la app real: lo que se ve aquí es exactamente lo que se verá en los chats.

`ChatStudioScreen(navController)` pantalla	Todo el aspecto de los chats en un sitio. La vista previa se queda fija arriba y usa las mismas piezas que la app real: lo que se ve aquí es exactamente lo que se verá en los chats.
`LivePreview()` pantalla interna	Dibuja en pantalla: live preview.
`PreviewBubble(text, own, style, color, accent, textScale)` pantalla interna	Dibuja en pantalla: preview bubble.

`ChoiceChips(options, selected, label)` pantalla interna	Dibuja en pantalla: choice chips.
`RowOptions()` pantalla interna	Dibuja en pantalla: row options.
`ColorChoice(title, current, onSelect)` pantalla interna	Dibuja en pantalla: color choice.
`BubbleOptions()` pantalla interna	Dibuja en pantalla: bubble options.
`EntryOptions()` pantalla interna	Dibuja en pantalla: entry options.
`HomeOptions()` pantalla interna	Dibuja en pantalla: home options.
`SettingsOptions()` pantalla interna	Dibuja en pantalla: settings options.
`LayoutOption(title, hint, selected, onClick)` pantalla interna	Una opción de diseño con su miniatura: se ve lo que hace antes de elegirla.
`HomeMiniature(style, large)` pantalla interna	Dibuja en pantalla: home miniature.
`bar(y, x, width, height, color)`	Sin comentario en el código. Por su nombre: bar.
`SettingsMiniature(style, large)` pantalla interna	Dibuja en pantalla: settings miniature.
`rect(x, y, width, height, color)`	Sin comentario en el código. Por su nombre: rect.
`VisualChoices(options, selected, label)` pantalla interna	Opciones como tarjetas con su dibujo: se ve cómo queda antes de elegirla.
`BubbleMiniature(style, ownColor)` pantalla interna	La burbuja real, con su forma y su relleno.
`EntryMiniature(style)` pantalla interna	El campo de escribir dibujado en pequeño: forma, botones y dónde quedan.
`pill(x, w, r)`	Sin comentario en el código. Por su nombre: pill.
`hint(x)`	Sin comentario en el código. Por su nombre: hint.
`NavMiniature(style)` pantalla interna	La barra inferior dibujada en pequeño.
`dots(n, from, to, selected, labels)`	Sin comentario en el código. Por su nombre: dots.
`fullBar(radius)`	Sin comentario en el código. Por su nombre: full bar.
`HomeLivePreview()` pantalla interna	Inicio en vivo: la cabecera elegida y la barra inferior real con sus iconos.
`SettingsLivePreview()` pantalla interna	Ajustes en vivo: el diseño elegido en grande.

DeviceInfoScreen.kt

`ui/screens/settings/DeviceInfoScreen.kt · 371 líneas`

`DeviceInfoScreen(navController)` pantalla	Dibuja en pantalla: device info screen.
`BatteryCard(batteryLevel, isCharging, temperature, themeColor)` pantalla	Dibuja en pantalla: battery card.
`DeviceInfoCard(icon, title, value, themeColor)` pantalla	Dibuja en pantalla: device info card.
`getDeviceInfo(context)` interna	Sin comentario en el código. Por su nombre: get device info.
`getBatteryInfo(context)` interna	Sin comentario en el código. Por su nombre: get battery info.

FontSizeScreen.kt

`ui/screens/settings/FontSizeScreen.kt · 157 líneas`

Pantalla de tamaño de fuente. ANTES era código muerto: escribía en SharedPreferences "nexus_prefs" con claves "Small/Medium/Large/XLarge", pero el tema lee de UserPreferences (fichero "nexus_chat_preferences") y su tabla de escalas esperaba "EXTRA_LARGE". Así que cambiar el tamaño NO tenía ningún efecto en la app. Ahora persiste a través de SettingsViewModel → UserPreferences.setFontSize, la MISMA fuente que consume `NexusChatTheme`, de modo que

el cambio recompone el árbol al instante y afecta a toda la app. Los valores guardados casan con `getTypographyForSize`.

`FontSizeScreen(navController, viewModel)` pantalla	Pantalla de tamaño de fuente. ANTES era código muerto: escribía en SharedPreferences "nexus_prefs" con claves "Small/Medium/Large/XLarge", pero el tema lee de UserPreferences (fichero "nexus_chat_preferences") y su tabla de escalas esperaba "EXTRA_LARGE". Así que cambiar el tamaño NO tenía ningún efecto en la app. Ahora persiste a través de SettingsViewModel → UserPreferences.setFontSize, la MISMA fuente que consume
`normalize(v)`	Sin comentario en el código. Por su nombre: normalize.

HelpSupportScreen.kt

`ui/screens/settings/HelpSupportScreen.kt · 312 líneas`

Ayuda y soporte. Revisión de la v5: esta pantalla enlazaba a un ecosistema que NO EXISTE — `azelmods.com`, `docs.azelmods.com`, `community.azelmods.com`, `feedback.azelmods.com`, `support@azelmods.com`, `bugs@azelmods.com`—. Todos esos botones abrían un navegador o un cliente de correo hacia la nada, que es peor que no ofrecer soporte: el usuario cree que hay un canal y no lo hay. Ahora todo apunta al único canal real del proyecto: **el repositorio de GitHub**. Las guías se leen dentro de la app, sin conexión. / private const val REPO_URL = "https://github.com/Azelmods677/NexusChat" private const val ISSUES_URL = "$REPO_URL/issues" @OptIn(ExperimentalMaterial3Api::class) @Composable fun Help

`HelpSupportScreen(navController)` pantalla	Dibuja en pantalla: help support screen.
`open(url)`	Abre una URL sin dejar que un dispositivo sin navegador tumbe la pantalla.
`SectionHeader(text)` pantalla interna	Dibuja en pantalla: section header.

NotificationsScreen.kt

`ui/screens/settings/NotificationsScreen.kt · 142 líneas`

`NotificationsScreen(navController, viewModel)` pantalla	Dibuja en pantalla: notifications screen.

PrivacyPolicyScreen.kt

`ui/screens/settings/PrivacyPolicyScreen.kt · 56 líneas`

`PrivacyPolicyScreen(navController)` pantalla	Dibuja en pantalla: privacy policy screen.

PrivacySecurityScreen.kt

`ui/screens/settings/PrivacySecurityScreen.kt · 605 líneas`

Acceso a AppLockPreferences desde un Composable sin pasar por el ViewModel.

`PrivacySecurityScreen(navController, viewModel)` pantalla	Dibuja en pantalla: privacy security screen.
`appLockPreferences()`	Sin comentario en el código. Por su nombre: app lock preferences.
`TwoFactorSetupDialog(appLockPreferences, onDismiss)` pantalla interna	Alta de 2FA: genera un secreto TOTP, lo muestra para que el usuario lo añada a su app de autenticación (o lo copie), y exige un código válido antes de activarlo. Confirmar con un código demuestra que el secreto quedó bien guardado en ambos lados. Requisito: debe existir ya un PIN de bloqueo, porque el 2FA se exige en la pantalla de desbloqueo (sin primer factor, no hay dónde pedir el segundo).
`TwoFactorDisableDialog(appLockPreferences, onDismiss)` pantalla interna	Baja de 2FA: confirma y borra el secreto.

`SettingsSwitchItem(title, subtitle, icon, checked, onCheckedChange)` pantalla	Dibuja en pantalla: settings switch item.

SettingsComponents.kt

`ui/screens/settings/SettingsComponents.kt · 25 líneas`

Entrada de ajustes con título primero (firma usada por las subpantallas).

`SettingsItem(title, subtitle, icon, iconTint, onClick)` pantalla	Entrada de ajustes con título primero (firma usada por las subpantallas).

SettingsDashboard.kt

`ui/screens/settings/SettingsDashboard.kt · 314 líneas`

Cabecera de Ajustes: quién eres y cómo está el teléfono de un vistazo, con datos en vivo. Tocar una tarjeta lleva a su sección.

`SettingsDashboard(userName, userHandle, userPhoto, userColor, userId, onOpenProfile)` pantalla	Cabecera de Ajustes: quién eres y cómo está el teléfono de un vistazo, con datos en vivo. Tocar una tarjeta lleva a su sección.
`batteryColor(percent, charging)` interna	Sin comentario en el código. Por su nombre: battery color.
`DashboardCard(onClick)` pantalla interna	Dibuja en pantalla: dashboard card.
`MetricCard(icon, iconColor, title, value, detail, onClick)` pantalla interna	Dibuja en pantalla: metric card.
`SettingsIconTile(icon, color)` pantalla	Icono de una entrada de ajustes sobre su color: ayuda a encontrar cada sección de un vistazo.

SettingsScreen.kt

`ui/screens/settings/SettingsScreen.kt · 716 líneas`

Secciones agrupadas en una tarjeta.

`SettingsScreen(navController, viewModel)` pantalla	Dibuja en pantalla: settings screen.
`LanguageDialog(current, onSelect)` pantalla interna	Dibuja en pantalla: language dialog.
`SettingsSection(title, content)` pantalla	Dibuja en pantalla: settings section.
`SettingsGroup(content)` pantalla interna	Secciones agrupadas en una tarjeta.
`rememberSettingsLinks(navController, context, onLanguage)` pantalla interna	Todos los accesos en una lista: cada diseño y la búsqueda parten de la misma.
`go(route)`	Sin comentario en el código. Por su nombre: go.
`SettingsSearch(query, onChange)` pantalla interna	Dibuja en pantalla: settings search.
`SettingsProfileCard(me, onClick)` pantalla interna	Dibuja en pantalla: settings profile card.
`SettingsHeader(me, onProfile)` pantalla interna	Título grande con un punto de color, tu foto y una cápsula con los accesos que más se usan.
`SettingsProfileRight(me, onClick)` pantalla interna	Título grande y una tarjeta de 75 dp con el nombre a la izquierda y la foto a la derecha.
`SettingsCardStack(items)` pantalla interna	Una tarjeta por entrada: redondeadas por fuera del grupo y casi rectas por dentro.
`SettingsGrid(items)` pantalla interna	Dibuja en pantalla: settings grid.

`SettingsCompactLayout(sections)` pantalla interna	Lo principal en dos columnas pequeñas dentro de una tarjeta; herramientas e información, en filas.
`SettingsCompactCell(link, modifier)` pantalla interna	Dibuja en pantalla: settings compact cell.
`SettingsCompactRow(link)` pantalla interna	Fila de una sola línea, sin descripción.
`SettingsEntry(link)` pantalla interna	Dibuja en pantalla: settings entry.
`SettingsEntry(icon, color, title, subtitle, titleColor, onClick)` pantalla interna	Entrada de ajustes con descripción: dice qué hay dentro sin tener que abrirla.

SettingsViewModel.kt

`ui/screens/settings/SettingsViewModel.kt · 362 líneas`

Shared ViewModel for all Settings screens Provides real-time access to user preferences

`logout(onDone)`	Cierra la sesión de verdad. Lo que necesita permiso (marcarse fuera de línea y retirar el token de notificaciones de este dispositivo) se hace antes de salir; si no, quien cierra sesión seguiría recibiendo los avisos de esa cuenta.
`loadBlockedContacts()`	Sin comentario en el código. Por su nombre: load blocked contacts.
`unblockContact(chatId, uid)`	Sin comentario en el código. Por su nombre: unblock contact.
`setSplashAnimation(enabled)`	Sin comentario en el código. Por su nombre: set splash animation.
`setLowDataMode(enabled)`	Sin comentario en el código. Por su nombre: set low data mode.
`setTranslationLanguage(language)`	Sin comentario en el código. Por su nombre: set translation language.
`updateDisplayName(name)`	Sin comentario en el código. Por su nombre: update display name.
`updateUsername(username)`	Sin comentario en el código. Por su nombre: update username.
`updateBio(bio)`	Sin comentario en el código. Por su nombre: update bio.
`updatePhoneNumber(phone)`	Sin comentario en el código. Por su nombre: update phone number.
`updateEmail(email)`	Sin comentario en el código. Por su nombre: update email.
`setLastSeenEnabled(enabled)`	Sin comentario en el código. Por su nombre: set last seen enabled.
`setProfilePhotoVisible(visible)`	Sin comentario en el código. Por su nombre: set profile photo visible.
`setReadReceiptsEnabled(enabled)`	Sin comentario en el código. Por su nombre: set read receipts enabled.
`setTwoFactorEnabled(enabled)`	Sin comentario en el código. Por su nombre: set two factor enabled.
`setNotificationsEnabled(enabled)`	Sin comentario en el código. Por su nombre: set notifications enabled.
`setSoundEnabled(enabled)`	Sin comentario en el código. Por su nombre: set sound enabled.
`setVibrationEnabled(enabled)`	Sin comentario en el código. Por su nombre: set vibration enabled.
`setMessagePreview(enabled)`	Sin comentario en el código. Por su nombre: set message preview.
`setGroupNotifications(enabled)`	Sin comentario en el código. Por su nombre: set group notifications.
`setNotificationSound(uri, name)`	Sin comentario en el código. Por su nombre: set notification sound.
`setDarkModeEnabled(enabled)`	Sin comentario en el código. Por su nombre: set dark mode enabled.
`setAccentColor(color)`	Sin comentario en el código. Por su nombre: set accent color.
`setFontSize(size)`	Sin comentario en el código. Por su nombre: set font size.
`setWallpaper(type, value)`	Sin comentario en el código. Por su nombre: set wallpaper.
`setGradientWallpaper(colors)`	Sin comentario en el código. Por su nombre: set gradient wallpaper.
`setAutoDownloadPhotos(enabled)`	Sin comentario en el código. Por su nombre: set auto download photos.
`setAutoDownloadVideos(enabled)`	Sin comentario en el código. Por su nombre: set auto download videos.
`setAutoDownloadFiles(enabled)`	Sin comentario en el código. Por su nombre: set auto download files.

`changePassword(currentPassword, newPassword)`	Sin comentario en el código. Por su nombre: change password.
`sendPasswordResetEmail()`	Enlace para crear una contraseña nueva, al correo de la cuenta. Sirve a cuentas de Google.
`deleteAccount()`	Sin comentario en el código. Por su nombre: delete account.
`clearAccountActionState()`	Sin comentario en el código. Por su nombre: clear account action state.
`clearAllData()`	Sin comentario en el código. Por su nombre: clear all data.

StorageDataScreen.kt

`ui/screens/settings/StorageDataScreen.kt · 244 líneas`

`StorageDataScreen(navController, viewModel)` pantalla	Dibuja en pantalla: storage data screen.
`formatBytes(bytes)`	Sin comentario en el código. Por su nombre: format bytes.

TranslationLanguageScreen.kt

`ui/screens/settings/TranslationLanguageScreen.kt · 119 líneas`

`TranslationLanguageScreen(navController, viewModel)` pantalla	Dibuja en pantalla: translation language screen.

Parte 11

Seguridad

Bloqueo, Tor y navegador privado.

AnonymousModeToggle.kt

`ui/screens/security/AnonymousModeToggle.kt · 258 líneas`

Toggle switch for Anonymous Mode with connection status display Displays: - Toggle switch for enabling/disabling Tor - Connection status (Disabled, Connecting X%, Connected, Error) - Disables toggle during bootstrap process Cuando se proporciona orbotStatus (derivado de `OrbotDetector` vía `mapOrbotStatus`), el estado de Orbot se presenta de forma clara y accionable y el toggle NUNCA queda en un estado de error permanente: ante un `TorState.Error` se muestra el mensaje/acción de Orbot y el toggle vuelve a un estado consistente pero operable para reintentar (degradación elegante). Requirements: 6.4, 7.1, 7.3, 17.1, 17.2, 17.3, 17.4, 17.5

`AnonymousModeToggle(torState, onToggle)` pantalla	Toggle switch for Anonymous Mode with connection status display Displays: - Toggle switch for enabling/disabling Tor - Connection status (Disabled, Connecting X%, Connected, Error) - Disables toggle during bootstrap process Cuando se proporciona orbotStatus (derivado de `OrbotDetector` vía `mapOrbotStatus`), el estado de Orbot se presenta de forma clara y accionable y el toggle NUNCA queda en un estado de error perma
`StatusText(torState, orbotStatus)` pantalla interna	Status text based on Tor state. Cuando se proporciona orbotStatus y Tor está en error, mostramos un estado derivado del estado real de Orbot (no un "Error" permanente).
`AnimatedConnectingDots()` pantalla interna	Animated dots for connecting state

AppLockScreen.kt

`ui/screens/security/AppLockScreen.kt · 598 líneas`

AppLockScreen - Pantalla de desbloqueo con PIN y biometría Features: - Entrada de PIN numérico (4-6 dígitos) - Autenticación biométrica (huella/Face ID) - Animación de error al ingresar PIN incorrecto - Indicadores visuales de PIN ingresado - Teclado numérico personalizado Requirements: 9.1, 9.2, 9.3

`AppLockScreen(onUnlocked)` pantalla	AppLockScreen - Pantalla de desbloqueo con PIN y biometría Features: - Entrada de PIN numérico (4-6 dígitos) - Autenticación biométrica (huella/Face ID) - Animación de error al ingresar PIN incorrecto - Indicadores visuales de PIN ingresado - Teclado numérico personalizado Requirements: 9.1, 9.2, 9.3
`TwoFactorStep(code, isError, errorMessage, onCodeChange)` pantalla interna	Segundo factor: entrada del código TOTP de 6 dígitos. Aparece SOLO después de que el PIN o la huella hayan sido correctos, de modo que exige de verdad los dos factores para desbloquear.
`PinIndicator(isFilled, isError)` pantalla interna	Dibuja en pantalla: pin indicator.
`NumericKeypad(onNumberClick)` pantalla interna	Dibuja en pantalla: numeric keypad.
`NumericKey(text, onClick)` pantalla interna	Dibuja en pantalla: numeric key.
`IconKey(icon, onClick)` pantalla interna	Dibuja en pantalla: icon key.
`onTotpChange(value)`	Sin comentario en el código. Por su nombre: on totp change.
`addDigit(digit)`	Sin comentario en el código. Por su nombre: add digit.
`removeLastDigit()`	Sin comentario en el código. Por su nombre: remove last digit.
`verifyPin()` asíncrona	Verifica el PIN. Devuelve `true` SOLO si el desbloqueo es total. Si el PIN es correcto pero el 2FA está activo, no desbloquea: activa requires2fa para que la pantalla pida el código TOTP, y devuelve `false`.
`verifyTotp(onSuccess)`	Comprueba el segundo factor. Desbloquea (y llama a onSuccess) solo si es válido.

`authenticateWithBiometrics(activity, onSuccess)`	Sin comentario en el código. Por su nombre: authenticate with biometrics.
`onAuthenticationSucceeded(result)`	Sin comentario en el código. Por su nombre: on authentication succeeded.
`onAuthenticationError(errorCode, errString)`	Sin comentario en el código. Por su nombre: on authentication error.
`onAuthenticationFailed()`	Sin comentario en el código. Por su nombre: on authentication failed.

OrbotWelcomeScreen.kt

`ui/screens/security/OrbotWelcomeScreen.kt · 622 líneas`

🌐 OrbotWelcomeScreen — Pantalla de bienvenida que guía al usuario a instalar y configurar Orbot para navegación anónima. Estados: 1. Orbot no instalado → Botón para descargar desde Guardian Project 2. Orbot instalado pero no activo → Botón para abrir Orbot 3. Orbot conectado y funcionando → Confirmación + acceso a Tor Browser

`OrbotWelcomeScreen(navController)` pantalla	🌐 OrbotWelcomeScreen — Pantalla de bienvenida que guía al usuario a instalar y configurar Orbot para navegación anónima. Estados: 1. Orbot no instalado → Botón para descargar desde Guardian Project 2. Orbot instalado pero no activo → Botón para abrir Orbot 3. Orbot conectado y funcionando → Confirmación + acceso a Tor Browser
`TorOnionAnimation(status)` pantalla interna	Dibuja en pantalla: tor onion animation.
`OrbotStatusCard(status)` pantalla interna	Dibuja en pantalla: orbot status card.
`OrbotActionButton(icon, text, description, color, enabled, onClick)` pantalla interna	Dibuja en pantalla: orbot action button.
`InstallStepCard(step, title, description)` pantalla interna	Dibuja en pantalla: install step card.
`NextStepItem(icon, title, description, onClick)` pantalla interna	Dibuja en pantalla: next step item.

SecurityScreen.kt

`ui/screens/security/SecurityScreen.kt · 283 líneas`

Punto de entrada a la seguridad: una lista corta en la que cada fila hace algo real.

`SecurityScreen(navController, viewModel)` pantalla	Punto de entrada a la seguridad: una lista corta en la que cada fila hace algo real.
`AppLockSetupDialog(onDismiss)` pantalla interna	Dibuja en pantalla: app lock setup dialog.
`backupManager()`	Sin comentario en el código. Por su nombre: backup manager.
`BackupRestoreDialog(onDismiss)` pantalla interna	Dibuja en pantalla: backup restore dialog.

SecurityViewModel.kt

`ui/screens/security/SecurityViewModel.kt · 65 líneas`

Estado del modo anónimo para las pantallas de seguridad. La preferencia se guarda al pulsar el interruptor, no al terminar de conectar: si el usuario lo activó, al reabrir la app debe seguir activo aunque Tor no llegara a conectar, porque es lo que evita que algo salga sin protección sin que se dé cuenta.

`setGhostMode(on)`	Sin comentario en el código. Por su nombre: set ghost mode.
`enableAnonymousMode()`	Sin comentario en el código. Por su nombre: enable anonymous mode.
`disableAnonymousMode()`	Sin comentario en el código. Por su nombre: disable anonymous mode.
`setEngine(engine)`	Sin comentario en el código. Por su nombre: set engine.

`verifyTor()`	Vuelve a preguntar a check.torproject.org si salimos por Tor y con qué IP.
`newIdentity()`	Pide circuitos nuevos. Devuelve false si el motor en uso no lo permite.
`requestOrbotStart(context)`	Pide a Orbot que arranque Tor sin salir de NexusChat.

TorBrowserScreenNew.kt

`ui/screens/security/TorBrowserScreenNew.kt · 732 líneas`

Con Tor activo, WebRTC seguiría hablando UDP directamente con servidores STUN y revelaría la IP real a cualquier web que lo pidiera. Tor Browser lo desactiva; aquí igual. / private const val PRIVACY_SCRIPT = """ (function () { try { 'RTCPeerConnection', 'webkitRTCPeerConnection', 'RTCDataChannel', 'RTCSessionDescription', 'RTCIceCandidate' .forEach(function (k) { try { Object.defineProperty(window, k, { value: undefined, writable: false, configurable: false }); } catch (e) { try { windowk = undefined; } catch (e2) {} } }); if (navigator.mediaDevices) { navigator.mediaDevices.getUserMedia = function () { return Promise.reject(new Error('Bloqueado')); }; navigator.mediaDevices.enumerateDevices

`enableAnonymousMode()`	Sin comentario en el código. Por su nombre: enable anonymous mode.
`TorBrowserScreenNew(navController, viewModel)` pantalla	Navegador privado. WebView tiene su propia pila de red, así que el enrutado de la app no le afecta: su proxy se fija aparte y, con el modo anónimo encendido, nunca carga nada por la red normal. Al salir se borran cookies, caché, almacenamiento e historial.
`open(url)`	Sin comentario en el código. Por su nombre: open.
`onProgressChanged(view, newProgress)`	Sin comentario en el código. Por su nombre: on progress changed.
`onPermissionRequest(request)`	Sin comentario en el código. Por su nombre: on permission request.
`onGeolocationPermissionsShowPrompt(origin, callback)`	Sin comentario en el código. Por su nombre: on geolocation permissions show prompt.
`syncNavigation(view)` interna	Sin comentario en el código. Por su nombre: sync navigation.
`onPageStarted(view, url, favicon)`	Sin comentario en el código. Por su nombre: on page started.
`onPageFinished(view, url)`	Sin comentario en el código. Por su nombre: on page finished.
`doUpdateVisitedHistory(view, url, isReload)`	Sin comentario en el código. Por su nombre: do update visited history.
`shouldOverrideUrlLoading(view, request)`	Sin comentario en el código. Por su nombre: should override url loading.
`onReceivedError(view, request, error)`	Sin comentario en el código. Por su nombre: on received error.
`onReceivedSslError(view, handler, error)`	Sin comentario en el código. Por su nombre: on received ssl error.
`onRenderProcessGone(view, detail)`	Sin comentario en el código. Por su nombre: on render process gone.
`RouteNotice(route, torState, proxySupported, onEnableTor)` pantalla interna	Solo aparece cuando hay algo que el usuario debe saber sobre por dónde sale.
`BrowserAction(icon, description, enabled, onClick)` pantalla interna	Dibuja en pantalla: browser action.
`BrowserMenuItem(label, icon, enabled, onClick)` pantalla interna	Dibuja en pantalla: browser menu item.
`homeFor(route)` interna	Sin comentario en el código. Por su nombre: home for.
`displayAddress(url)` interna	Lo que se lee en la barra sin tocarla: el dominio, que es lo que importa para no caer en un engaño.
`toUrl(typed, route)` interna	Convierte lo escrito en una dirección, o en una búsqueda si no parece una.
`errorAdvice(url, route, detail)` interna	Sin comentario en el código. Por su nombre: error advice.
`showOnionNeedsTor(view, url)` interna	Sin comentario en el código. Por su nombre: show onion needs tor.
`showMessagePage(view, url, title, body)` interna	Página propia para errores. Todo lo que viene de fuera va escapado: la URL la controla la web.
`configurePrivacy(web)` interna	Sin comentario en el código. Por su nombre: configure privacy.

`wipeBrowsingData(web)` interna	Sin comentario en el código. Por su nombre: wipe browsing data.

TorCircuitInfo.kt

`ui/screens/security/TorCircuitInfo.kt · 208 líneas`

Displays current Tor circuit information Shows: - Entry, middle, and exit nodes - Circuit ID - Bandwidth information - "New Identity" button to create a new circuit Requirements: 17.6, 17.7, 17.8, 4.1, 4.2, 4.3, 4.4, 4.5, 4.6, 4.7

`TorCircuitInfo(circuitInfo, onNewIdentity)` pantalla	Displays current Tor circuit information Shows: - Entry, middle, and exit nodes - Circuit ID - Bandwidth information - "New Identity" button to create a new circuit Requirements: 17.6, 17.7, 17.8, 4.1, 4.2, 4.3, 4.4, 4.5, 4.6, 4.7
`CircuitInfoRow(label, value, description)` pantalla interna	Row displaying circuit information
`formatBandwidth(bytes)` interna	Formats bandwidth in human-readable format

TorControlScreen.kt

`ui/screens/security/TorControlScreen.kt · 274 líneas`

Modo anónimo. Lo que aparece aquí sale del propio Tor (progreso y circuito), del Proyecto Tor (IP de salida verificada) y del enrutado real (qué ha salido ya por Tor).

`TorControlScreen(navController, viewModel)` pantalla	Modo anónimo. Lo que aparece aquí sale del propio Tor (progreso y circuito), del Proyecto Tor (IP de salida verificada) y del enrutado real (qué ha salido ya por Tor).
`statusText(state)` interna	Sin comentario en el código. Por su nombre: status text.
`ConnectionDetails(info)` pantalla interna	Dibuja en pantalla: connection details.
`EngineOption(title, subtitle, selected, onClick)` pantalla interna	Dibuja en pantalla: engine option.
`CoverageRow(label, covered, note)` pantalla interna	Dibuja en pantalla: coverage row.

Parte 12

Registro e inicio de sesión

Correo, Google y teléfono.

AuthComponents.kt

`ui/screens/auth/AuthComponents.kt · 176 líneas`

Marca en las pantallas de entrada.

`AuthBrand(subtitle, modifier)` pantalla	Marca en las pantallas de entrada.
`authFieldColors()` pantalla	Dibuja en pantalla: auth field colors.
`AuthField(value, onValueChange)` pantalla	Dibuja en pantalla: auth field.
`AuthOutlinedAction(text, onClick)` pantalla	Dibuja en pantalla: auth outlined action.
`GoogleMark()` pantalla	Letra de Google a modo de icono, sin reproducir su logotipo.
`AuthDivider()` pantalla	Dibuja en pantalla: auth divider.
`AuthError(message)` pantalla	Dibuja en pantalla: auth error.
`AnonymousModeLoginNotice(onDisable)` pantalla	Con el modo anónimo activo la sesión pasa por Tor, que puede estar conectando todavía. Sin esta salida, alguien que no pudiera entrar no tendría cómo apagarlo.

GoogleIdTokenProvider.kt

`ui/screens/auth/GoogleIdTokenProvider.kt · 99 líneas`

Pide a Google una sesión con el selector de cuentas del sistema. Necesita el contexto de la pantalla: con el de la aplicación, Android no puede mostrar el selector encima de la app y la petición falla sin que se vea nada.

`request(activity)` asíncrona	Sin comentario en el código. Por su nombre: request.
`getCredential(activity, authorizedOnly)` interna asíncrona	Sin comentario en el código. Por su nombre: get credential.
`signatureHint(context)` interna	En compilaciones de desarrollo dice qué huella hay que registrar en Firebase.
`appSignatureSha1(context)`	Huella SHA-1 del certificado con el que está firmada la app instalada.

LoginScreen.kt

`ui/screens/auth/LoginScreen.kt · 136 líneas`

`LoginScreen(navController, viewModel)` pantalla	Dibuja en pantalla: login screen.

LoginViewModel.kt

`ui/screens/auth/LoginViewModel.kt · 142 líneas`

activity debe ser la pantalla: el selector de cuentas se muestra sobre ella.

`onEmailChange(email)`	Sin comentario en el código. Por su nombre: on email change.
`onPasswordChange(password)`	Sin comentario en el código. Por su nombre: on password change.
`signInWithGoogle(activity)`	activity debe ser la pantalla: el selector de cuentas se muestra sobre ella.
`login()`	Sin comentario en el código. Por su nombre: login.
`sendPasswordReset()`	Envía el enlace para crear una contraseña nueva. Sirve también para quien entró siempre con Google y nunca llegó a tener una.

`onSignedIn()` interna asíncrona	Sin comentario en el código. Por su nombre: on signed in.
`disableAnonymousMode()`	Sin comentario en el código. Por su nombre: disable anonymous mode.
`clearError()`	Sin comentario en el código. Por su nombre: clear error.

PhoneLoginScreen.kt

`ui/screens/auth/PhoneLoginScreen.kt · 268 líneas`

Acceso con número de teléfono. Dos pasos en una sola pantalla (número → código) en lugar de dos destinos de navegación: volver atrás desde el código debe llevar al número sin perder lo escrito, y con dos rutas eso obliga a subir el estado o a duplicarlo.

`PhoneLoginScreen(navController, viewModel)` pantalla	Acceso con número de teléfono. Dos pasos en una sola pantalla (número → código) en lugar de dos destinos de navegación: volver atrás desde el código debe llevar al número sin perder lo escrito, y con dos rutas eso obliga a subir el estado o a duplicarlo.
`nexusFieldColors()` pantalla interna	Dibuja en pantalla: nexus field colors.

PhoneLoginViewModel.kt

`ui/screens/auth/PhoneLoginViewModel.kt · 251 líneas`

Paso del flujo de acceso por teléfono.

`onCountryCodeChange(value)`	Sin comentario en el código. Por su nombre: on country code change.
`onPhoneNumberChange(value)`	Sin comentario en el código. Por su nombre: on phone number change.
`onCodeChange(value)`	Sin comentario en el código. Por su nombre: on code change.
`backToPhoneStep()`	Sin comentario en el código. Por su nombre: back to phone step.
`sendVerificationCode(activity, resend)`	Pide el SMS. resend fuerza un envío nuevo reusando el token de reenvío.
`verifyCode()`	Comprueba el código que ha escrito el usuario.
`onVerificationCompleted(credential)`	Verificación instantánea: en algunos dispositivos Android lee el SMS o reconoce el número sin que el usuario escriba nada.
`onVerificationFailed(e)`	Sin comentario en el código. Por su nombre: on verification failed.
`says(words)`	Sin comentario en el código. Por su nombre: says.
`onCodeSent(id, token)`	Sin comentario en el código. Por su nombre: on code sent.
`signIn(credential)` interna	Sin comentario en el código. Por su nombre: sign in.
`startResendCooldown()` interna	Cuenta atrás antes de permitir reenviar. Sin ella el usuario pulsa "Reenviar" varias veces seguidas y Firebase acaba bloqueando el número por abuso, que es un error del que no se sale en horas.
`clearError()`	Sin comentario en el código. Por su nombre: clear error.
`onCleared()`	Sin comentario en el código. Por su nombre: on cleared.

RegisterScreen.kt

`ui/screens/auth/RegisterScreen.kt · 132 líneas`

`RegisterScreen(navController, viewModel)` pantalla	Dibuja en pantalla: register screen.

RegisterViewModel.kt

`ui/screens/auth/RegisterViewModel.kt · 111 líneas`

activity debe ser la pantalla: el selector de cuentas se muestra sobre ella.

`onDisplayNameChange(displayName)`	Sin comentario en el código. Por su nombre: on display name change.
`onEmailChange(email)`	Sin comentario en el código. Por su nombre: on email change.
`onPasswordChange(password)`	Sin comentario en el código. Por su nombre: on password change.
`onConfirmPasswordChange(confirmPassword)`	Sin comentario en el código. Por su nombre: on confirm password change.
`register()`	Sin comentario en el código. Por su nombre: register.
`signInWithGoogle(activity)`	activity debe ser la pantalla: el selector de cuentas se muestra sobre ella.
`onSignedIn()` interna asíncrona	Sin comentario en el código. Por su nombre: on signed in.
`clearError()`	Sin comentario en el código. Por su nombre: clear error.

Parte 13

Inteligencia artificial

Chat con la IA y agentes.

AzelAIRepository.kt

`ui/screens/azelai/AzelAIRepository.kt · 286 líneas`

🚀 AZEL AI REPOSITORY - REPOSITORIO AVANZADO CON STREAMING Gestiona la comunicación con la API y Firebase

`buildApiMessages(conversationHistory, userMessage)` interna	Sin comentario en el código. Por su nombre: build api messages.
`sendMessageStream(userMessage, conversationHistory, model)`	🔥 ENVIAR MENSAJE CON STREAMING EN TIEMPO REAL
`sendMessage(userMessage, conversationHistory, model)` asíncrona	💬 ENVIAR MENSAJE SIN STREAMING (FALLBACK)
`getAvailableModels()`	📊 OBTENER MODELOS DISPONIBLES
`checkApiHealth()` asíncrona	🔍 VERIFICAR SALUD DE LA API
`isProviderConfigured()`	true si el proveedor activo tiene clave/URL suficientes para responder.
`activeProviderSummary(selectedGeminiModel)`	(Proveedor, modelo) activos para mostrarlos en la cabecera.
`getMessageHistory(chatPath)`	Obtener historial en tiempo real desde Firebase
`saveMessage(chatPath, message)` asíncrona	Guardar mensaje en Firebase (historial persistente)
`getChatHistory(userId)` asíncrona	Obtener historial de chats
`updateChatHistory(userId, chatId, title, lastMessage)` asíncrona	Actualizar historial de chat
`clearHistory(userId)` asíncrona	Limpiar historial de conversación
`getChatStats(userId)` asíncrona	Sin comentario en el código. Por su nombre: get chat stats.

AzelAIScreen.kt

`ui/screens/azelai/AzelAIScreen.kt · 876 líneas`

`AzelAIScreen(onBack)` pantalla	Dibuja en pantalla: azel aiscreen.
`AzelAITopBar(onBack)` pantalla	Dibuja en pantalla: azel aitop bar.
`AzelAIWelcome(isConfigured, onOpenSettings)` pantalla	Dibuja en pantalla: azel aiwelcome.
`AzelAIQuickTools(onPick)` pantalla	Dibuja en pantalla: azel aiquick tools.
`AzelAIMessageBubble(message)` pantalla	Dibuja en pantalla: azel aimessage bubble.
`AzelAIInputBar(value, onValueChange)` pantalla	Dibuja en pantalla: azel aiinput bar.
`ErrorBanner(error, onClose)` pantalla	Dibuja en pantalla: error banner.
`AzelAIThinkingBubble(elapsedMs, timeoutMs)` pantalla	Dibuja en pantalla: azel aithinking bubble.
`AzelConfirmDialog(title, desc, onConfirm)` pantalla	Dibuja en pantalla: azel confirm dialog.
`AzelStatsDialog(stats, onDismiss)` pantalla	Dibuja en pantalla: azel stats dialog.
`StatItem(label, value)` pantalla	Dibuja en pantalla: stat item.

`AzelAISidebar(modifier, searchQuery, onSearchChange)` pantalla	Dibuja en pantalla: azel aisidebar.

AzelAIViewModel.kt

`ui/screens/azelai/AzelAIViewModel.kt · 502 líneas`

🚀 AZEL AI VIEW MODEL

`currentUserId()` interna	Sin comentario en el código. Por su nombre: current user id.
`loadAvailableModels()` interna	📊 CARGAR MODELOS DISPONIBLES
`selectModel(modelId)`	🔄 CAMBIAR MODELO
`refreshProviderStatus()`	Relee proveedor, modelo y si hay clave. La pantalla lo llama al volver de los ajustes de IA para que la cabecera refleje el cambio al momento.
`checkApiHealth()` interna	🔍 VERIFICAR SALUD DE LA API
`loadMessages()` interna	📥 CARGAR MENSAJES DEL CHAT ACTUAL
`loadChatHistory()` interna	📜 CARGAR HISTORIAL DE CHATS
`loadStats()` interna	📊 CARGAR ESTADÍSTICAS
`startNewChat()`	🆕 INICIAR NUEVO CHAT
`loadChat(chatId)`	📂 CARGAR CHAT ESPECÍFICO
`updateChatTitle(firstMessage)` interna	📝 ACTUALIZAR TÍTULO DEL CHAT
`sendMessage(content, useStreaming)`	💬 ENVIAR MENSAJE CON STREAMING Plazos: antes había un tope FIJO de 60 s para toda la respuesta, así que una respuesta larga que seguía llegando se cortaba a la mitad con "Tiempo de espera agotado". Ahora el límite es de inactividad: FIRST_TOKEN_TIMEOUT_MS para el primer trozo y IDLE_TIMEOUT_MS entre trozos; mientras el modelo siga escribiendo, la respuesta no se corta.
`importAgentResult(goal, answer)`	Trae el resultado de los agentes a la conversación actual, para poder seguir preguntando sobre él en el chat ("Seguir en el chat").
`stopGeneration()`	Corta la respuesta en curso (botón "Detener").
`showSendError(errorMessage)` interna	Sin comentario en el código. Por su nombre: show send error.
`mapError(e)` interna	Sin comentario en el código. Por su nombre: map error.
`clearHistory()`	🗑️ LIMPIAR HISTORIAL
`clearError()`	❌ LIMPIAR ERROR
`retryLastMessage()`	🔄 REINTENTAR ÚLTIMO MENSAJE Quita el mensaje de error y la pregunta que lo provocó antes de reenviarla: antes la pregunta se añadía otra vez y el historial quedaba duplicado.
`timeoutBetweenChunks(firstMs, idleMs, isFirst)`	Lanza AiResponseTimeoutException si el flujo pasa más de firstMs sin emitir nada al principio, o más de idleMs entre dos trozos. Es una excepción normal y no la `TimeoutCancellationException` de `withTimeout` a propósito: esa es una cancelación y, al cruzar el canal, se confundiría con que el usuario pulsó "Detener".

AgentsPanel.kt

`ui/screens/azelai/agents/AgentsPanel.kt · 1209 líneas`

Pestaña "Agentes" de Azel IA. Sin ejecución en curso muestra la configuración (agente o equipo, objetivo, ejemplos e historial). Con una ejecución, el "espacio de trabajo": los agentes como nodos animados que se iluminan al pensar o usar herramientas, el plan del equipo, el registro en vivo de lo que hacen y el resultado final.

`AgentsPanel(geminiModel, onOpenSettings)` pantalla	Pestaña "Agentes" de Azel IA. Sin ejecución en curso muestra la configuración (agente o equipo, objetivo, ejemplos e historial). Con una ejecución, el "espacio de trabajo": los agentes como nodos animados que se iluminan al pensar o usar herramientas, el plan del equipo, el registro en vivo de lo que hacen y el resultado final.
`AgentSetup(ui, onTeamMode)` pantalla interna	Dibuja en pantalla: agent setup.
`ProviderBanner(onOpenSettings)` pantalla interna	Dibuja en pantalla: provider banner.
`ModeSwitch(isTeam, onChange)` pantalla interna	Dibuja en pantalla: mode switch.
`AgentCard(agent, selected, onClick)` pantalla interna	Dibuja en pantalla: agent card.
`TeamCard(team, selected, onClick)` pantalla interna	Dibuja en pantalla: team card.
`GoalInput(goal, onGoalChange)` pantalla interna	Dibuja en pantalla: goal input.
`HistoryRow(summary, onClick)` pantalla interna	Dibuja en pantalla: history row.
`AgentWorkspace(run, onCancel)` pantalla interna	Dibuja en pantalla: agent workspace.
`RunHeader(run, onCancel)` pantalla interna	Dibuja en pantalla: run header.
`StatusPill(status, elapsedMs)` pantalla interna	Dibuja en pantalla: status pill.
`AgentStage(nodes)` pantalla interna	Los agentes en fila, unidos por conectores; el que trabaja brilla.
`Connector(active, reached, color)` pantalla interna	Dibuja en pantalla: connector.
`AgentNodeView(node, wide)` pantalla interna	Dibuja en pantalla: agent node view.
`AgentAvatar(agent, size)` pantalla	Dibuja en pantalla: agent avatar.
`PlanCard(plan, currentStep, agentsById, running)` pantalla interna	Dibuja en pantalla: plan card.
`EventRow(event, agent, modifier)` pantalla interna	Dibuja en pantalla: event row.
`ExpandableBlock(title, body, accent, monospace, markdown)` pantalla interna	Dibuja en pantalla: expandable block.
`WorkingIndicator(run)` pantalla interna	Dibuja en pantalla: working indicator.
`SavedFilesCard(files, onOpenEditor)` pantalla interna	Dibuja en pantalla: saved files card.
`ResultCard(answer, animate, onSendToChat)` pantalla interna	Dibuja en pantalla: result card.
`StatusMessage(icon, color, title, text, primaryLabel, onPrimary)` pantalla interna	Dibuja en pantalla: status message.
`wordCount(text)` interna	Sin comentario en el código. Por su nombre: word count.
`formatElapsed(ms)` interna	Sin comentario en el código. Por su nombre: format elapsed.
`AgentFilePermissionDialog(exporter)` pantalla interna	Un agente nunca guarda fuera de la app sin que el usuario vea qué y dónde, y lo apruebe.

AgentsViewModel.kt

`ui/screens/azelai/agents/AgentsViewModel.kt · 254 líneas`

Ejecución guardada en el historial (lo que se ve en "Recientes").

`refreshProvider()`	Sin comentario en el código. Por su nombre: refresh provider.

`setTeamMode(team)`	Sin comentario en el código. Por su nombre: set team mode.
`selectAgent(id)`	Sin comentario en el código. Por su nombre: select agent.
`selectTeam(id)`	Sin comentario en el código. Por su nombre: select team.
`start(goal, geminiModel)`	Sin comentario en el código. Por su nombre: start.
`cancel()`	Detiene la ejecución en curso; lo hecho hasta ahora sigue en pantalla.
`close()`	Vuelve a la pantalla de configuración.
`openFromHistory(summary)`	Abre una ejecución del historial (solo lectura).
`teamRoster(team)` interna	Sin comentario en el código. Por su nombre: team roster.
`friendlyError(e)` interna	Sin comentario en el código. Por su nombre: friendly error.
`historyRef()` interna	Sin comentario en el código. Por su nombre: history ref.
`loadHistory()` interna	Sin comentario en el código. Por su nombre: load history.
`saveToHistory(state)` interna	Sin comentario en el código. Por su nombre: save to history.

Parte 14

Editor de código

Editor con pestañas y resaltado.

CodeEditorScreen.kt

`ui/screens/editor/CodeEditorScreen.kt · 801 líneas`

Editor basado en Sora Editor: numeración de líneas, deshacer ilimitado, autocompletado, buscar y reemplazar, pestañas y guardado automático.

`CodeEditorScreen(navController, viewModel)` pantalla	Editor basado en Sora Editor: numeración de líneas, deshacer ilimitado, autocompletado, buscar y reemplazar, pestañas y guardado automático.
`currentText()`	Sin comentario en el código. Por su nombre: current text.
`log(message)`	Sin comentario en el código. Por su nombre: log.
`error(message)`	Sin comentario en el código. Por su nombre: error.
`EditorToolKey(icon, description, onClick)` pantalla interna	Dibuja en pantalla: editor tool key.
`EditorMenuItem(label, icon, onClick)` pantalla interna	Dibuja en pantalla: editor menu item.
`FileList(files, onBack)` pantalla interna	Dibuja en pantalla: file list.
`formatSize(bytes)` interna	Sin comentario en el código. Por su nombre: format size.
`SearchBar(editor)` pantalla interna	Dibuja en pantalla: search bar.
`EditorField(value, onValueChange)` pantalla interna	Dibuja en pantalla: editor field.
`NameDialog(title, initial, hint, confirm, onDismiss)` pantalla interna	Dibuja en pantalla: name dialog.

CodeEditorViewModel.kt

`ui/screens/editor/CodeEditorViewModel.kt · 371 líneas`

Editor de código: pestañas, guardado automático, importar y exportar, y ejecución. Los archivos se guardan en la cuenta (Firebase) para tenerlos en cualquier dispositivo, y se pueden abrir o guardar en el almacenamiento del teléfono. Qué se ejecuta de verdad: JavaScript (motor del WebView), HTML/CSS (vista previa), shell (proceso real del sistema) y JSON (validar y formatear). Python, Kotlin, C o TypeScript necesitan un compilador o intérprete que Android no incluye; se dice claramente en vez de fingir que corren.

`loadFiles()` interna	Sin comentario en el código. Por su nombre: load files.
`onCleared()`	Sin comentario en el código. Por su nombre: on cleared.
`push(file)` interna	Sin comentario en el código. Por su nombre: push.
`newFile(name)`	Sin comentario en el código. Por su nombre: new file.
`importDocument(uri)`	Abre un archivo del teléfono en una pestaña nueva.
`exportTo(uri, content)`	Guarda el archivo abierto en el teléfono.
`openFile(file)`	Sin comentario en el código. Por su nombre: open file.
`closeTab(fileId, latestContent)`	Cierra una pestaña guardando antes lo pendiente.
`closeAllTabs()`	Sin comentario en el código. Por su nombre: close all tabs.
`onContentChanged(fileId, content)`	El usuario ha escrito: se guarda solo poco después de dejar de teclear.
`saveFile(fileId, content)`	Sin comentario en el código. Por su nombre: save file.

`renameFile(fileId, newName)`	Sin comentario en el código. Por su nombre: rename file.
`deleteFile(file)`	Sin comentario en el código. Por su nombre: delete file.
`executeCode(code, language)`	Sin comentario en el código. Por su nombre: execute code.
`needsTool(what, command)` interna	Sin comentario en el código. Por su nombre: needs tool.
`runShell(script)` interna	Ejecuta el script en una shell real y muestra la salida a medida que llega.
`stopExecution()`	Sin comentario en el código. Por su nombre: stop execution.
`clearHtmlPreview()`	Sin comentario en el código. Por su nombre: clear html preview.
`wrapCssInHtml(css)` interna	Sin comentario en el código. Por su nombre: wrap css in html.
`onJsResult(result, error)`	Sin comentario en el código. Por su nombre: on js result.
`formatJson(code)` interna	Sin comentario en el código. Por su nombre: format json.
`template(language)` interna	Sin comentario en el código. Por su nombre: template.
`main()`	Sin comentario en el código. Por su nombre: main.

CodeFile.kt

`ui/screens/editor/CodeFile.kt · 11 líneas`

NexusCodeLanguage.kt

`ui/screens/editor/NexusCodeLanguage.kt · 351 líneas`

Idiomas que el editor reconoce por la extensión del archivo.

`fromFileName(name)`	Sin comentario en el código. Por su nombre: from file name.
`label(language)`	Sin comentario en el código. Por su nombre: label.
`nexusEditorScheme(accent)`	Colores del editor, sobre la misma superficie que el resto de la app.
`getAnalyzeManager()`	Sin comentario en el código. Por su nombre: get analyze manager.
`getInterruptionLevel()`	Sin comentario en el código. Por su nombre: get interruption level.
`getSymbolPairs()`	Sin comentario en el código. Por su nombre: get symbol pairs.
`useTab()`	Sin comentario en el código. Por su nombre: use tab.
`getIndentAdvance(content, line, column)`	Sin comentario en el código. Por su nombre: get indent advance.
`requireAutoComplete(content, position, publisher, extraArguments)`	Sin comentario en el código. Por su nombre: require auto complete.
`analyze(text, delegate)`	Sin comentario en el código. Por su nombre: analyze.
`advance(style, count)`	Sin comentario en el código. Por su nombre: advance.
`emit(style, count)`	Sin comentario en el código. Por su nombre: emit.
`keywordsFor(language)`	Sin comentario en el código. Por su nombre: keywords for.

Parte 15

Terminal

Shell del teléfono y Kali Linux.

TerminalScreen.kt

`ui/screens/terminal/TerminalScreen.kt · 434 líneas`

Terminal real: emulador VT100/xterm conectado a una shell del sistema. Dos tipos de sesión: la shell de Android, con los comandos del sistema, y Kali Linux, un sistema completo con `apt` que se instala bajo petición (ver LinuxEnvironment).

`TerminalScreen(onBack)` pantalla	Terminal real: emulador VT100/xterm conectado a una shell del sistema. Dos tipos de sesión: la shell de Android, con los comandos del sistema, y Kali Linux, un sistema completo con `apt` que se instala bajo petición (ver LinuxEnvironment).
`clipboard()`	Sin comentario en el código. Por su nombre: clipboard.
`pasteInto(session)`	Sin comentario en el código. Por su nombre: paste into.
`onTextChanged(session)`	Sin comentario en el código. Por su nombre: on text changed.
`onTitleChanged(session)`	Sin comentario en el código. Por su nombre: on title changed.
`onSessionFinished(session)`	Sin comentario en el código. Por su nombre: on session finished.
`onCopyTextToClipboard(session, text)`	Sin comentario en el código. Por su nombre: on copy text to clipboard.
`onPasteTextFromClipboard(session)`	Sin comentario en el código. Por su nombre: on paste text from clipboard.
`onBell(session)`	Sin comentario en el código. Por su nombre: on bell.
`onColorsChanged(session)`	Sin comentario en el código. Por su nombre: on colors changed.
`onTerminalCursorStateChange(state)`	Sin comentario en el código. Por su nombre: on terminal cursor state change.
`getTerminalCursorStyle()`	Sin comentario en el código. Por su nombre: get terminal cursor style.
`logError(tag, message)`	Sin comentario en el código. Por su nombre: log error.
`logWarn(tag, message)`	Sin comentario en el código. Por su nombre: log warn.
`logInfo(tag, message)`	Sin comentario en el código. Por su nombre: log info.
`logDebug(tag, message)`	Sin comentario en el código. Por su nombre: log debug.
`logVerbose(tag, message)`	Sin comentario en el código. Por su nombre: log verbose.
`logStackTraceWithMessage(tag, message, e)`	Sin comentario en el código. Por su nombre: log stack trace with message.
`logStackTrace(tag, e)`	Sin comentario en el código. Por su nombre: log stack trace.
`send(text)`	Sin comentario en el código. Por su nombre: send.
`key(code)`	Sin comentario en el código. Por su nombre: key.
`TermKey(label, active, onClick)` pantalla interna	Dibuja en pantalla: term key.
`onScale(scale)`	Sin comentario en el código. Por su nombre: on scale.
`onSingleTapUp(e)`	Sin comentario en el código. Por su nombre: on single tap up.
`shouldBackButtonBeMappedToEscape()`	Sin comentario en el código. Por su nombre: should back button be mapped to escape.
`shouldEnforceCharBasedInput()`	Sin comentario en el código. Por su nombre: should enforce char based input.
`shouldUseCtrlSpaceWorkaround()`	Sin comentario en el código. Por su nombre: should use ctrl space workaround.
`isTerminalViewSelected()`	Sin comentario en el código. Por su nombre: is terminal view selected.
`copyModeChanged(copyMode)`	Sin comentario en el código. Por su nombre: copy mode changed.
`onKeyDown(keyCode, e, session)`	Sin comentario en el código. Por su nombre: on key down.
`onKeyUp(keyCode, e)`	Sin comentario en el código. Por su nombre: on key up.

`onLongPress(event)`	Sin comentario en el código. Por su nombre: on long press.
`readControlKey()`	Sin comentario en el código. Por su nombre: read control key.
`readAltKey()`	Sin comentario en el código. Por su nombre: read alt key.
`readShiftKey()`	Sin comentario en el código. Por su nombre: read shift key.
`readFnKey()`	Sin comentario en el código. Por su nombre: read fn key.
`onCodePoint(codePoint, ctrlDown, session)`	Sin comentario en el código. Por su nombre: on code point.
`onEmulatorSet()`	Sin comentario en el código. Por su nombre: on emulator set.
`LinuxPanel(state, onInstall)` pantalla interna	Dibuja en pantalla: linux panel.

TerminalViewModel.kt

`ui/screens/terminal/TerminalViewModel.kt · 155 líneas`

Sesiones de terminal. Son shells reales sobre una pseudoterminal (PTY) del sistema: `/system/bin/sh` con un proceso vivo. Se conserva el estado entre comandos (`cd`, `export`), funcionan los programas interactivos (`top`, `vi`), Ctrl+C interrumpe y los colores se ven. La versión anterior lanzaba cada comando por separado y esperaba a que terminara antes de mostrar nada, con un tope de 60 segundos. Viven en el ViewModel para sobrevivir a rotar la pantalla o abrir otra un momento.

`installLinux()`	Sin comentario en el código. Por su nombre: install linux.
`uninstallLinux()`	Sin comentario en el código. Por su nombre: uninstall linux.
`isLinuxSession(session)`	Sin comentario en el código. Por su nombre: is linux session.
`newLinuxSession()`	Sin comentario en el código. Por su nombre: new linux session.
`onTextChanged(session)`	Sin comentario en el código. Por su nombre: on text changed.
`onTitleChanged(session)`	Sin comentario en el código. Por su nombre: on title changed.
`onSessionFinished(session)`	Sin comentario en el código. Por su nombre: on session finished.
`onCopyTextToClipboard(session, text)`	Sin comentario en el código. Por su nombre: on copy text to clipboard.
`onPasteTextFromClipboard(session)`	Sin comentario en el código. Por su nombre: on paste text from clipboard.
`onBell(session)`	Sin comentario en el código. Por su nombre: on bell.
`onColorsChanged(session)`	Sin comentario en el código. Por su nombre: on colors changed.
`onTerminalCursorStateChange(state)`	Sin comentario en el código. Por su nombre: on terminal cursor state change.
`getTerminalCursorStyle()`	Sin comentario en el código. Por su nombre: get terminal cursor style.
`logError(tag, message)`	Sin comentario en el código. Por su nombre: log error.
`logWarn(tag, message)`	Sin comentario en el código. Por su nombre: log warn.
`logInfo(tag, message)`	Sin comentario en el código. Por su nombre: log info.
`logDebug(tag, message)`	Sin comentario en el código. Por su nombre: log debug.
`logVerbose(tag, message)`	Sin comentario en el código. Por su nombre: log verbose.
`logStackTraceWithMessage(tag, message, e)`	Sin comentario en el código. Por su nombre: log stack trace with message.
`logStackTrace(tag, e)`	Sin comentario en el código. Por su nombre: log stack trace.
`newSession()`	Sin comentario en el código. Por su nombre: new session.
`select(session)`	Sin comentario en el código. Por su nombre: select.
`close(session)`	Sin comentario en el código. Por su nombre: close.
`removeFinished(session)`	Sin comentario en el código. Por su nombre: remove finished.
`onCleared()`	Sin comentario en el código. Por su nombre: on cleared.

Parte 16

Descargas

Descargador de vídeos.

VideoDownloaderScreen.kt

`ui/screens/downloader/VideoDownloaderScreen.kt · 245 líneas`

Descarga con yt-dlp, integrado en la app: YouTube, Facebook, X, Instagram, TikTok y más.

`download(url, audioOnly)`	Sin comentario en el código. Por su nombre: download.
`prepare()` interna	Sin comentario en el código. Por su nombre: prepare.
`publish(file, audioOnly)` interna	Lo mueve a Descargas/NexusChat, donde el usuario puede encontrarlo con cualquier app.
`friendly(raw)` interna	Sin comentario en el código. Por su nombre: friendly.
`VideoDownloaderScreen(navController, initialUrl, viewModel)` pantalla	Dibuja en pantalla: video downloader screen.

Parte 17

Otras pantallas

Resto de pantallas.

BackgroundPickerScreen.kt

`ui/screens/background/BackgroundPickerScreen.kt · 879 líneas`

Background picker screen - Enhanced Chat Wallpaper Features: - Top tabs for App/Chat scope - Live preview with chat bubble mockup (40% of screen) - Type selector (None, Color, Image, Video, Gradient) - Beautiful gradient presets organized by category - Overlay opacity slider - Apply/Cancel buttons

`BackgroundPickerScreen(navController, chatId, viewModel)` pantalla	Background picker screen - Enhanced Chat Wallpaper Features: - Top tabs for App/Chat scope - Live preview with chat bubble mockup (40% of screen) - Type selector (None, Color, Image, Video, Gradient) - Beautiful gradient presets organized by category - Overlay opacity slider - Apply/Cancel buttons
`BackgroundPreview(config)` pantalla interna	Dibuja en pantalla: background preview.
`TypeSelector(selectedType, onTypeSelected)` pantalla interna	Dibuja en pantalla: type selector.
`TypeChip(text, selected, onClick)` pantalla interna	Dibuja en pantalla: type chip.
`ColorContent(selectedColor, onColorSelected)` pantalla interna	Dibuja en pantalla: color content.
`ImageContent(onPickImage)` pantalla interna	Dibuja en pantalla: image content.
`VideoContent(onPickVideo)` pantalla interna	Dibuja en pantalla: video content.
`GradientContent(color1, color2, angle, onColor1Click)` pantalla interna	Dibuja en pantalla: gradient content.
`ColorButton(label, colorHex, onClick)` pantalla interna	Dibuja en pantalla: color button.
`BlurContent(blurRadius, onBlurChange)` pantalla interna	Dibuja en pantalla: blur content.
`OverlaySlider(alpha, onAlphaChange)` pantalla interna	Dibuja en pantalla: overlay slider.

BackgroundPickerViewModel.kt

`ui/screens/background/BackgroundPickerViewModel.kt · 231 líneas`

ViewModel for background picker screen Manages: - Background configuration state - App-wide or per-chat scope - Apply changes to AppBackgroundManager or ChatBackgroundRepository

`clearError()`	Sin comentario en el código. Por su nombre: clear error.
`loadCurrentBackground()` interna	Sin comentario en el código. Por su nombre: load current background.
`updateConfig(config)`	Sin comentario en el código. Por su nombre: update config.
`setType(type)`	Sin comentario en el código. Por su nombre: set type.
`setSolidColor(colorHex)`	Sin comentario en el código. Por su nombre: set solid color.
`pickImage(uri)`	Sin comentario en el código. Por su nombre: pick image.
`pickVideo(uri)`	Sin comentario en el código. Por su nombre: pick video.
`setGradient(colors, angle)`	Sin comentario en el código. Por su nombre: set gradient.
`setBlurRadius(radius)`	Sin comentario en el código. Por su nombre: set blur radius.

`setOverlayAlpha(alpha)`	Sin comentario en el código. Por su nombre: set overlay alpha.
`applyBackground(onComplete)`	Sin comentario en el código. Por su nombre: apply background.
`uploadLocalMediaIfNeeded(config)` interna asíncrona	Sube a Firebase Storage la imagen/video elegido de galería (content://) y reemplaza el URI local por la URL de descarga. Sin esto el URI local caduca al reiniciar la app o al revocarse el permiso → el fondo desaparecía y, en chats, no era visible para el otro participante. En scope CHAT reutiliza las rutas de media del chat; en scope APP usa una ruta propia del usuario (user_backgrounds/{uid}).
`checkFileSize(uri, maxBytes, message)` interna	Lanza IllegalArgumentException con mensaje legible si el archivo excede el límite. El catch de applyBackground lo muestra en _error. Si el tamaño no se puede determinar (statSize = -1) se permite subir: el límite es proteccion de UX, no de seguridad (esa la ponen las reglas de Storage).
`clearBackground()`	Sin comentario en el código. Por su nombre: clear background.

CallsScreen.kt

`ui/screens/calls/CallsScreen.kt · 408 líneas`

Texto legible del estado guardado en la base de datos.

`CallsScreen(navController, viewModel)` pantalla	Dibuja en pantalla: calls screen.
`CallItemRow(call, onCallClick)` pantalla	Dibuja en pantalla: call item row.
`CallItemRowBody(call, onCallClick)` pantalla interna	Dibuja en pantalla: call item row body.
`formatCallTime(timestamp)` interna	Sin comentario en el código. Por su nombre: format call time.
`callStatusLabel(status)` interna	Texto legible del estado guardado en la base de datos.

CallsViewModel.kt

`ui/screens/calls/CallsViewModel.kt · 150 líneas`

`loadCallHistory()` interna	Sin comentario en el código. Por su nombre: load call history.
`buildCallHistoryItem(data, currentUserId)` interna asíncrona	Sin comentario en el código. Por su nombre: build call history item.
`clearError()`	Sin comentario en el código. Por su nombre: clear error.

NewConversationScreen.kt

`ui/screens/conversation/NewConversationScreen.kt · 680 líneas`

`NewConversationScreen(navController, viewModel)` pantalla	Dibuja en pantalla: new conversation screen.
`QuickActionCard(icon, text, onClick)` pantalla	Dibuja en pantalla: quick action card.
`ContactRow(contact, onClick)` pantalla	Dibuja en pantalla: contact row.
`ContactRowBody(contact, onClick)` pantalla interna	Dibuja en pantalla: contact row body.
`AddContactSheet(onDismiss)` pantalla	Dibuja en pantalla: add contact sheet.
`NewGroupSheet(contacts, onDismiss)` pantalla	Dibuja en pantalla: new group sheet.

NewConversationViewModel.kt

`ui/screens/conversation/NewConversationViewModel.kt · 319 líneas`

Start or resume a 1-to-1 conversation. Chat document lives at chats/{chatId} where chatId is the two UIDs sorted alphabetically and joined with "_". This is the same path that ChatViewModel and HomeScreen read from, so there is no double-prefixing. After ensuring the document exists the user is navigated to "chat/{chatId}" — NOT "chat/{otherUid}".

`loadContacts()` interna	Sin comentario en el código. Por su nombre: load contacts.
`searchContacts(query)`	Sin comentario en el código. Por su nombre: search contacts.
`startConversation(otherUid, navController)`	Start or resume a 1-to-1 conversation. Chat document lives at chats/{chatId} where chatId is the two UIDs sorted alphabetically and joined with "_". This is the same path that ChatViewModel and HomeScreen read from, so there is no double-prefixing. After ensuring the document exists the user is navigated to "chat/{chatId}" — NOT "chat/{otherUid}".
`clearError()`	Limpia el error actual tras mostrarlo en la UI.
`createGroup(groupName, selectedUserIds)` asíncrona	Create a new group chat and return its ID.
`searchUserByUsername(username)` asíncrona	Search for a user by their username handle.
`sendFriendRequest(toUserId)` asíncrona	Send a friend / connection request to another user.
`createDemoChat(navController)`	Create demo chat with Azel Assistant

QRScannerScreen.kt

`ui/screens/conversation/QRScannerScreen.kt · 448 líneas`

`QRScannerScreen(navController, onQRScanned)` pantalla	Dibuja en pantalla: qrscanner screen.
`CameraPreview(onQRCodeDetected)` pantalla	Dibuja en pantalla: camera preview.
`ScanningOverlay()` pantalla	Dibuja en pantalla: scanning overlay.
`parseQRCode(qrContent)`	Sin comentario en el código. Por su nombre: parse qrcode.

PremiumScreen.kt

`ui/screens/premium/PremiumScreen.kt · 407 líneas`

Emblema con un anillo dorado que gira: es lo primero que se ve y marca el tono de la pantalla.

`PremiumScreen(navController, viewModel)` pantalla	Dibuja en pantalla: premium screen.
`activeDescription(expiresAt)` interna	Sin comentario en el código. Por su nombre: active description.
`PremiumEmblem(active)` pantalla interna	Emblema con un anillo dorado que gira: es lo primero que se ve y marca el tono de la pantalla.
`Benefit(icon, title, text, index)` pantalla interna	Dibuja en pantalla: benefit.
`PlanRow(plan, selected, onClick)` pantalla interna	Dibuja en pantalla: plan row.
`UnavailableCard(message, onRetry)` pantalla interna	Dibuja en pantalla: unavailable card.
`ActivationCelebration(onDone)` pantalla interna	Confeti dorado de un par de segundos cuando el pago se confirma con la pantalla abierta.

PremiumViewModel.kt

`ui/screens/premium/PremiumViewModel.kt · 100 líneas`

`loadPlans()`	Sin comentario en el código. Por su nombre: load plans.

`selectPlan(id)`	Sin comentario en el código. Por su nombre: select plan.
`startCheckout()`	Sin comentario en el código. Por su nombre: start checkout.
`onCheckoutOpened()`	Sin comentario en el código. Por su nombre: on checkout opened.
`onCelebrationShown()`	Sin comentario en el código. Por su nombre: on celebration shown.
`friendly(error)` interna	Sin comentario en el código. Por su nombre: friendly.

SplashScreen.kt

`ui/screens/splash/SplashScreen.kt · 307 líneas`

Progreso de 0 a 1 de un tramo start..end de la animación completa.

`phase(t, start, end)` interna	Progreso de 0 a 1 de un tramo start..end de la animación completa.
`SplashScreen(navController, viewModel)` pantalla	La entrada de la app. "Nexus" es conexión, y eso es lo que se ve: puntos dispersos que se buscan y se enlazan, un anillo que se cierra a su alrededor y, dentro, la N trazándose.
`Wordmark(t)` pantalla interna	Letra a letra, subiendo: el nombre aparece cuando el emblema ya está completo.
`drawSplashField(t, accent)` interna	Lo que rodea al emblema: motas que caen hacia el centro con distinta velocidad, lo que da profundidad, y una onda que sale cuando el anillo se cierra.
`drawNexusEmblem(t, accent)`	El emblema de NexusChat. Con t = 1 se ve completo, sin animar.

SplashViewModel.kt

`ui/screens/splash/SplashViewModel.kt · 40 líneas`

TutorialContent.kt

`ui/screens/tutorial/TutorialContent.kt · 585 líneas`

Contenido de las guías de la app. REGLA DE ORO: aquí solo se documenta lo que la app HACE de verdad, hoy. Un tutorial que promete una función inexistente es peor que no tener tutorial: el usuario la busca, no la encuentra y concluye que la app está rota. Revisión completa en la v5. Se corrigieron: - La marca: los textos arrastraban un nombre antiguo; la app se llama **NexusChat**. - "15 colores de acento" → son 25 (`AppTheme.ACCENT_SWATCHES`). - "3 tamaños de fuente" → son 4 (Pequeño, Normal, Grande, Muy Grande). - "Tema automático según el sistema": no existe, solo hay interruptor de tema oscuro. - "Privacidad de Stories: Todos / Solo contactos / Contactos excepto…": nunca se implementó; no

`getTutorialById(id)`	Sin comentario en el código. Por su nombre: get tutorial by id.

TutorialScreen.kt

`ui/screens/tutorial/TutorialScreen.kt · 124 líneas`

`TutorialScreen(navController, tutorialId)` pantalla	Dibuja en pantalla: tutorial screen.
`TutorialSectionCard(section)` pantalla	Dibuja en pantalla: tutorial section card.

ImageCropScreen.kt

`ui/screens/viewer/ImageCropScreen.kt · 181 líneas`

`ImageCropScreen(imageUri, photoType, navController)` pantalla	Dibuja en pantalla: image crop screen.

PhotoViewerScreen.kt

`ui/screens/viewer/PhotoViewerScreen.kt · 294 líneas`

`PhotoViewerScreen(imageUrl, navController)` pantalla	Dibuja en pantalla: photo viewer screen.

Parte 18

Componentes

Piezas de interfaz reutilizables.

AnimatedEmoji.kt

`ui/components/AnimatedEmoji.kt · 75 líneas`

Prefijo de los stickers animados: el resto del texto es el emoji que representan. */ const val ANIMATED_STICKER_PREFIX = "noto:" /** Emoji animado (Noto Animated Emoji de Google, licencia CC BY 4.0). Mientras se descarga, si no existe animación para ese emoji o sin conexión se ve el emoji normal. En modo anónimo no se piden: la descarga saldría fuera de Tor y revelaría la IP a Google.

`AnimatedEmoji(emoji, size, modifier, iterations)` pantalla	Emoji animado (Noto Animated Emoji de Google, licencia CC BY 4.0). Mientras se descarga, si no existe animación para ese emoji o sin conexión se ve el emoji normal. En modo anónimo no se piden: la descarga saldría fuera de Tor y revelaría la IP a Google.
`notoAnimationUrl(emoji)`	"😜" → .../1f600/lottie.json; las secuencias se unen con guion bajo.
`splitGraphemes(text)`	Separa un texto en emojis completos: una familia o una bandera cuentan como uno solo.

AnimatedIconButton.kt

`ui/components/AnimatedIconButton.kt · 46 líneas`

El botón de icono de toda la app: se encoge bajo el dedo y vuelve con un pequeño rebote, con un leve giro. La animación responde a un gesto, así que no distrae ni gasta batería en reposo.

`AnimatedIconButton(onClick)` pantalla	El botón de icono de toda la app: se encoge bajo el dedo y vuelve con un pequeño rebote, con un leve giro. La animación responde a un gesto, así que no distrae ni gasta batería en reposo.

AnimatedRingAvatar.kt

`ui/components/AnimatedRingAvatar.kt · 116 líneas`

Tu foto dentro de un aro vivo: el aro da una vuelta cada 10 s mientras su arco crece y se recoge cada 2 s, dejando una estela de segmentos que se afinan. Al tocarla se encoge un instante.

`AnimatedRingAvatar(name, photoUrl, size, modifier, onClick)` pantalla	Tu foto dentro de un aro vivo: el aro da una vuelta cada 10 s mientras su arco crece y se recoge cada 2 s, dejando una estela de segmentos que se afinan. Al tocarla se encoge un instante.
`eased(t)`	Sin comentario en el código. Por su nombre: eased.

AppBackground.kt

`ui/components/AppBackground.kt · 160 líneas`

App-wide background component Wraps entire app content and renders background behind all screens Features: - Solid color, image, video, gradient support - Configurable overlay opacity (0-80%) - Smooth crossfade transitions (500ms) - Lifecycle-aware for video backgrounds

`AppBackground(backgroundManager, modifier, content)` pantalla	App-wide background component Wraps entire app content and renders background behind all screens Features: - Solid color, image, video, gradient support - Configurable overlay opacity (0-80%) - Smooth crossfade transitions (500ms) - Lifecycle-aware for video backgrounds

AppUpdateDialog.kt

`ui/components/AppUpdateDialog.kt · 140 líneas`

Aparece en cuanto hay una versión nueva, también con la app ya abierta.

`AppUpdateDialog(manager)` pantalla	Aparece en cuanto hay una versión nueva, también con la app ya abierta.

AttachmentBottomSheet.kt

`ui/components/AttachmentBottomSheet.kt · 205 líneas`

`AttachmentBottomSheet(onAttachmentSelected)` pantalla	Dibuja en pantalla: attachment bottom sheet.
`AttachmentOption(icon, label, color, onClick)` pantalla	Dibuja en pantalla: attachment option.

AudioMessagePlayer.kt

`ui/components/AudioMessagePlayer.kt · 336 líneas`

`AudioMessagePlayer(audioUrl, modifier, accentColor)` pantalla	Dibuja en pantalla: audio message player.
`formatDuration(milliseconds)` interna	Sin comentario en el código. Por su nombre: format duration.

AutoTutorial.kt

`ui/components/AutoTutorial.kt · 85 líneas`

Auto Tutorial Composable Automatically shows tutorial dialog on first visit to a feature Usage:

`AutoTutorial(feature, tutorialPreferences)` pantalla	Auto Tutorial Composable Automatically shows tutorial dialog on first visit to a feature Usage:
`TutorialButton(feature, onClick)` pantalla	Manual Tutorial Button Shows a help button that opens the tutorial on demand

ChatLook.kt

`ui/components/ChatLook.kt · 877 líneas`

Cómo se dibuja cada conversación en la lista.

`coloredIcons()`	Estilos en los que todos los iconos llevan el color de acento, sin círculo detrás.
`cameraOutside()`	Emoji dentro del campo a la derecha, y cámara fuera junto al micrófono.
`setHomeStyle(style)`	Sin comentario en el código. Por su nombre: set home style.
`setSettingsStyle(style)`	Sin comentario en el código. Por su nombre: set settings style.
`setLinkPreviews(on)`	Sin comentario en el código. Por su nombre: set link previews.
`setRowTint(tint)`	Sin comentario en el código. Por su nombre: set row tint.
`setAvatarEffect(effect)`	Sin comentario en el código. Por su nombre: set avatar effect.
`setHideEntryEmoji(on)`	Sin comentario en el código. Por su nombre: set hide entry emoji.
`setEntryTranslate(on)`	Sin comentario en el código. Por su nombre: set entry translate.
`setBubblePhoto(on)`	Sin comentario en el código. Por su nombre: set bubble photo.
`setHideStoriesTab(on)`	Sin comentario en el código. Por su nombre: set hide stories tab.
`setQuickFunctions(on)`	Sin comentario en el código. Por su nombre: set quick functions.

`flag(key, value)` interna	Sin comentario en el código. Por su nombre: flag.
`setFontStyle(style)`	Sin comentario en el código. Por su nombre: set font style.
`setHomeSearchBar(on)`	Sin comentario en el código. Por su nombre: set home search bar.
`setQuickBar(on)`	Sin comentario en el código. Por su nombre: set quick bar.
`setGhostButton(on)`	Sin comentario en el código. Por su nombre: set ghost button.
`setHideArchived(on)`	Sin comentario en el código. Por su nombre: set hide archived.
`setConfirmSticker(on)`	Sin comentario en el código. Por su nombre: set confirm sticker.
`setStoryCards(on)`	Sin comentario en el código. Por su nombre: set story cards.
`init(context)`	Sin comentario en el código. Por su nombre: init.
`setRowStyle(style)`	Sin comentario en el código. Por su nombre: set row style.
`setBubbleStyle(style)`	Sin comentario en el código. Por su nombre: set bubble style.
`setTickStyle(style)`	Sin comentario en el código. Por su nombre: set tick style.
`setNavIconSet(set)`	Sin comentario en el código. Por su nombre: set nav icon set.
`setNavBarStyle(style)`	Sin comentario en el código. Por su nombre: set nav bar style.
`setOwnBubbleColor(color)`	Sin comentario en el código. Por su nombre: set own bubble color.
`setPeerBubbleColor(color)`	Sin comentario en el código. Por su nombre: set peer bubble color.
`setTextScale(scale)`	Sin comentario en el código. Por su nombre: set text scale.
`setEntryHint(hint)`	Sin comentario en el código. Por su nombre: set entry hint.
`setHomeAvatar(show)`	Sin comentario en el código. Por su nombre: set home avatar.
`setHideFilters(hide)`	Sin comentario en el código. Por su nombre: set hide filters.
`setAnimatedCallIcons(enabled)`	Sin comentario en el código. Por su nombre: set animated call icons.
`setTextFilters(enabled)`	Sin comentario en el código. Por su nombre: set text filters.
`setWallpaperDim(dim)`	Sin comentario en el código. Por su nombre: set wallpaper dim.
`setEntryStyle(style)`	Sin comentario en el código. Por su nombre: set entry style.
`ChatRowFrame(style, contactColors, photoUrl, content)` pantalla	Marco de una fila. contactColors es el color de perfil del contacto.
`ListRowFrame(seed, photoUrl, content)` pantalla	Fila de cualquier otra lista (llamadas, historias, contactos, búsqueda): mismo estilo.
`drawRowBevel(tint)` interna	Bisel de la fila en relieve: una franja que se apaga hacia fuera en los lados y abajo, y unos trazos en diagonal en dos esquinas. Las medidas son proporcionales a la fila, así que se estira con ella y no deforma nada en filas más altas o más anchas.
`fade(stops)`	Sin comentario en el código. Por su nombre: fade.
`at(x, y)`	Sin comentario en el código. Por su nombre: at.
`margin()` interna	Sin comentario en el código. Por su nombre: margin.
`DepthCard(colors, content)` pantalla interna	Tarjeta con volumen real: un canto de varias capas con el color del contacto debajo de la cara, y un giro en perspectiva que depende de dónde está en la pantalla y de dónde la toques.
`neonGlow(color)` interna	Halo de luz alrededor de una pieza, dibujado con la sombra del lienzo.
`chatEntryModifier(style, accent)` pantalla	Marco del campo donde se escribe. El contenido (iconos y texto) es siempre el mismo.
`chatBubbleShape(style, isOwn)`	Forma de una burbuja según el estilo y de quién es.
`chatBubbleSurface(style, shape, color, accent)`	Superficie de una burbuja: incluye su recorte, por eso va en lugar de `.clip`.
`ReceiptMark(style, color, double, read, modifier)` pantalla	Marca de enviado o leído en el estilo elegido.
`tick(x)`	Sin comentario en el código. Por su nombre: tick.

`RowDivider(modifier, startInset)` pantalla	Separador de listas: entre tarjetas sobra, ellas ya se separan solas.

ColorPickerDialog.kt

`ui/components/ColorPickerDialog.kt · 198 líneas`

Color picker dialog with hex input and preset swatches Features: - 20 preset colors in red/dark theme - Custom hex input - Live preview circle - Confirm/cancel buttons

`ColorPickerDialog(initialColor, onDismiss)` pantalla	Color picker dialog with hex input and preset swatches Features: - 20 preset colors in red/dark theme - Custom hex input - Live preview circle - Confirm/cancel buttons

CompleteEmojiPicker.kt

`ui/components/CompleteEmojiPicker.kt · 273 líneas`

😜 COMPLETE EMOJI PICKER Selector de emojis completo con TODOS los emojis de Unicode Organizado por categorías como WhatsApp/Telegram

`getAllCategories()`	Sin comentario en el código. Por su nombre: get all categories.
`CompleteEmojiPicker(onEmojiSelected)` pantalla	Dibuja en pantalla: complete emoji picker.

DraggableEmoji.kt

`ui/components/DraggableEmoji.kt · 74 líneas`

Draggable emoji component for stories Allows user to drag emojis anywhere on the screen

`DraggableEmoji(emoji, initialOffset, onPositionChange)` pantalla	Draggable emoji component for stories Allows user to drag emojis anywhere on the screen

DraggableText.kt

`ui/components/DraggableText.kt · 78 líneas`

Draggable text component for stories Allows user to drag text anywhere on the screen

`DraggableText(text, initialOffset, onPositionChange)` pantalla	Draggable text component for stories Allows user to drag text anywhere on the screen

FeatureTutorialDialog.kt

`ui/components/FeatureTutorialDialog.kt · 322 líneas`

Generic Feature Tutorial Dialog Displays interactive tutorials for any app feature with markdown-style formatting.

`FeatureTutorialDialog(title, tutorialText, onDismiss)` pantalla	Generic Feature Tutorial Dialog Displays interactive tutorials for any app feature with markdown-style formatting.
`TutorialSection(section)` pantalla interna	Dibuja en pantalla: tutorial section.
`CodeBlock(code)` pantalla interna	Dibuja en pantalla: code block.
`CalloutBlock(icon, content, containerColor, contentColor)` pantalla interna	Dibuja en pantalla: callout block.
`parseTutorialSections(tutorialText)` interna	Parses markdown-style tutorial text into structured sections. Supports: - `/ / ` headers → new TEXT section - Triple backticks ``` → CODE block - Lines starting with ⚠️ → WARNING callout - Lines starting with ℹ️ → INFO callout
`flushCurrentSection()`	Sin comentario en el código. Por su nombre: flush current section.

FullScreenImageViewer.kt

`ui/components/FullScreenImageViewer.kt · 662 líneas`

🖼️ FULL SCREEN IMAGE VIEWER Visor de imágenes a pantalla completa con zoom, pan, descarga y swipe-to-dismiss Similar a WhatsApp/Telegram - Usa Dialog fullscreen para cubrir barras del sistema Los `@Suppress("USELESS_IS_CHECK")` y `@Suppress("KotlinConstantConditions")` que llevaba este archivo silenciaban los avisos del compilador sobre las ramas `is AsyncImagePainter.State.Loading/Error`, que efectivamente eran inalcanzables porque se comparaba contra un `StateFlow` en vez de contra el estado. Corregido el tipo, los avisos desaparecen y las supresiones sobran. Recibe `true` cuando el usuario elige borrar para todos.

`FullScreenImageViewer(imageUrl, senderName, timestamp, onDismiss)` pantalla	🖼️ FULL SCREEN IMAGE VIEWER Visor de imágenes a pantalla completa con zoom, pan, descarga y swipe-to-dismiss Similar a WhatsApp/Telegram - Usa Dialog fullscreen para cubrir barras del sistema Los `@Suppress("USELESS_IS_CHECK")` y `@Suppress("KotlinConstantConditions")` que llevaba este archivo silenciaban los avisos del compilador sobre las ramas `is AsyncImagePainter.State.Loading/Error`, que efectivamente eran inal
`FullScreenImageContent(imageUrl, senderName, timestamp, onDismiss)` pantalla interna	Dibuja en pantalla: full screen image content.
`extractBitmap()`	Sin comentario en el código. Por su nombre: extract bitmap.
`ImageViewerAction(icon, label, color, onClick)` pantalla	Dibuja en pantalla: image viewer action.

LiveCallIcon.kt

`ui/components/LiveCallIcon.kt · 66 líneas`

Movimiento propio sobre los iconos de llamada: ondas que salen del teléfono o un aro que late en la cámara. Se dibuja encima del icono y no recibe toques. Se apaga desde el estudio de diseño.

`CallIconMotion(video, modifier)` pantalla	Movimiento propio sobre los iconos de llamada: ondas que salen del teléfono o un aro que late en la cámara. Se dibuja encima del icono y no recibe toques. Se apaga desde el estudio de diseño.

LoopingVideo.kt

`ui/components/LoopingVideo.kt · 57 líneas`

Vídeo en bucle y sin sonido que se puede recortar en cualquier forma, como un avatar.

`LoopingVideo(url, modifier)` pantalla	Vídeo en bucle y sin sonido que se puede recortar en cualquier forma, como un avatar.

MarkdownText.kt

`ui/components/MarkdownText.kt · 452 líneas`

Renderiza markdown en Compose sin dependencias externas. Las respuestas de la IA llegan en markdown — encabezados, negritas, listas y bloques de código— pero se pintaban con un `Text` plano, así que el usuario leía los asteriscos, las almohadillas y las comillas invertidas tal cual. Lo único que hacía la pantalla era cambiar toda la burbuja a monoespaciada si el texto contenía ```, con lo que un mensaje con un solo fragmento de código convertía también la explicación en código. Se soporta el subconjunto que un asistente usa de verdad: - encabezados `#`, `##`, `###` - **negrita**, *cursiva*, `código`, ~~tachado~~ y enlaces `texto(url)` - bloques de código con lenguaje, scroll horizontal y botón de c

`MarkdownText(markdown, modifier, color, fontSize, lineHeight, accent)` pantalla	Renderiza markdown en Compose sin dependencias externas. Las respuestas de la IA llegan en markdown —encabezados, negritas, listas y bloques de código— pero se pintaban con un `Text` plano, así que el usuario leía los asteriscos, las almohadillas y las comillas invertidas tal cual. Lo único que hacía la pantalla era cambiar toda la burbuja a monoespaciada si el texto contenía ```, con lo que un mensaje con un solo fr
`MarkdownListRow(marker, text, indent, color, accent, fontSize)` pantalla interna	Dibuja en pantalla: markdown list row.
`MarkdownCodeBlock(code, language, accent)` pantalla interna	Bloque de código con etiqueta de lenguaje y botón de copiar. El scroll horizontal es deliberado: partir una línea de código por la mitad la vuelve ilegible, y en un móvil casi cualquier línea real se sale del ancho.
`parseBlocks(markdown)` interna	Divide el texto en bloques. Los párrafos consecutivos se unen para que un salto de línea simple no rompa la frase, igual que en markdown estándar.
`flushParagraph()`	Sin comentario en el código. Por su nombre: flush paragraph.
`inlineMarkdown(text, accent)` interna	Aplica los estilos de línea (negrita, cursiva, código, tachado, enlaces). Se recorre el texto una sola vez en lugar de encadenar reemplazos con regex: los reemplazos encadenados se pisan entre sí cuando un estilo contiene a otro (`**texto con `código` dentro**`) y descuadran los índices.

NexusButton.kt

`ui/components/NexusButton.kt · 141 líneas`

Botón de la app. Una sola fuente de verdad para color, altura y estado de carga. Accesibilidad: rol semántico de botón, ripple, y alturas ≥ 48dp en las tallas Large/Medium (objetivo táctil). La talla Small (40dp) es para contextos densos.

`NexusButton(text, onClick)` pantalla	Dibuja en pantalla: nexus button.

NexusComponents.kt

`ui/components/NexusComponents.kt · 75 líneas`

Superficie plana con un filo de 1 dp. Conserva el nombre antiguo para las pantallas que aún la usan, pero ya no hay vidrio, degradado ni rebote al pulsar.

`nexusGlass(glow, shape)`	Superficie plana con un filo de 1 dp. Conserva el nombre antiguo para las pantallas que aún la usan, pero ya no hay vidrio, degradado ni rebote al pulsar.
`NexusGlassCard(modifier, onClick)` pantalla	Tarjeta plana. Úsala con moderación: la mayoría del contenido va mejor en listas.
`NexusStatusBadge(text, color, modifier)` pantalla	Etiqueta de estado: texto del tono indicado sobre un fondo apenas teñido.

NexusKit.kt

`ui/components/NexusKit.kt · 447 líneas`

Barra superior opaca. Al ser opaca, el contenido nunca se superpone al título al desplazarse.

`NexusTopBar(title, modifier, subtitle, onBack)` pantalla	Barra superior opaca. Al ser opaca, el contenido nunca se superpone al título al desplazarse.
`NexusSectionHeader(text, modifier, trailing)` pantalla	Cabecera de sección en una lista.
`NexusDivider(modifier, startInset)` pantalla	Separador fino, alineado con el texto de la fila y no con su icono.
`NexusListRow(title, modifier, subtitle, icon, iconTint, titleColor)` pantalla	Fila de lista: icono monocromo, título, subtítulo opcional y contenido al final. Es la pieza principal de Ajustes, Seguridad, Acerca de y cualquier listado.
`NexusRowValue(text, color)` pantalla	Valor corto al final de una fila ("Jade", "5 min").

`NexusSwitchRow(title, checked, onCheckedChange)` pantalla	Fila con interruptor: toda la fila conmuta, no solo el control.
`NexusDetailRow(label, value, modifier, valueColor, monospace, onClick)` pantalla	Dato en una línea: etiqueta a la izquierda, valor a la derecha.
`NexusEmptyState(icon, title, modifier, body, actionLabel, onAction)` pantalla	Estado vacío: una frase que dice qué pasa y, si tiene sentido, qué hacer.
`NexusFilterChip(label, selected, onClick)` pantalla	Chip de filtro compacto.
`NexusCountBadge(count, modifier, muted)` pantalla	Contador de no leídos.
`NexusNotice(text, modifier, icon, tone, actionLabel, onAction)` pantalla	Aviso dentro de una pantalla (no un Toast que desaparece antes de leerlo).

NexusMotion.kt

`ui/components/NexusMotion.kt · 87 líneas`

Respuesta física al pulsar: el elemento cede un poco bajo el dedo.

`pressScale(interactionSource, pressedScale)` pantalla	Respuesta física al pulsar: el elemento cede un poco bajo el dedo.
`staggeredEnter(index, enabled)` pantalla	Entrada escalonada para las filas visibles al abrir una lista. Solo la primera vez: repetirla al desplazarse se notaría como parpadeo.
`messageEnter(animate, fromEnd)` pantalla	Aparición de un mensaje recién llegado: sube y crece un poco desde su esquina.

PhotoAdjuster.kt

`ui/components/PhotoAdjuster.kt · 328 líneas`

📸 PHOTO ADJUSTER COMPONENT Permite ajustar la posición vertical de una foto con gestos táctiles Usado en Stories y Profile para posicionar fotos de perfil/portada

`PhotoAdjuster(imageUri, onPositionChange)` pantalla	📸 PHOTO ADJUSTER COMPONENT Permite ajustar la posición vertical de una foto con gestos táctiles Usado en Stories y Profile para posicionar fotos de perfil/portada

ReadReceiptIndicator.kt

`ui/components/ReadReceiptIndicator.kt · 192 líneas`

✓✓ Indicador de estado de mensaje. Antes usaba los iconos genéricos de Material (`Check` / `DoneAll`): trazos gruesos, separación fija y el mismo violeta de marca para "leído", que no se distinguía de cualquier otro acento de la app. Ahora los checks se DIBUJAN a medida, lo que permite controlar grosor, remates redondeados y el solape exacto entre ambos ticks (como en las apps de mensajería de referencia), y el estado "leído" usa el azul de confirmación del design system (MessageRead) con un halo suave y un pequeño "pop" al pasar a leído.

`ReadReceiptIndicator(status, modifier)` pantalla	✓✓ Indicador de estado de mensaje. Antes usaba los iconos genéricos de Material (`Check` / `DoneAll`): trazos gruesos, separación fija y el mismo violeta de marca para "leído", que no se distinguía de cualquier otro acento de la app. Ahora los checks se DIBUJAN a medida, lo que permite controlar grosor, remates redondeados y el solape exacto entre ambos ticks (como en las apps de mensajería de referencia), y el estad
`CheckMarks(color, double, glow, modifier)` pantalla interna	Dibuja uno o dos ticks con trazo redondeado. Se dibuja en vez de usar iconos para poder ajustar el solape entre ticks: los `DoneAll` de Material dejan los dos checks demasiado separados a tamaños pequeños y se leen como un borrón. Cuando glow es cierto se pinta primero una copia difusa y más gruesa que actúa de halo.
`tickPath(startX)`	Sin comentario en el código. Por su nombre: tick path.

SafeClickable.kt

`ui/components/SafeClickable.kt · 53 líneas`

Safe clickable modifier that prevents ACTION_HOVER_EXIT crashes Use this instead of .clickable { } throughout the app

`safeClickable(enabled, onClick)`	Safe clickable modifier that prevents ACTION_HOVER_EXIT crashes Use this instead of .clickable { } throughout the app

StickerEditor.kt

`ui/components/StickerEditor.kt · 277 líneas`

Crear un sticker: encuadrar con dos dedos, quitar el fondo, añadir texto y borde. Se guarda en WebP sin pérdida de 512×512, con el fondo transparente.

`StickerEditorDialog(source, saving, onDismiss)` pantalla	Crear un sticker: encuadrar con dos dedos, quitar el fondo, añadir texto y borde. Se guarda en WebP sin pérdida de 512×512, con el fondo transparente.
`removeBackground(image)` interna asíncrona	Recorta la persona u objeto principal con ML Kit. Funciona en el teléfono: la foto no sale de él.
`renderSticker(source, scale, offset, text, textColor, textOnTop)`	Sin comentario en el código. Por su nombre: render sticker.

StickerPicker.kt

`ui/components/StickerPicker.kt · 384 líneas`

Guarda en "Mis stickers" uno recibido en un chat.

`readRecent()` interna	Sin comentario en el código. Por su nombre: read recent.
`readList(key)` interna	Sin comentario en el código. Por su nombre: read list.
`toggleFavorite(sticker)`	Sin comentario en el código. Por su nombre: toggle favorite.
`forgetRecent(sticker)`	Sin comentario en el código. Por su nombre: forget recent.
`markUsed(sticker)`	Sin comentario en el código. Por su nombre: mark used.
`saveEdited(file, packId, onDone)`	Sin comentario en el código. Por su nombre: save edited.
`createPack(name)`	Sin comentario en el código. Por su nombre: create pack.
`removeSticker(packId, sticker)`	Sin comentario en el código. Por su nombre: remove sticker.
`saveReceived(sticker)`	Guarda en "Mis stickers" uno recibido en un chat.
`share(pack, onCode)`	Sin comentario en el código. Por su nombre: share.
`install(code)`	Sin comentario en el código. Por su nombre: install.
`StickerImage(sticker, size, modifier, playOnce)` pantalla	Dibuja un sticker: animado de la colección, o imagen o GIF subido por alguien.
`StickerPicker(onStickerSelected)` pantalla	Dibuja en pantalla: sticker picker.

UnifiedTopBar.kt

`ui/components/UnifiedTopBar.kt · 199 líneas`

Barra superior compartida: opaca y sin animaciones. El contenido de la pantalla pasa por debajo de un fondo sólido, así que el título siempre se lee. Los parámetros antiguos de efectos ya no hacen nada; se conservan para no romper a quien los pasa.

`UnifiedTopBar(title, modifier, userName, userPhotoUrl, userSubtitle, onUserClick)` pantalla	Barra superior compartida: opaca y sin animaciones. El contenido de la pantalla pasa por debajo de un fondo sólido, así que el título siempre se lee. Los parámetros antiguos de efectos ya no hacen nada; se conservan para no romper a quien los pasa.
`ModernIconButton(icon, contentDescription, onClick)` pantalla	Botón de icono de barra.
`UnifiedProfileTopBar(userName, userPhotoUrl, modifier, onBackClick)` pantalla	Barra de las pantallas de perfil.
`TopBarActionIcon(icon, contentDescription, onClick)` pantalla	Icono de acción de barra.

UserAvatar.kt

`ui/components/UserAvatar.kt · 304 líneas`

Colores de perfil: cada persona elige el suyo y se ve en su aro.

`indexFor(seed, chosen)`	Índice del elegido o, si no eligió, uno fijo gratuito derivado de seed.
`forUser(seed, chosen)`	El elegido, o uno fijo derivado de seed: la misma persona siempre tiene el mismo.
`UserAvatar(name, photoUrl, size, modifier, backgroundColor, ring)` pantalla	Dibuja en pantalla: user avatar.
`drawInsetPhoto()` interna	Foto hundida: una media luna de sombra por dentro arriba, como si la tarjeta le hiciera sombra, y un labio de luz abajo. Se dibuja por código para que valga a cualquier tamaño.
`drawOrbPhoto()` interna	Foto en esfera: lo contrario de hundida, con luz arriba a la izquierda y sombra abajo.
`avatarRing(ring)` pantalla interna	Dibuja en pantalla: avatar ring.

VideoBackgroundPlayer.kt

`ui/components/VideoBackgroundPlayer.kt · 134 líneas`

Reusable video background player component Features: - ExoPlayer with infinite loop - Always muted - Lifecycle-aware (pause/resume) - Auto-release on dispose - Loading indicator - Fallback to solid color on error

`VideoBackgroundPlayer(videoUri, modifier, fallbackColor)` pantalla	Reusable video background player component Features: - ExoPlayer with infinite loop - Always muted - Lifecycle-aware (pause/resume) - Auto-release on dispose - Loading indicator - Fallback to solid color on error
`onPlaybackStateChanged(playbackState)`	Sin comentario en el código. Por su nombre: on playback state changed.
`onPlayerError(error)`	Sin comentario en el código. Por su nombre: on player error.

VideoRenderer.kt

`ui/components/VideoRenderer.kt · 92 líneas`

Superficie de vídeo WebRTC. ser ese y no uno propio: los fotogramas llegan como texturas que solo existen en él, y con otro contexto la pantalla se queda en negro aunque la imagen esté llegando. necesitan decir cuál va encima, o el recuadro queda tapado por la imagen grande.

`VideoRenderer(videoTrack, eglContext, modifier, mirror, overlay)` pantalla	Superficie de vídeo WebRTC. ser ese y no uno propio: los fotogramas llegan como texturas que solo existen en él, y con otro contexto la pantalla se queda en negro aunque la imagen esté llegando. necesitan decir cuál va encima, o el recuadro queda tapado por la imagen grande.

VideoWallpaper.kt

`ui/components/VideoWallpaper.kt · 101 líneas`

Video wallpaper component for chat background. Features: - Looping playback - Muted audio - Scales to fill screen - Fallback to solid color if video fails - Adjustable opacity

`VideoWallpaper(videoUri, modifier, alpha)` pantalla	Video wallpaper component for chat background. Features: - Looping playback - Muted audio - Scales to fill screen - Fallback to solid color if video fails - Adjustable opacity
`onPlayerError(error)`	Sin comentario en el código. Por su nombre: on player error.

MessageBubble.kt

`ui/components/chat/MessageBubble.kt · 1127 líneas`

Burbuja de mensaje. Plana y legible: el color distingue lo enviado de lo recibido y no hay sombras, brillos ni animaciones por mensaje. Tocar un medio lo abre. Mantener pulsado abre las acciones (reaccionar, responder, copiar, traducir, editar, eliminar).

`MessageBubble(message, isOwnMessage, repliedMessage, repliedSenderName, onQuoteClick)` pantalla	Burbuja de mensaje. Plana y legible: el color distingue lo enviado de lo recibido y no hay sombras, brillos ni animaciones por mensaje. Tocar un medio lo abre. Mantener pulsado abre las acciones (reaccionar, responder, copiar, traducir, editar, eliminar).
`SystemNotice(text)` pantalla interna	Dibuja en pantalla: system notice.
`EphemeralCountdown(message, serverNow)` pantalla interna	Tiempo restante; antes de que lo abran, cuánto durará.
`ViewOncePlaceholder(isOwnMessage)` pantalla interna	Dibuja en pantalla: view once placeholder.
`ViewOnceViewer(message, onClose)` pantalla interna	Visor de una sola vista: a pantalla completa y con capturas bloqueadas. Al cerrarlo el mensaje se borra para todos.
`MediaContent(message, autoplayVideos, onImageClick)` pantalla interna	Dibuja en pantalla: media content.
`tapAndHold(onTap)` interna	Sin comentario en el código. Por su nombre: tap and hold.
`openExternal(context, intent)` interna	Sin comentario en el código. Por su nombre: open external.
`AttachmentRow(icon, title, subtitle, onTap)` pantalla interna	Dibuja en pantalla: attachment row.
`MediaTag(text, modifier)` pantalla interna	Dibuja en pantalla: media tag.
`InlineVideo(url, muted, showControls, modifier)` pantalla	Vídeo dentro de la conversación. El reproductor vive solo mientras está en pantalla.
`FullscreenVideo(url, onClose)` pantalla interna	Dibuja en pantalla: fullscreen video.
`ReplyQuote(replied, isOwnMessage, senderName, onClick)` pantalla interna	Dibuja en pantalla: reply quote.
`StoryReplyQuote()` pantalla interna	Cita de una respuesta a historia. La historia puede haber caducado, así que la cita no intenta cargar el medio original.
`ActionRow(icon, label, color, onClick)` pantalla interna	Dibuja en pantalla: action row.
`isEmojiOnly()` interna	Hasta cinco emojis sueltos se muestran grandes y sin burbuja.
`formatTime(timestamp)` interna	Sin comentario en el código. Por su nombre: format time.
`formatCountdown(seconds)` interna	Sin comentario en el código. Por su nombre: format countdown.
`ReactorsSheet(reactions, myUid, nameOf)` pantalla interna	Quién reaccionó y con qué. Arriba, una pestaña por emoji con su cuenta; tocar tu propia fila quita tu reacción, como en los demás mensajeros.
`ReactorTab(label, selected, onClick)` pantalla interna	Dibuja en pantalla: reactor tab.

Parte 19

Tema

Colores, tipografía y tokens de diseño.

Color.kt

`ui/theme/Color.kt · 92 líneas`

ColorUtils.kt

`ui/theme/ColorUtils.kt · 54 líneas`

Parse a hex color string (RRGGBB or AARRGGBB) to Compose Color. Falls back to fallback on any parse error.

`parseHexColor(hex, fallback)`	Parse a hex color string (RRGGBB or AARRGGBB) to Compose Color. Falls back to fallback on any parse error.
`linearGradientBrush(gradientColors, gradientAngle)`	Convert gradient colors + angle into a Compose Brush.linearGradient. Returns null when fewer than 2 colors.

DynamicTheme.kt

`ui/theme/DynamicTheme.kt · 178 líneas`

Acentos de la app: una lista corta y curada. Fuente ÚNICA de los acentos de la app. SWATCHES define el orden, el id (clave persistida), el nombre visible y el triplete de colores (primary + variantes para gradientes). Tanto el selector de la UI (ACCENT_SWATCHES) como la resolución de color (getPrimaryColor/getThemeColors) leen de aquí, evitando listas duplicadas y "drift" entre pantallas.

`accent(id, name, hex)` interna	Un acento con sus tres roles. `secondary` coincide con `primary`: el código que aún dibuja `linearGradient(primary, secondary)` obtiene un color sólido.
`normalise(name)` interna	Normaliza un nombre a su clave canónica.
`isCustom(name)`	Un color libre elegido por el usuario se guarda como "RRGGBB".
`customColors(name)` interna	Sin comentario en el código. Por su nombre: custom colors.
`lookup(name)` interna	Busca los colores de un acento; si no se reconoce, Jade.
`canonicalId(name)`	Id canónico de un valor guardado, para marcar el acento activo en el selector.
`getPrimaryColor(theme)`	Sin comentario en el código. Por su nombre: get primary color.
`getSecondaryColor(theme)`	Sin comentario en el código. Por su nombre: get secondary color.
`getTertiaryColor(theme)`	Sin comentario en el código. Por su nombre: get tertiary color.
`getThemeColors(theme)`	Sin comentario en el código. Por su nombre: get theme colors.
`rememberThemeColor()` pantalla	Dibuja en pantalla: remember theme color.
`rememberThemeSecondaryColor()` pantalla	Dibuja en pantalla: remember theme secondary color.

NexusDesignTokens.kt

`ui/theme/NexusDesignTokens.kt · 198 líneas`

Sistema de diseño de NexusChat (v7). Dirección: grafito neutro + un único acento. Es una app de mensajería y privacidad: la interfaz tiene que desaparecer para que se lea la conversación. Reglas que evitan volver al aspecto de plantilla: - Sin degradados: los Gradient son planos a propósito. El código antiguo que todavía pide un gradiente recibe un color sólido. - Sin vidrio ni brillos: Glass son superficies opacas con un filo de 1 dp. - Sin animaciones infinitas

decorativas: solo comunican estado real (escribiendo, timbre, conectando). - Los iconos son Material y nunca emojis; los emojis son contenido del usuario. - El acento marca lo importante (no leídos, acción primaria). Si todo tiene c

`springDefault()`	Sin comentario en el código. Por su nombre: spring default.
`springBouncy()`	Sin comentario en el código. Por su nombre: spring bouncy.

Shape.kt

`ui/theme/Shape.kt · 24 líneas`

Theme.kt

`ui/theme/Theme.kt · 229 líneas`

Factor de escala del tamaño de fuente elegido por el usuario. Es la ÚNICA fuente de verdad del escalado: se usa para escalar la densidad de fuente de toda la app (ver NexusChatTheme). Antes solo se escalaba la `Typography` de Material, pero casi todos los `Text(...)` de la app usan `fontSize = XX.sp` a pelo y no leen de la Typography, así que "Muy grande" no agrandaba nada visible (mensajes del chat, tarjetas de la home, etc.). Se aceptan todas las variantes que puede guardar FontSizeScreen (ES/EN, con o sin guion bajo) para tolerar valores heredados.

`getAccentColor(colorName)`	Sin comentario en el código. Por su nombre: get accent color.
`fontScaleFor(sizeName)`	Factor de escala del tamaño de fuente elegido por el usuario. Es la ÚNICA fuente de verdad del escalado: se usa para escalar la densidad de fuente de toda la app (ver NexusChatTheme). Antes solo se escalaba la `Typography` de Material, pero casi todos los `Text(...)` de la app usan `fontSize = XX.sp` a pelo y no leen de la Typography, así que "Muy grande" no agrandaba nada visible (mensajes del chat, tarjetas de la h
`getTypographyForSize(sizeName)` pantalla	Dibuja en pantalla: get typography for size.
`readableOn(background)` interna	Texto legible sobre un color: oscuro sobre acentos claros, blanco sobre los oscuros. Con un `onPrimary` fijo en blanco, acentos como Jade o Ámbar no pasaban contraste.
`createDarkColorScheme(accentColor)` interna	Esquema oscuro completo. Antes `secondary`/`tertiary` eran el acento con alfa: los contenedores de Material salían translúcidos y cambiaban según lo que hubiera detrás.
`createLightColorScheme(accentColor)` interna	Sin comentario en el código. Por su nombre: create light color scheme.
`NexusChatTheme(darkTheme)` pantalla	Dibuja en pantalla: nexus chat theme.
`typographyWith(style)` interna	La misma escala de texto con otra familia de letra: solo cambia el aspecto, no los tamaños.

Type.kt

`ui/theme/Type.kt · 44 líneas`

Parte 20

Rutas

Grafo de navegación.

NavGraph.kt

`ui/navigation/NavGraph.kt · 489 líneas`

`NavGraph(navController, startDestination)` pantalla	Dibuja en pantalla: nav graph.

NavRoutes.kt

`ui/navigation/NavRoutes.kt · 83 líneas`

Centralized navigation routes for Nexus Chat All route constants are defined here to avoid typos and ensure consistency

`chatRoute(chatId)`	Sin comentario en el código. Por su nombre: chat route.
`storyViewerRoute(userId)`	Sin comentario en el código. Por su nombre: story viewer route.
`incomingCallRoute(callId, callType)`	Sin comentario en el código. Por su nombre: incoming call route.
`activeCallRoute(callId, callType, isCaller)`	Sin comentario en el código. Por su nombre: active call route.
`profileRoute(userId)`	Sin comentario en el código. Por su nombre: profile route.
`profileViewerRoute(userId)`	Sin comentario en el código. Por su nombre: profile viewer route.

Screen.kt

`ui/navigation/Screen.kt · 68 líneas`

`createRoute(chatId)`	Sin comentario en el código. Por su nombre: create route.
`createRoute(callId, callType)`	Sin comentario en el código. Por su nombre: create route.
`createRoute(callId, callType, isCaller)`	Sin comentario en el código. Por su nombre: create route.

Parte 21

Interfaz, varios

Utilidades de interfaz.

ImageActions.kt

`ui/utils/ImageActions.kt · 89 líneas`

Robust, reusable image actions: save to the public gallery and share via a standard Android share sheet. Works across API 31–36 (scoped storage aware).

`saveToGallery(context, bitmap)`	Saves bitmap to the public Pictures/NexusChat album so it shows up in the device gallery. Uses MediaStore on API 29+ (no storage permission needed).
`shareImage(context, bitmap)`	Writes bitmap to a cache file and launches the system share sheet. Lets the user forward/share the image to any app (including this chat).

UserProfileHelper.kt

`ui/utils/UserProfileHelper.kt · 63 líneas`

Centralized helper for resolving user display data with sensible fallbacks. Priority for the display name: displayName > email prefix > phone tail > "Anónimo". Never returns the generic "User"/"Usuario" placeholder.

`getDisplayName(user)`	Sin comentario en el código. Por su nombre: get display name.
`getDisplayNameFromSnapshot(snapshot)`	Sin comentario en el código. Por su nombre: get display name from snapshot.
`resolveDisplayName(displayName, name, email)`	Resolves a display name from already-fetched fields (e.g. Realtime DB map).
`getPhotoUrl(user)`	Sin comentario en el código. Por su nombre: get photo url.
`getPhotoUrlFromSnapshot(snapshot)`	Sin comentario en el código. Por su nombre: get photo url from snapshot.
`getInitials(name)`	Generates up to two uppercase initials for an avatar placeholder.

Parte 22

Repositorios

Acceso a Firebase y almacenamiento.

AuthRepository.kt

`data/repository/AuthRepository.kt · 177 líneas`

Los errores de Firebase en palabras claras, con el aviso de actualizar cuando no se sabe la causa.

`login(email, password)` asíncrona	Sin comentario en el código. Por su nombre: login.
`register(email, password, displayName)` asíncrona	Sin comentario en el código. Por su nombre: register.
`loginWithGoogle(idToken)` asíncrona	Sin comentario en el código. Por su nombre: login with google.
`logout()` asíncrona	Sin comentario en el código. Por su nombre: logout.
`getCurrentUser()`	Sin comentario en el código. Por su nombre: get current user.
`isUserAuthenticated()`	Sin comentario en el código. Por su nombre: is user authenticated.
`authErrorMessage(e)`	Los errores de Firebase en palabras claras, con el aviso de actualizar cuando no se sabe la causa.
`UPDATE_HINT()`	Muchos errores de registro desaparecen con la última versión de la app.

ChatBackgroundRepository.kt

`data/repository/ChatBackgroundRepository.kt · 117 líneas`

Repository for per-chat background management Features: - Save/load background per chat from Firebase - Local cache in memory with StateFlow (se pierde al cerrar el proceso; la persistencia offline con Room no está integrada en este repositorio)

`getBackground(chatId)`	Get background configuration for a chat
`saveBackground(chatId, config)` asíncrona	Save background configuration for a chat
`loadBackground(chatId)` asíncrona	Load background from Firebase
`deleteBackground(chatId)` asíncrona	Delete background configuration (reset to DEFAULT)

ChatRepository.kt

`data/repository/ChatRepository.kt · 378 líneas`

`getChats(userId)`	Sin comentario en el código. Por su nombre: get chats.
`getMessages(chatId, currentUserId)`	Sin comentario en el código. Por su nombre: get messages.
`sendMessage(chatId, message)` asíncrona	Sin comentario en el código. Por su nombre: send message.
`updateMessageStatus(chatId, messageId, status)` asíncrona	Sin comentario en el código. Por su nombre: update message status.
`addReaction(chatId, messageId, userId, emoji)` asíncrona	Sin comentario en el código. Por su nombre: add reaction.
`createChat(participants)` asíncrona	Sin comentario en el código. Por su nombre: create chat.
`setTypingStatus(chatId, userId, isTyping)` asíncrona	Sin comentario en el código. Por su nombre: set typing status.
`observeTypingStatus(chatId, userId)`	Sin comentario en el código. Por su nombre: observe typing status.

`deleteMessage(chatId, messageId, userId, deleteForEveryone)` asíncrona	Sin comentario en el código. Por su nombre: delete message.
`editMessage(chatId, messageId, newContent, userId)` asíncrona	Sin comentario en el código. Por su nombre: edit message.

InternalBotRepository.kt

`data/repository/InternalBotRepository.kt · 51 líneas`

Repository for internal bot features

`getAutoReplyPrivate()` interna	Sin comentario en el código. Por su nombre: get auto reply private.
`setAutoReplyPrivate(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set auto reply private.
`getAutoReplyPrivateMessage()`	Sin comentario en el código. Por su nombre: get auto reply private message.
`setAutoReplyPrivateMessage(message)` asíncrona	Sin comentario en el código. Por su nombre: set auto reply private message.
`getAutoReplyGroups()`	Sin comentario en el código. Por su nombre: get auto reply groups.
`setAutoReplyGroups(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set auto reply groups.
`getAutoReplyGroupsMessage()`	Sin comentario en el código. Por su nombre: get auto reply groups message.
`setAutoReplyGroupsMessage(message)` asíncrona	Sin comentario en el código. Por su nombre: set auto reply groups message.
`getGhostMode()`	Sin comentario en el código. Por su nombre: get ghost mode.
`setGhostMode(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set ghost mode.
`getMassMessages()`	Sin comentario en el código. Por su nombre: get mass messages.
`setMassMessages(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set mass messages.
`getTranslator()`	Sin comentario en el código. Por su nombre: get translator.
`setTranslator(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set translator.
`getTranslatorLanguage()`	Sin comentario en el código. Por su nombre: get translator language.
`setTranslatorLanguage(language)` asíncrona	Sin comentario en el código. Por su nombre: set translator language.
`getCustomQuoted()`	Sin comentario en el código. Por su nombre: get custom quoted.
`setCustomQuoted(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set custom quoted.
`getMentionAll()`	Sin comentario en el código. Por su nombre: get mention all.
`setMentionAll(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set mention all.

PremiumRepository.kt

`data/repository/PremiumRepository.kt · 138 líneas`

Suscripción de una persona. La escribe solo el servidor, tras confirmar el pago.

`isActiveNow(now)`	Sin comentario en el código. Por su nombre: is active now.
`observe(uid)`	Sin comentario en el código. Por su nombre: observe.
`loadPlans()` asíncrona	Planes y precios vigentes, según el servidor.
`createCheckout(planId)` asíncrona	Crea el cobro en el servidor y devuelve el enlace de pago de Mercado Pago.
`call(builder)` interna	Sin comentario en el código. Por su nombre: call.

RealtimeDatabaseRepository.kt

`data/repository/RealtimeDatabaseRepository.kt · 1807 líneas`

Estados con los que una llamada ya no admite cambios. */ private val TERMINAL_CALL_STATUSES = setOf("ENDED", "DECLINED", "MISSED", "FAILED", "BUSY") @Suppress("UNCHECKED_CAST") @Singleton /** Lo que la lista de chats necesita de cada conversación, sin su historial. */ private val CHAT_PREVIEW_FIELDS = listOf("members", "participants", "type", "isGroup", "groupName", "groupPhoto", "lastMessage", "lastMessagePayload", "lastMessageKeyId", "lastMessageId", "lastMessageTime", "lastMessageSenderId", "unreadCount", "isTyping", "isE2EE") class RealtimeDatabaseRepository @Inject constructor(private val groupCrypto: com.Azelmods.App.data.security.encryption.GroupCrypto, private val e2eeCryptoServic

`getChatMessages(chatId)`	Sin comentario en el código. Por su nombre: get chat messages.
`getChatMessagesPaginated(chatId, limit)`	Sin comentario en el código. Por su nombre: get chat messages paginated.
`loadMoreMessages(chatId, beforeTimestamp, limit)` asíncrona	Sin comentario en el código. Por su nombre: load more messages.
`hasMoreMessages(chatId, beforeTimestamp)` asíncrona	Sin comentario en el código. Por su nombre: has more messages.
`sendMessage(chatId, content, replyTo, forwarded)` asíncrona	Sin comentario en el código. Por su nombre: send message.
`sendEphemeralMessage(chatId, content, replyTo, isViewOnce, selfDestructDuration)` asíncrona	Sin comentario en el código. Por su nombre: send ephemeral message.
`sendMessageInternal(chatId, content, replyTo, isEphemeral, isViewOnce, selfDestructDuration)` interna asíncrona	Sin comentario en el código. Por su nombre: send message internal.
`decryptGroupMessage(chatId, keyId, payload)` asíncrona	Texto de un mensaje de grupo cifrado, o null si este teléfono no tiene su clave.
`serverNow()`	Hora actual según el servidor.
`observeChatTimer(chatId)`	Temporizador del chat en segundos (0 = desactivado). Es común a los participantes.
`setChatTimer(chatId, seconds, notice)` asíncrona	Cambia el temporizador del chat y deja constancia en la conversación.
`markEphemeralMessageViewed(chatId, messageId)` asíncrona	El destinatario ha visto el mensaje: empieza la cuenta atrás (o, si es de una sola vista, queda registrado que se abrió). Solo la primera apertura marca el tiempo.
`deleteEphemeralMessage(chatId, messageId, mediaUrl)` asíncrona	Borra de verdad un mensaje temporal: el nodo, el archivo en Storage y, si era la vista previa del chat, también el rótulo. Puede ejecutarlo cualquier participante y más de una vez: borrar lo que ya no está no hace nada.
`editMessage(chatId, messageId, newContent, wasEncrypted)` asíncrona	Edita un mensaje conservando su cifrado. Antes el texto nuevo se escribía en claro en `content`: quedaba legible en el servidor y el destinatario seguía viendo el texto antiguo, porque se descifraba el `encryptedPayload` sin cambios.
`sendMessageAs(chatId, senderId, content)` asíncrona	Escribe un mensaje atribuido a OTRO remitente (p. ej. el bot de bienvenida "Azel Assistant"), en texto plano y sin cifrado E2EE. El chat de demo ya existe cuando se llama a esto, así que se omite toda la lógica de creación/índice de sendMessageInternal. La escritura la ejecuta el usuario autenticado —que ES miembro del chat—, de modo que las reglas la permiten aunque el `senderId` sea el del asistente.
`blockUser(chatId, otherUid)` asíncrona	Bloquea a otherUid en chatId: deja de poder escribir en esa conversacion.
`unblockUser(chatId, otherUid)` asíncrona	Deshace el bloqueo. Solo funciona para quien bloqueo: la regla impide que el bloqueado se libere.
`isBlockedByMe(chatId, otherUid)` asíncrona	`true` si el usuario actual tiene bloqueado a otherUid en chatId. Ante un fallo de lectura devuelve `false`: es el valor que deja al usuario intentar la accion, y la regla del servidor decide de verdad.
`observeChatBlocks(chatId)`	Uids bloqueados en chatId, en tiempo real.
`getBlockedContacts()` asíncrona	Recorre los chats del usuario y devuelve a quien tiene bloqueado, con el chatId donde ocurre para poder deshacerlo desde Ajustes.

`getUserChats(userId)`	Los chats del usuario, en vivo: cualquier mensaje nuevo actualiza la lista. Se escucha campo por campo y no el nodo del chat, que contiene todo su historial: antes se leía entero una sola vez y la lista no se enteraba de los mensajes nuevos, así que al salir de un chat seguía mostrando el mensaje anterior.
`publish()`	Sin comentario en el código. Por su nombre: publish.
`watch(chatId)`	Sin comentario en el código. Por su nombre: watch.
`settled()` interna	Sin comentario en el código. Por su nombre: settled.
`unwatch(chatId)`	Sin comentario en el código. Por su nombre: unwatch.
`setChatBooleanField(chatId, field, value)` asíncrona	Sin comentario en el código. Por su nombre: set chat boolean field.
`observeChatSettings(userId)`	Escucha los ajustes de todos los chats del usuario actual.
`toChatSettings(chatId)` interna	Sin comentario en el código. Por su nombre: to chat settings.
`getChatSettings(userId, chatId)` asíncrona	Lectura puntual, para quien no puede suscribirse (el servicio de notificaciones).
`setChatPinned(chatId, pinned)` asíncrona	Sin comentario en el código. Por su nombre: set chat pinned.
`setChatArchived(chatId, archived)` asíncrona	Sin comentario en el código. Por su nombre: set chat archived.
`setChatMuted(chatId, muteUntil)` asíncrona	Silencia el chat hasta muteUntil. para que no caduque. Pasar `null` lo reactiva.
`deleteChat(chatId)` asíncrona	Borra una conversación entera. Por qué no basta con quitar `chats/{chatId}` La pantalla de inicio no escucha `chats`: escucha el índice `userChats/{uid}` (ver getUserChats). Borrar sólo el nodo del chat dejaba el índice intacto, así que el listener no se disparaba y **la conversación seguía en la lista hasta reiniciar la app**: exactamente el "toco eliminar y no borra" que se reportó. Además el índice acumulaba entra
`clearChatMessages(chatId)` asíncrona	Vacía los mensajes de un chat pero conserva la conversación. Se borra el subárbol `messages` y se reinicia la vista previa, para que la lista de inicio no siga enseñando el último mensaje de algo que ya no existe.
`getUserCallHistory(userId)`	Historial de llamadas del usuario, leído del índice `userCalls/{uid}`. Antes consultaba `calls` con orderByChild("callerId"): esa consulta se ejecuta SOBRE el nodo `calls`, y Firebase exige permiso de lectura ahí. Las reglas solo lo dan en `calls/$callId`, así que la consulta se denegaba siempre y el historial salía vacío. Además solo miraba `callerId`, por lo que las llamadas recibidas nunca habrían aparecido.
`getAllStories()`	Sin comentario en el código. Por su nombre: get all stories.
`getStoriesForUser(userId)`	Lee SOLO las stories de un usuario concreto (`/stories/{userId}`) en lugar de todo el árbol `/stories`. Esto evita el "Permission denied" al ver la story de otra persona cuando las reglas de Firebase no permiten leer el nodo raíz de stories, y es además más eficiente (menos datos y menos exigencia de permisos).
`addReaction(chatId, messageId, reaction)` asíncrona	Sin comentario en el código. Por su nombre: add reaction.
`createGroup(groupName, members)` asíncrona	Sin comentario en el código. Por su nombre: create group.
`searchUserByUsername(username)` asíncrona	Sin comentario en el código. Por su nombre: search user by username.
`sendFriendRequest(targetUserId)` asíncrona	Sin comentario en el código. Por su nombre: send friend request.
`updateProfilePhoto(userId, photoUrl)` asíncrona	Sin comentario en el código. Por su nombre: update profile photo.
`createStory(mediaUrl, mediaType, isVideo, caption, text, backgroundColor)` asíncrona	Sin comentario en el código. Por su nombre: create story.
`sendEphemeralMediaMessage(chatId, mediaUrl, mediaType, caption, isViewOnce, selfDestructDuration)` asíncrona	Sin comentario en el código. Por su nombre: send ephemeral media message.
`sendMediaMessage(chatId, mediaUrl, mediaType, caption, forwarded, extra)` asíncrona	Sin comentario en el código. Por su nombre: send media message.
`getUserById(userId)` asíncrona	Sin comentario en el código. Por su nombre: get user by id.

`getUserProfile(userId)`	Sin comentario en el código. Por su nombre: get user profile.
`getAllUsers()`	Sin comentario en el código. Por su nombre: get all users.
`updateUserProfile(userId, data)` asíncrona	Sin comentario en el código. Por su nombre: update user profile.
`createCall(callData)` asíncrona	Sin comentario en el código. Por su nombre: create call.
`getCallData(callId)` asíncrona	Sin comentario en el código. Por su nombre: get call data.
`listenToCall(callId)`	Sin comentario en el código. Por su nombre: listen to call.
`updateCallStatus(callId, status)` asíncrona	Sin comentario en el código. Por su nombre: update call status.
`setCallOffer(callId, offer)` asíncrona	Sin comentario en el código. Por su nombre: set call offer.
`setCallAnswer(callId, answer)` asíncrona	Sin comentario en el código. Por su nombre: set call answer.
`addIceCandidate(callId, candidate)` asíncrona	Sin comentario en el código. Por su nombre: add ice candidate.
`observeIceCandidates(callId)`	Candidatos ICE del nodo de la llamada, UNO A UNO según llegan. La versión anterior emitía la lista acumulada en cada `onChildAdded`, y el ViewModel aplicaba la lista entera cada vez: el primer candidato se añadía N veces, el segundo N-1… Con 20-30 candidatos por llamada eran cientos de `addIceCandidate` repetidos durante el arranque de la conexión.
`markCallConnected(callId)` asíncrona	Marca el instante en que el medio quedó conectado (base de la duración real).
`endCall(callId, endedBy, durationSec)` asíncrona	Cierra la llamada. - No pisa un estado final: si el otro ya la rechazó (DECLINED) o la terminó, se respeta. Antes el emisor escribía ENDED encima de DECLINED y el historial mostraba "terminada" en vez de "rechazada". - Escribe `endTime` (lo que lee el historial; antes se escribía `endedAt` y la duración nunca aparecía), `endedBy` y `durationSec` medida desde que el medio se conectó, no desde que empezó a sonar.
`uploadFile(uri, folder, fileName)` asíncrona	Sin comentario en el código. Por su nombre: upload file.
`sendImageMessage(chatId, imageUri, caption)` asíncrona	Sin comentario en el código. Por su nombre: send image message.
`sendVideoMessage(chatId, videoUri, caption, duration)` asíncrona	Sin comentario en el código. Por su nombre: send video message.
`sendDocumentMessage(chatId, documentUri, fileName, fileSize, mimeType)` asíncrona	Sin comentario en el código. Por su nombre: send document message.
`sendAudioMessage(chatId, audioUri, duration)` asíncrona	Sin comentario en el código. Por su nombre: send audio message.
`sendLocationMessage(chatId, latitude, longitude, address)` asíncrona	Sin comentario en el código. Por su nombre: send location message.
`sendStickerMessage(chatId, stickerUrl, stickerPack, forwarded, replyTo)` asíncrona	Sin comentario en el código. Por su nombre: send sticker message.
`markStoryAsViewed(storyId)` asíncrona	Sin comentario en el código. Por su nombre: mark story as viewed.
`markMessagesAsRead(chatId)` asíncrona	Marca como leídos los mensajes ENTRANTES (no propios) de un chat que aún no estén en "read". Se invoca mientras el usuario tiene el chat abierto. Las reglas permiten a cualquier miembro del chat escribir en /messages, así que el receptor puede actualizar el status. Solo escribe los que cambian, para evitar writes y re-emisiones innecesarias.
`sendStoryReply(storyOwnerId, storyId, content)` asíncrona	Responde a una historia: el mensaje llega al chat privado 1:1 con el autor (se crea si no existe, vía sendMessageInternal) con referencia a la historia en replyTo ("story:{storyId}") para que la UI muestre la cita. NOTA de firma: el destinatario va PRIMERO. La versión anterior recibía (storyId, recipientId, …) y el ViewModel pasaba (ownerId, storyId, …) → el chatId se construía con el storyId y la respuesta iba a un
`sendStoryStickerReply(storyOwnerId, storyId, sticker)` asíncrona	Respuesta a una historia con un sticker.

`addStoryReaction(storyOwnerId, storyId, emoji)` asíncrona	Reacción de emoji a una historia. Se guarda en stories_reactions/{ownerId}/{storyId}/{uid} = { emoji, timestamp } — espejo del patrón de stories_views (el nodo de la historia solo lo puede escribir el dueño según las rules, por eso las reacciones viven aparte). Una reacción por usuario: reaccionar de nuevo reemplaza el emoji.
`getStoryReactions(storyOwnerId, storyId)` asíncrona	Reacciones de una historia como mapa uid → emoji (para el sheet del autor y el contador). Resolver nombres/fotos es responsabilidad del caller (igual que getStoryViewers).
`getStoryViewerIds(storyId)` asíncrona	UIDs que han visto una historia (claves de stories_views/{storyId}). Fuente de verdad para el estado "visto/no visto" y el conteo de vistas.
`getStoryViewers(storyId)` asíncrona	Lista de espectadores con perfil (nombre + foto). Antes casteaba el timestamp de cada vista a Map<String, Any> y devolvía SIEMPRE lista vacía; ahora lee los UID (claves) y resuelve cada perfil con getUserById.
`deleteStory(storyId, ownerId, mediaUrl)` asíncrona	Elimina una historia propia: borra el nodo en Realtime Database y, si es media (imagen/video), intenta borrar también el archivo en Storage (best-effort). Solo el dueño puede borrar (lo exigen las reglas y lo validamos aquí antes de tocar la red).
`getChatMediaGallery(chatId)` asíncrona	Sin comentario en el código. Por su nombre: get chat media gallery.
`updatePresence(isOnline)` asíncrona	Publica la presencia del usuario actual. Por qué se escriben DOS campos En la base de datos convivían dos banderas distintas con el mismo significado aparente: - `online`, la de verdad: la escribía esta función y tenía un `onDisconnect()` que la ponía a `false` al perderse la conexión. - `isOnline`, la que lee **toda la interfaz** (Home, chat, perfil, llamadas, nueva conversación): se ponía a `true` al iniciar sesión
`observeUserPresence(userId)`	Sin comentario en el código. Por su nombre: observe user presence.
`sendFcmForMessage(chatId, senderId, content, mediaType, senderPhotoUrl)` interna	No hace nada a propósito: las notificaciones las envía la Cloud Function `onMessageCreate` (functions/index.js), que se dispara con cada escritura en `chats/{chatId}/messages/{messageId}` — es decir, con TODOS los tipos de mensaje que llamaban aquí (texto, foto, vídeo, audio, documento, ubicación, sticker). Se eliminó el envío desde el cliente por dos razones, cada una decisiva: 1. Usaba `https://fcm.googleapis.com/f
`observeTyping(chatId)`	Sin comentario en el código. Por su nombre: observe typing.
`setTypingStatus(chatId, isTyping)` asíncrona	Sin comentario en el código. Por su nombre: set typing status.
`updateUserChatsIndex(chatId, memberIds)` interna asíncrona	🔥 CRÍTICO: Helper function to maintain userChats index. Escribe userChats/{uid}/{chatId} = true para cada miembro. Esto permite listado eficiente de chats sin leer todo /chats. **SIN ESTA FUNCIÓN, LOS CHATS NO APARECEN EN HOMESCREEN** Estructura en Firebase: /userChats /{userId1} /{chatId}: true /{userId2} /{chatId}: true

SecurityRepositoryImpl.kt

`data/repository/SecurityRepositoryImpl.kt · 40 líneas`

Implementación de SecurityRepository que delega operaciones a TorService (detección de Orbot) en lugar de un TorServiceManager embebido.

`startTor()`	Sin comentario en el código. Por su nombre: start tor.
`stopTor()` asíncrona	Sin comentario en el código. Por su nombre: stop tor.
`getTorState()`	Sin comentario en el código. Por su nombre: get tor state.
`getTorStatus()`	Sin comentario en el código. Por su nombre: get tor status.

StorageRepository.kt

`data/repository/StorageRepository.kt · 464 líneas`

Metadata explícita con contentType para cada subida. putFile() sobre un URI file:// (archivos temporales de historias compuestas, crops, videos con overlays, etc.) no puede resolver el MIME por ContentResolver y sube como

application/octet-stream — y las reglas de Storage que exigen tipo imagen o video rechazan esa subida con permission denied. Ese era el motivo por el que las historias con foto o video no se publicaban.

`mediaMetadata(contentType)` interna	Metadata explícita con contentType para cada subida. putFile() sobre un URI file:// (archivos temporales de historias compuestas, crops, videos con overlays, etc.) no puede resolver el MIME por ContentResolver y sube como application/octet-stream — y las reglas de Storage que exigen tipo imagen o video rechazan esa subida con permission denied. Ese era el motivo por el que las historias con foto o video no se publica
`getFileSizeBytes(uri)`	Tamaño en bytes de un content:// URI (−1 si no se puede determinar). Permite validar límites antes de subir a Storage.
`uploadChatImage(imageUri, chatId)` asíncrona	Upload image to Firebase Storage Pattern: chats/{chatId}/images/{timestamp}.jpg
`uploadChatAudio(audioUri, chatId)` asíncrona	Upload audio to Firebase Storage Pattern: chats/{chatId}/audio/{timestamp}.aac
`uploadChatVideo(videoUri, chatId)` asíncrona	Upload video to Firebase Storage Pattern: chats/{chatId}/videos/{timestamp}.mp4
`uploadUserBackground(uri, userId, isVideo)` asíncrona	Upload an app-wide background (image or video) for the current user. Pattern: user_backgrounds/{userId}/{timestamp}.{jpg|mp4} Used by the APP-scope wallpaper so the reference survives process death and permission revocation — a local content:// URI does not, which is why picking an image/video for the background "did not work" before.
`putFileAndGetUrl(path, uri, contentType)` interna asíncrona	Sube un archivo sin recomprimirlo y devuelve su URL.
`uploadChatGif(uri, chatId)` asíncrona	GIF de un chat, tal cual: recomprimirlo o etiquetarlo como JPEG perdería la animación.
`uploadStoryGif(uri, userId)` asíncrona	GIF publicado como historia.
`uploadStory(imageUri, userId)` asíncrona	Upload story to Firebase Storage Pattern: stories/{userId}/{timestamp}.jpg
`uploadStoryMusic(audioUri, userId)` asíncrona	Sube la pista de audio que acompaña a una historia. Pattern: stories/{userId}/{timestamp}_music Va bajo `stories/{userId}/` a propósito: así reutiliza exactamente las mismas reglas de Storage que la imagen o el vídeo de la historia, sin abrir una ruta nueva que habría que autorizar aparte.
`uploadProfilePhoto(imageUri, userId)` asíncrona	Upload profile photo to Firebase Storage Pattern: user_photos/{userId}/profile.jpg
`uploadCoverPhoto(imageUri, userId)` asíncrona	Upload cover photo to Firebase Storage Pattern: user_photos/{userId}/cover.jpg
`uploadChatDocument(documentUri, chatId, fileName)` asíncrona	Upload document to Firebase Storage Pattern: chats/{chatId}/documents/{timestamp}_{fileName}
`uploadStoryVideo(videoUri, userId, onProgress)` asíncrona	Upload story video to Firebase Storage Pattern: stories/{userId}/{timestamp}.mp4
`deleteFile(downloadUrl)` asíncrona	Delete file from Firebase Storage

UserRepository.kt

`data/repository/UserRepository.kt · 152 líneas`

`getUserById(userId)` asíncrona	Sin comentario en el código. Por su nombre: get user by id.
`updateProfile(user)` asíncrona	Sin comentario en el código. Por su nombre: update profile.
`updateOnlineStatus(userId, isOnline)` asíncrona	Sin comentario en el código. Por su nombre: update online status.
`searchUsers(query)` asíncrona	Sin comentario en el código. Por su nombre: search users.
`getAllUsers()` asíncrona	Sin comentario en el código. Por su nombre: get all users.
`observeUserStatus(userId)`	Sin comentario en el código. Por su nombre: observe user status.

Parte 23

Seguridad y cifrado

Cifrado de extremo a extremo, chats secretos y Tor.

AppLockManager.kt

`data/security/AppLockManager.kt · 132 líneas`

AppLockManager - Gestiona el estado de bloqueo de la aplicación Funcionalidades: - Verificación de PIN - Control de sesión de desbloqueo - Auto-bloqueo basado en tiempo - Estado de bloqueo reactivo Requirements: 9.1, 9.2, 9.3, 9.4

`shouldLockOnResume()` asíncrona	Verifica si la app debe estar bloqueada cuando vuelve al primer plano
`lock()`	Marca la app como bloqueada
`unlock()`	Desbloquea la app después de verificar el PIN correctamente
`updateLastActiveTime()`	Actualiza el timestamp cuando la app pasa a segundo plano
`verifyPin(pin)` asíncrona	Verifica si un PIN es correcto
`isLockEnabled()` asíncrona	Verifica si el bloqueo está habilitado en la configuración
`isBiometricEnabled()` asíncrona	Verifica si la biometría está habilitada
`is2faEnabled()` asíncrona	`true` si el segundo factor (TOTP) está activo. Cuando lo está, desbloquear exige, además del PIN o la huella, el código de 6 dígitos de la app de autenticación. Así el 2FA deja de ser decorativo.
`verifyTotp(code)` asíncrona	Comprueba un código TOTP contra el secreto guardado (±30 s de tolerancia).
`reset()`	Resetea el estado de bloqueo (usado al cerrar sesión)

GhostMode.kt

`data/security/GhostMode.kt · 37 líneas`

Modo fantasma: lees y escribes sin que nadie vea que estás conectado, que escribes o que has leído. A diferencia del modo anónimo no pasa por Tor, así que tu IP no cambia.

`set(on)`	Sin comentario en el código. Por su nombre: set.

IntegrityGuard.kt

`data/security/IntegrityGuard.kt · 176 líneas`

IntegrityGuard — firma de autoría y anti-manipulación de NexusChat (Azel Mods). Qué hace Comprueba, en el arranque, que el APK esté firmado con el certificado del autor. Si alguien recompila la app, la reempaqueta o la vuelve a firmar con OTRA clave (que es lo que hay que hacer para redistribuirla como propia), la firma cambia y, en un build de RELEASE, la app se autodestruye con un fallo. Es una marca de propiedad que viaja dentro del binario: dice "esto es de Azel Mods" de una forma que no se puede borrar simplemente cambiando un texto en pantalla. La "criptografía oculta" La huella del certificado NO se guarda en claro. Se guarda **el SHA-256 de la huella** (DEBUG_MARK / RELEASE_MARK). En

`owner()`	Reconstruye el nombre del autor en memoria a partir del blob ofuscado.
`ownershipIntact()` interna	`true` si el nombre reconstruido casa con su hash: nadie tocó el blob ni la clave.
`verify(context)`	Punto de entrada. Verifica DOS cosas —la firma del APK y la marca de autoría— y, si alguna no cuadra en un release configurado, programa la autodestrucción. Cualquier error de lectura se traga: un fallo del guard nunca debe tumbar la app por sí mismo (solo lo hace una manipulación real en release).
`signingFingerprints(context)` interna	Huellas SHA-256 (hex, mayúsculas) de los certificados con los que se firmó el APK.

`mark(fingerprint)` interna	Hash de la huella: lo que realmente se guarda y compara (la huella nunca en claro).
`sha256Hex(bytes)` interna	Sin comentario en el código. Por su nombre: sha256 hex.
`selfDestruct()` interna	Autodestrucción: un fallo fatal diferido. Se lanza desde el bucle principal con un retardo para que el crash no apunte de inmediato a esta comprobación.

TotpAuthenticator.kt

`data/security/TotpAuthenticator.kt · 134 líneas`

TOTP (RFC 6238) — segundo factor compatible con Google Authenticator, Aegis, etc. Es criptografía estándar, sin dependencias ni servidor: el secreto se genera y se guarda en el dispositivo, y el código de 6 dígitos se deriva del reloj. La misma semilla en la app de autenticación del usuario produce el mismo código, así que cualquier app TOTP sirve como segundo factor. Por eso el 2FA de NexusChat deja de ser un adorno: hay un secreto real que verificar. - HMAC-SHA1 (el algoritmo por defecto de las apps de autenticación). - Ventana de 30 s. - 6 dígitos. - Verificación con tolerancia de ±1 ventana para absorber el desfase de reloj.

`generateSecret()`	Genera un secreto nuevo en Base32 (160 bits), el formato que esperan las apps TOTP.
`currentCode(secretBase32, atMillis)`	Código actual para secretBase32. Cadena de 6 dígitos con ceros a la izquierda.
`verify(secretBase32, code, atMillis)`	`true` si code es válido ahora mismo, admitiendo la ventana anterior y la siguiente (±30 s) para que un reloj ligeramente desajustado no bloquee al usuario.
`secondsRemaining(atMillis)`	Segundos que le quedan de vida al código actual (para la barra de cuenta atrás).
`provisioningUri(secretBase32, account, issuer)`	URI `otpauth://` que las apps de autenticación leen desde un QR o al pegarla. account suele ser el usuario/teléfono; issuer es el nombre de la app.
`codeForCounter(secretBase32, counter)` interna	Sin comentario en el código. Por su nombre: code for counter.
`constantTimeEquals(a, b)` interna	Sin comentario en el código. Por su nombre: constant time equals.
`encode(data)`	Sin comentario en el código. Por su nombre: encode.
`decode(encoded)`	Sin comentario en el código. Por su nombre: decode.

E2EECryptoService.kt

`data/security/encryption/E2EECryptoService.kt · 364 líneas`

E2EE: ECDH (curva elíptica del sistema) + AES-256-GCM. Claves públicas en Firebase: users/{uid}/keys/identityPublic Por qué las claves se guardan POR UID La versión anterior guardaba un único par de claves bajo claves fijas (`identity_private_pkcs8`), compartidas por todas las cuentas que hubieran iniciado sesión en el dispositivo. Con dos cuentas —algo normal al probar la app— la segunda reutilizaba la clave privada de la primera mientras Firebase seguía publicando bajo su uid una clave pública que ya no le correspondía. El secreto ECDH que derivaba el otro extremo no coincidía, la etiqueta GCM fallaba y todos los mensajes se leían como "No se pudo descifrar". Ahora cada uid tiene su propio

`privateKeyPref(uid)` interna	Sin comentario en el código. Por su nombre: private key pref.
`publicKeyPref(uid)` interna	Sin comentario en el código. Por su nombre: public key pref.
`ensureLocalKeys()` asíncrona	Sin comentario en el código. Por su nombre: ensure local keys.
`publishPublicKeyIfNeeded(userId, publicB64)` interna asíncrona	Sube la clave pública sólo si el servidor no tiene ya exactamente esa. Marca el uid como reconciliado únicamente cuando la escritura (o la comprobación) termina bien, para que un fallo de red se reintente en el siguiente mensaje en lugar de quedarse colgado para siempre.
`hasLocalKeys()`	Sin comentario en el código. Por su nombre: has local keys.
`clearSessionCache()`	Olvida los secretos derivados. Debe llamarse al cerrar sesión: si no, la siguiente cuenta que entre en el mismo proceso seguiría usando los secretos de la anterior.

`invalidatePeer(localUid, peerId)` interna	Invalida el secreto con un interlocutor concreto. Se usa cuando un descifrado falla: si el otro extremo rotó su clave (reinstaló la app), el secreto en caché quedó obsoleto y sin esto no se recuperaría nunca sin matar el proceso.
`safetyNumber(peerId)` asíncrona	60 dígitos en grupos, iguales en los dos teléfonos: resumen de las dos claves públicas en orden fijo. null si la otra persona aún no tiene claves.
`getLocalKeyPair(userId)` interna	Sin comentario en el código. Por su nombre: get local key pair.
`fetchRemotePublicKeyBytes(userId)` interna asíncrona	Sin comentario en el código. Por su nombre: fetch remote public key bytes.
`sharedAesKey(localUid, peerId)` interna asíncrona	Sin comentario en el código. Por su nombre: shared aes key.
`encryptFor(peerId, plaintext)` asíncrona	Sin comentario en el código. Por su nombre: encrypt for.
`decryptFrom(peerId, ciphertext)` asíncrona	Descifra un payload intercambiado con peerId. peerId es **el otro extremo de la conversación**, no "quien envió el mensaje": el secreto ECDH es simétrico, así que el mismo par de claves descifra tanto lo que llega como lo que uno mismo escribió. Gracias a eso el emisor puede releer sus propios mensajes en lugar de ver el marcador " 🔒 Mensaje cifrado de extremo a extremo" que se guarda en el servidor.
`decryptOnce(localUid, peerId, ciphertext)` interna asíncrona	renovado (clave ausente o etiqueta GCM inválida).
`safetyDigits(digest)`	Convierte un resumen en 12 grupos de 5 dígitos, fáciles de comparar en voz alta.

GroupCrypto.kt

`data/security/encryption/GroupCrypto.kt · 101 líneas`

Mensaje de grupo ya cifrado, listo para guardar.

`cacheName(chatId, keyId)` interna	Sin comentario en el código. Por su nombre: cache name.
`encrypt(chatId, text)` asíncrona	null si algún miembro aún no tiene claves: entonces el mensaje no se puede cifrar para todos.
`decrypt(chatId, keyId, payload)` asíncrona	Sin comentario en el código. Por su nombre: decrypt.
`currentKey(chatId)` interna asíncrona	Sin comentario en el código. Por su nombre: current key.
`createKey(chatId, members)` interna asíncrona	Sin comentario en el código. Por su nombre: create key.
`fetchKey(chatId, keyId)` interna asíncrona	Sin comentario en el código. Por su nombre: fetch key.

SignalProtocolManager.kt

`data/security/encryption/SignalProtocolManager.kt · 99 líneas`

Punto de entrada del cifrado de extremo a extremo de la app. Qué pasó con el Signal Protocol Este archivo describía una implementación del Signal Protocol completa —Double Ratchet, PreKeys, secreto hacia adelante— y arrastraba dos clases de apoyo (`SignalKeyStore`, 418 líneas, y `PreKeyManager`, 323) más la dependencia `libsignal-android`. Nada de eso llegó a ejecutarse: el cifrado real siempre lo hizo E2EECryptoService con ECDH P-256 + AES-256-GCM, y los métodos de PreKeys devolvían `0`, `false` o `null` sin hacer nada. Se han eliminado en la v6, porque un andamiaje que sólo se referencia a sí mismo confunde a quien lee el código y, peor, sugiere garantías criptográficas que la app no da. L

`initialize(userId)` asíncrona	Genera y publica el par de claves del usuario si aún no existe.
`encryptMessage(recipientId, plaintext)` asíncrona	Cifra plaintext para recipientId.
`decryptMessage(senderId, ciphertext, messageType)` asíncrona	Descifra un payload intercambiado con senderId. Ojo: senderId es el **otro extremo del chat**, no necesariamente quien envió el mensaje. Ver E2EECryptoService.decryptFrom.

SecretChatRepository.kt

`data/security/secret/SecretChatRepository.kt · 332 líneas`

Chats secretos: viven solo en los dos teléfonos. El servidor es un mero cartero: guarda el texto cifrado hasta que se entrega y lo borra a continuación. Los mensajes quedan en un archivo cifrado en cada teléfono, no hay copia en la nube y no se pueden abrir en otro dispositivo: perder el teléfono es perder el chat.

`messages(chatId)`	Sin comentario en el código. Por su nombre: messages.
`observeChats()`	Los chats secretos de este usuario, con los datos de la otra persona.
`start(peerId)` asíncrona	Crea el chat con peerId y publica la clave pública de este teléfono.
`establish(chatId, keysNode, peerId)` interna asíncrona	Acuerda las claves del chat. Quien lo recibe publica su clave al abrirlo por primera vez; quien lo creó deriva la raíz cuando esa clave aparece. Se llama cada vez que cambia el nodo.
`connect(chatId, peerId)`	Mientras se observa, recibe y descifra mensajes y los borra del servidor.
`receive(chatId, peerId, snapshot)` interna	Sin comentario en el código. Por su nombre: receive.
`send(chatId, text)` asíncrona	Sin comentario en el código. Por su nombre: send.
`timerSeconds(chatId)`	Sin comentario en el código. Por su nombre: timer seconds.
`setTimer(chatId, seconds)`	Sin comentario en el código. Por su nombre: set timer.
`purgeExpired(chatId)`	Sin comentario en el código. Por su nombre: purge expired.
`delete(chatId)` asíncrona	Borra el chat de este teléfono y del servidor.
`append(chatId, message)` interna	Sin comentario en el código. Por su nombre: append.
`loadLocal(chatId)` interna	Sin comentario en el código. Por su nombre: load local.
`saveLocal(chatId, messages)` interna	Sin comentario en el código. Por su nombre: save local.
`encode(bytes)` interna	Sin comentario en el código. Por su nombre: encode.
`decode(text)` interna	Sin comentario en el código. Por su nombre: decode.
`privateKeyName(id)` interna	Sin comentario en el código. Por su nombre: private key name.
`rootName(id)` interna	Sin comentario en el código. Por su nombre: root name.
`sendChainName(id)` interna	Sin comentario en el código. Por su nombre: send chain name.
`receiveChainName(id)` interna	Sin comentario en el código. Por su nombre: receive chain name.
`sendCountName(id)` interna	Sin comentario en el código. Por su nombre: send count name.
`receiveCountName(id)` interna	Sin comentario en el código. Por su nombre: receive count name.
`skippedName(id, n)` interna	Sin comentario en el código. Por su nombre: skipped name.
`timerName(id)` interna	Sin comentario en el código. Por su nombre: timer name.
`messagesName(id)` interna	Sin comentario en el código. Por su nombre: messages name.

SecretRatchet.kt

`data/security/secret/SecretRatchet.kt · 79 líneas`

Cifrado de los chats secretos. - Cada chat acuerda su propio secreto con claves efímeras (ECDH P-256) que solo existen en los dos teléfonos, y la privada se destruye en cuanto se ha usado. - Cada mensaje usa una clave distinta, derivada de una cadena que avanza y olvida la anterior (HMAC-SHA256): quien consiga el estado actual no puede leer los mensajes pasados.

`newKeyPair()`	Sin comentario en el código. Por su nombre: new key pair.
`sharedRoot(privateKey, peerPublicKey, chatId)`	Sin comentario en el código. Por su nombre: shared root.
`chainFor(root, senderUid)`	Cada participante cifra con su propia cadena, así nunca se reutiliza una clave.

`advance(chain)`	Sin comentario en el código. Por su nombre: advance.
`seal(messageKey, plaintext, context)`	Devuelve IV + texto cifrado. context liga el mensaje a su chat, remitente y número.
`open(messageKey, sealed, context)`	Lanza una excepción si el mensaje se alteró o la clave no es la suya.
`hmac(key, data)` interna	Sin comentario en el código. Por su nombre: hmac.

AnonymityPolicy.kt

`data/security/tor/AnonymityPolicy.kt · 25 líneas`

Qué actividad deja de publicarse. Ir por Tor oculta la IP, pero no sirve de mucho si la app sigue publicando cuándo estás conectado, cuándo escribes y cuándo lees. Mientras hidesActivity sea cierto: - No se publica presencia ni última conexión. - No se envía el indicador de "escribiendo". - No se envían confirmaciones de lectura. Lo activa el modo anónimo desde que se enciende, aunque Tor aún esté conectando: esos datos no deben salir "mientras tanto". El modo fantasma hace lo mismo sin pasar por Tor.

EmbeddedTor.kt

`data/security/tor/EmbeddedTor.kt · 163 líneas`

Progreso de arranque de Tor, tal como lo informa el propio Tor.

`onServiceConnected(name, binder)`	Sin comentario en el código. Por su nombre: on service connected.
`onServiceDisconnected(name)`	Sin comentario en el código. Por su nombre: on service disconnected.
`start()`	Sin comentario en el código. Por su nombre: start.
`removeStaleControlSocket()` interna	Si Android mató la app, queda el socket de control de la ejecución anterior. La librería lo da por bueno, se conecta antes de que Tor escuche, recibe "Connection refused" y no vuelve a intentarlo: Tor se queda sin control hasta reiniciar la app.
`stop()`	Sin comentario en el código. Por su nombre: stop.
`socksPort()`	Puerto SOCKS real, o null si Tor aún no lo ha abierto.
`bootstrap()`	Progreso actual. Null mientras el puerto de control aún no responde.
`info(key)` interna	Sin conexión de control la librería registra un error por cada consulta: se evita.
`pinExit(fingerprint)`	Obliga a salir por este nodo. No se guarda: al reiniciar Tor vuelve a elegirse.
`clearExit()`	Vuelve a dejar que Tor elija la salida.
`exitOfIsolation(isolation)`	Huella del nodo de salida del circuito abierto para este usuario SOCKS.
`currentCircuit()`	Nodos del circuito de uso general más reciente.
`parseBootstrap(phase)`	"NOTICE BOOTSTRAP PROGRESS=45 TAG=requesting_descriptors SUMMARY=..."
`describe(tag, percent)` interna	Sin comentario en el código. Por su nombre: describe.
`parseExitFor(status, isolation)`	`12 BUILT $FP~a,$FP~b,$FP~salida PURPOSE=GENERAL SOCKS_USERNAME="x"`.
`parseCircuit(status)`	Última línea BUILT de propósito general: "12 BUILT $FP~nick,$FP~nick,... PURPOSE=GENERAL".

FirebaseProxyConfigurator.kt

`data/security/tor/FirebaseProxyConfigurator.kt · 182 líneas`

🔥 FirebaseProxyConfigurator — Configura Firebase para funcionar a través de Tor ¿Por qué es necesario? Aunque TorProxySelector enruta el tráfico HTTP global por Tor, Firebase tiene su propia gestión de conexiones y puede **cachear** sockets abiertos antes de que Tor esté activo. Este configurador: 1. Ajusta timeouts de Firebase para la latencia adicional de Tor 2. Habilita persistencia offline (ya configurada en FirebaseModule) 3. Asegura que las conexiones de Firebase pasen por el ProxySelector correcto 4. Proporciona health checks específicos de Firebase a través de Tor Uso El TorService llama a enableTorMode cuando Tor se conecta y a disableTorMode cuando Tor se desconecta.

`enableTorMode()` asíncrona	Activa el modo Tor para Firebase. Efectos: - Aumenta timeouts para conexiones lentas de Tor - Marca la bandera isTorActive - Fuerza reconexión de Firebase (desconecta y reconecta)
`disableTorMode()` asíncrona	Desactiva el modo Tor para Firebase. Efectos: - Restaura timeouts normales - Marca la bandera isTorActive como false - Reconecta Firebase para que use la red directa
`checkFirebaseConnectivity()` asíncrona	Verifica que Firebase pueda alcanzar sus servidores a través de Tor. Intenta leer un path conocido (users) desde Firebase. Si Tor está activo pero Firebase no responde, puede indicar que el proxy Tor no está funcionando correctamente.
`checkAuthConnectivity()` asíncrona	Verifica que Firebase Auth pueda contactar sus servidores. Intenta refrescar el token del usuario actual. Si la red no funciona (por ejemplo, Tor no está enrutando correctamente), el refresh fallará.
`getConnectivityStatus()` asíncrona	Obtiene un resumen del estado de conectividad de Firebase a través de Tor.

FirebaseReachability.kt

`data/security/tor/FirebaseReachability.kt · 102 líneas`

Comprueba si los servidores de Firebase aceptan una salida concreta de Tor. Google rechaza la mayoría de nodos de salida: medido en esta app, 8 de cada 10 responden 403 a `securetoken.googleapis.com` o cortan la conexión con Realtime Database. Sin renovar ese token la sesión deja de valer y la app parece vacía, aunque Tor funcione. Cada usuario SOCKS distinto obliga a Tor a usar un circuito distinto, y eso permite probar varias salidas a la vez. Las peticiones no llevan credenciales ni datos de la cuenta: solo importa el código de respuesta.

`databaseHost()`	Sin comentario en el código. Por su nombre: database host.
`isReachable(socksPort, isolation, databaseHost)`	`true` si desde este circuito se llega tanto al token de sesión como a la base de datos.
`httpStatus(socksPort, isolation, host, path)` interna	Código HTTP de una petición por el circuito indicado, o null si no se pudo completar.
`openSocksTunnel(socket, isolation, host)` interna	Protocolo SOCKS5 con usuario y contraseña (RFC 1928 y 1929). El nombre lo resuelve Tor.

OrbotDetector.kt

`data/security/tor/OrbotDetector.kt · 203 líneas`

🔍 OrbotDetector — Utilidad para detectar Orbot en el dispositivo Proporciona métodos para verificar si Orbot está instalado, si el proxy SOCKS5 (puerto 9050) está accesible, y si el proxy HTTP (puerto 8118) está accesible.

`isOrbotInstalled(context)`	Verifica si Orbot está instalado en el dispositivo. Soporta ambos nombres de paquete oficiales: - org.torproject.android (Play Store) - org.torproject.orbot (F-Droid)
`isSocksProxyAvailable(timeoutMs)`	Verifica si el proxy SOCKS5 de Orbot (127.0.0.1:9050) está accesible. Si este puerto responde, Orbot está funcionando y Tor está conectado.
`isHttpProxyAvailable()`	Verifica si el proxy HTTP de Orbot (127.0.0.1:8118) está accesible. Orbot provee un proxy HTTP en el puerto 8118 además del SOCKS5. ✅ FIX: Timeout aumentado a 3s para dispositivos lentos
`isTorAvailable()`	Verificación completa: ¿Orbot funciona y podemos usar Tor?
`isOrbotUsable(context)`	¿Está Orbot presente y utilizable? El orden importa: se pregunta PRIMERO si el proxy responde. Un proxy Tor escuchando en 127.0.0.1 es prueba directa de que Orbot está corriendo, y esa evidencia manda sobre `isOrbotInstalled`, que puede devolver false por restricciones de visibilidad de paquetes (Android 11+) aunque Orbot esté ahí. Debe llamarse desde un hilo de background: abre sockets.
`getStatus(context)`	Obtiene información de estado para mostrar al usuario.
`launchOrbot(context)`	Abre la app de Orbot si está instalada.
`getOrbotStatusInfo(context)`	Obtiene información detallada del estado de Orbot con acciones sugeridas

OrbotIntents.kt

`data/security/tor/OrbotIntents.kt · 117 líneas`

API pública de Orbot por intents (la misma que usa NetCipher/OrbotHelper). Con ella NexusChat puede: - pedir a Orbot que arranque Tor SIN que el usuario cambie de app (requestStart); antes había que abrir Orbot y pulsar "Iniciar" a mano; - saber el estado real (ON/STARTING/OFF…) y los PUERTOS reales que usa Orbot (status). Orbot cambia de puerto si el 9050 está ocupado; con los puertos fijos de antes, en ese caso el modo anónimo "no detectaba" Orbot.

`onReceive(context, intent)`	Sin comentario en el código. Por su nombre: on receive.
`register(context)`	Empieza a escuchar el estado de Orbot (idempotente).
`requestStart(context)`	Pide a Orbot que arranque Tor. Orbot responde con un broadcast de estado (y con STARTS_DISABLED si el usuario no lo permite). Devuelve false si no se pudo enviar (Orbot no instalado).
`readPort(intent, key)` interna	Sin comentario en el código. Por su nombre: read port.

OrbotStatus.kt

`data/security/tor/OrbotStatus.kt · 28 líneas`

Estado del servicio Orbot para el modo anónimo

OrbotStatusMapper.kt

`data/security/tor/OrbotStatusMapper.kt · 66 líneas`

🧅 OrbotStatusMapper — Mapeo puro del estado de Orbot a UI clara y accionable. Convierte el par de booleanos `(installed, active)` en un OrbotUiStatus con mensaje en español, etiqueta de acción coherente y un estado de toggle consistente que NUNCA queda "roto" / en error permanente. La función mapOrbotStatus es PURA y TOTAL: definida para las cuatro combinaciones posibles de `(installed, active)`. Property 3 (design.md): el mapeo de estado de Orbot es total y consistente. / /** Estado lógico de Orbot derivado de (installed, active).

`mapOrbotStatus(installed, active)`	Mapea el estado de Orbot a un OrbotUiStatus consistente. Reglas: - `active == true` → OrbotState.ACTIVE, sin acción, toggle operable. - `installed && !active` → OrbotState.INSTALLED_INACTIVE, acción "Abrir Orbot". - `!installed && !active` → OrbotState.NOT_INSTALLED, acción "Descargar Orbot". El toggle queda siempre operable (`toggleEnabled = true`) para permitir reintentar; nunca refleja un estado de error permanent

ProxyManager.kt

`data/security/tor/ProxyManager.kt · 142 líneas`

Resultado de una operación de proxy gestionada por ProxyManager. - Applied: el proxy se aplicó correctamente (la solicitud se entregó a ProxyController). - Cleared: el proxy se limpió correctamente. - Failed: ocurrió un error o la funcionalidad no está soportada; la app debe continuar en modo directo (clearnet) sin crashear.

`applyOrbotProxy(httpHost, httpPort, onApplied)`	Aplica el proxy de Orbot a nivel de WebView. de proxy se ha aplicado. Útil para actualizar la UI o cargar la primera URL. o ProxyResult.Failed si la funcionalidad no está soportada u ocurre una excepción.
`applyProxyRule(proxyUrl, onApplied)`	Aplica una regla de proxy arbitraria a nivel de WebView. `socks5://127.0.0.1:9050`. Orbot expone HTTP en 8118 y SOCKS5 en 9050. override de proxy se ha aplicado. o ProxyResult.Failed si la funcionalidad no está soportada u ocurre una excepción.
`clearProxy(onCleared)`	Limpia la configuración del proxy para restaurar el tráfico directo. de proxy se ha limpiado. o ProxyResult.Failed si la funcionalidad no está soportada u ocurre una excepción.

TorConnectivityChecker.kt

`data/security/tor/TorConnectivityChecker.kt · 67 líneas`

Comprueba de verdad si el tráfico sale por Tor. Antes "Conectado" significaba solo que el puerto de Orbot aceptaba conexiones, y la tarjeta del circuito mostraba datos inventados ("Orbot → Red Tor → Nodo de salida"). Ahora se pregunta al servicio oficial del Proyecto Tor (check.torproject.org), que contesta si la petición le llegó desde un nodo de salida de Tor y con qué IP: eso es lo que ve cualquier web.

`checkViaSocks(port)`	Petición forzada por el SOCKS de Orbot (modo proxy).
`checkDirect()`	Petición SIN proxy. Solo tiene sentido con una VPN activa: si aun así sale por Tor, Orbot está en modo VPN y TODA la app (Firebase incluido) va por Tor.
`isVpnActive(context)`	Sin comentario en el código. Por su nombre: is vpn active.
`check(proxy)` interna	Sin comentario en el código. Por su nombre: check.

TorDnsResolver.kt

`data/security/tor/TorDnsResolver.kt · 131 líneas`

🌐 TorDnsResolver — DNS resolver que enruta consultas DNS a través de Tor ¿Por qué es necesario? Aunque TorProxySelector enruta el tráfico HTTP/HTTPS por Tor, las consultas DNS nativas de Java (InetAddress.getAllByName) NO pasan por el ProxySelector y pueden **filtrar** los dominios que estás visitando a tu ISP incluso cuando Tor está activo. Cómo funciona Cuando Tor está habilitado, `TorDnsResolver.lookup()` no usa el DNS del sistema. En su lugar, resuelve los nombres DNS a través del proxy SOCKS5 de Orbot: 1. Abre una conexión TCP directa a 127.0.0.1:9050 (SOCKS de Orbot) 2. Realiza un handshake SOCKS5 para resolver el dominio 3. Retorna 127.0.0.1 como IP (la conexión real se hace luego por

`lookup(hostname)`	Resuelve un nombre de host a una lista de direcciones IP. - Si isTorEnabled es `true` → retorna LOOPBACK_IP como marcador y la conexión SOCKS5 se encarga del enrutamiento real. - Si isTorEnabled es `false` → delega a Dns.SYSTEM. La razón de retornar loopback es que OkHttp necesita una IP para abrir el socket, pero la IP real no importa porque el socket se abrirá a través del proxy SOCKS que es quien realmente resuelv
`isOrbotAvailable()`	Verifica si Orbot está disponible para resolución DNS.

TorPreferences.kt

`data/security/tor/TorPreferences.kt · 112 líneas`

Manages persistence of Tor settings using SharedPreferences Handles saving and restoring: - Anonymous Mode enabled/disabled state - Bridge configuration (obfs4 bridge addresses) Requirements: 23.1, 23.2, 23.3, 23.4

`setAnonymousModeEnabled(enabled)`	Saves Anonymous Mode preference Requirement 23.1: Save Anonymous Mode preference to SharedPreferences
`isAnonymousModeEnabled()`	Gets Anonymous Mode preference Requirement 23.3: Restore the previous Anonymous Mode state
`isGhostModeEnabled()`	Sin comentario en el código. Por su nombre: is ghost mode enabled.
`setGhostModeEnabled(enabled)`	Sin comentario en el código. Por su nombre: set ghost mode enabled.
`getEngine()`	Motor elegido. Por defecto el integrado: el modo anónimo funciona sin instalar nada.
`setEngine(engine)`	Sin comentario en el código. Por su nombre: set engine.
`saveBridgeConfiguration(bridges)`	Saves bridge configuration Requirement 23.2: Save bridge configuration to SharedPreferences
`getBridgeAddresses()`	Gets saved bridge addresses Requirement 23.3: Restore bridge configuration
`areBridgesConfigured()`	Checks if bridges are configured

`clearAll()`	Clears all Tor preferences

TorProxySelector.kt

`data/security/tor/TorProxySelector.kt · 156 líneas`

🌐 TorProxySelector — ProxySelector global que enruta todo el tráfico HTTP/S a través del proxy SOCKS5 de Orbot cuando Tor está activo. Cómo funciona: - Se instala vía ProxySelector.setDefault cuando Tor se conecta. - Mientras isTorEnabled sea `true`, todas las conexiones HTTP/HTTPS y WebSocket se enrutan a través de 127.0.0.1:9050 (SOCKS5 de Orbot). - Orbot internamente traduce SOCKS5 → Tor, dando anonimato a TODAS las conexiones de red de la app: APIs, Firebase, Coil, etc. - Cuando Tor se desactiva, restore restaura el ProxySelector original.

`setSocksPort(port)`	Sin comentario en el código. Por su nombre: set socks port.
`install(enableTor)`	Instala este ProxySelector como el predeterminado del sistema. A partir de este momento, todas las conexiones HTTP/HTTPS/WebSocket de la app se enrutarán a través de Orbot → Tor.
`restore()`	Restaura el ProxySelector original. Después de llamar a este método, las conexiones HTTP volverán a usar la configuración de red normal del dispositivo.
`setEnabled(enabled)`	Activa o desactiva el enrutamiento Tor sin cambiar el selector instalado.
`select(uri)`	Retorna el proxy a usar para una URI dada. - Si Tor está activo → retorna el proxy SOCKS5 de Orbot - Si Tor está inactivo → delega al selector original o retorna DIRECT
`isLoopback(host)` interna	Sin comentario en el código. Por su nombre: is loopback.
`connectFailed(uri, sa, ioe)`	Callback cuando una conexión a través del proxy falla. Delega al selector original si existe.

TorService.kt

`data/security/tor/TorService.kt · 411 líneas`

Qué motor pone Tor.

`startTor()`	Activa el modo anónimo. Idempotente.
`connectEmbedded()` interna asíncrona	Sigue el arranque real de Tor por su puerto de control.
`connectOrbot()` interna asíncrona	Pide a Orbot que arranque y espera a que su SOCKS responda.
`onTorReady(port)` interna asíncrona	Sin comentario en el código. Por su nombre: on tor ready.
`findFirebaseExit(port, onAttempt)` interna asíncrona	Prueba varias salidas a la vez y fija la primera que Firebase acepte. Cada circuito de prueba tarda segundos y la mayoría falla, así que en paralelo se tarda mucho menos.
`newIsolation()` interna	Sin comentario en el código. Por su nombre: new isolation.
`routeTrafficTo(port)` interna	Sin comentario en el código. Por su nombre: route traffic to.
`verifyConnection()`	Pregunta al Proyecto Tor si el tráfico sale por su red y con qué IP, y actualiza el circuito y qué partes de la app están pasando realmente por Tor.
`newIdentity()`	Busca otra salida compatible y circuitos nuevos: cambia la IP con la que se te ve.
`markFirebaseReachable(reachable)` interna	Sin comentario en el código. Por su nombre: mark firebase reachable.
`startWatchdog(startDown)` interna	Vigila que Tor siga vivo. Si cae, el tráfico sigue bloqueado hasta que vuelva.
`recheckFirebaseExit()` interna asíncrona	Google puede empezar a bloquear la salida fijada: si pasa, se busca otra sin avisar.
`currentSocksPort()` interna	Sin comentario en el código. Por su nombre: current socks port.
`baseInfo(port)` interna	Sin comentario en el código. Por su nombre: base info.
`withCircuit(info)` interna	Añade lo que solo se sabe en vivo: el circuito real y si Firebase ya va por Tor.
`stopTor()`	Apaga el modo anónimo y restaura la conexión normal.
`setEngine(engine)`	Cambia de motor. Si el modo estaba activo, reconecta con el nuevo.

`restoreIfEnabled(preferences)`	Restaura el modo anónimo al abrir la app, antes de que haga ninguna petición.
`getProxySelector()`	Sin comentario en el código. Por su nombre: get proxy selector.
`getSocksPort()`	Sin comentario en el código. Por su nombre: get socks port.
`getHttpPort()`	Sin comentario en el código. Por su nombre: get http port.
`isPortOpen(port, timeoutMs)` interna	Sin comentario en el código. Por su nombre: is port open.

TorSocketRouter.kt

`data/security/tor/TorSocketRouter.kt · 216 líneas`

Enruta por Tor las conexiones de la app que no consultan ningún proxy. El TorProxySelector solo cubre lo que pregunta al ProxySelector (OkHttp, HttpURLConnection). Firebase Realtime Database abre su WebSocket con un socket directo y Android ignora el ProxySelector en esos casos, de modo que **los mensajes en tiempo real seguían saliendo con la IP real** salvo que Orbot estuviera en modo VPN. Android tampoco ofrece una forma de configurar un proxy en ese SDK. Aquí se intercepta en el único punto común: la fábrica de sockets de Java. Con el modo anónimo activo, toda conexión TCP que no vaya al propio dispositivo pasa por el SOCKS de Tor, y el nombre del servidor lo resuelve el nodo de salida.

`enable(port)`	Sin comentario en el código. Por su nombre: enable.
`disable()`	Sin comentario en el código. Por su nombre: disable.
`isLocal(address)` interna	Sin comentario en el código. Por su nombre: is local.
`create(stream)`	Sin comentario en el código. Por su nombre: create.
`connect(host, port)`	Sin comentario en el código. Por su nombre: connect.
`bind(host, port)`	Sin comentario en el código. Por su nombre: bind.
`listen(backlog)`	Sin comentario en el código. Por su nombre: listen.
`accept(s)`	Sin comentario en el código. Por su nombre: accept.
`getInputStream()`	Sin comentario en el código. Por su nombre: get input stream.
`getOutputStream()`	Sin comentario en el código. Por su nombre: get output stream.
`requireFd()` interna	Sin comentario en el código. Por su nombre: require fd.
`available()`	Sin comentario en el código. Por su nombre: available.
`close()`	Sin comentario en el código. Por su nombre: close.
`shutdownInput()`	Sin comentario en el código. Por su nombre: shutdown input.
`shutdownOutput()`	Sin comentario en el código. Por su nombre: shutdown output.
`sendUrgentData(data)`	Sin comentario en el código. Por su nombre: send urgent data.
`setOption(optID, value)`	Sin comentario en el código. Por su nombre: set option.
`getOption(optID)`	Sin comentario en el código. Por su nombre: get option.
`applyOption(socket, optID, value)` interna	Sin comentario en el código. Por su nombre: apply option.

TorState.kt

`data/security/tor/TorState.kt · 93 líneas`

Represents the current state of the Tor service. States: - Disconnected – Tor is not running or has been cleanly stopped. - Connecting – Tor is bootstrapping / connecting to the network. - Connected – Tor is fully connected and ready to route traffic. - Error – A fatal or recoverable error occurred.

Parte 24

Motor de IA

Proveedores, agentes y herramientas.

AiAssistService.kt

`data/ai/AiAssistService.kt · 223 líneas`

Tono al que reescribir un borrador.

`isAvailable()`	`false` si no hay proveedor de IA configurado; la UI se oculta entonces.
`withAiTimeout(block)` interna asíncrona	Ejecuta block con un tope de tiempo DURO. Aunque el cliente HTTP ya tiene sus timeouts, la cola reintenta con backoff (5s + 15s) ante límites de cuota, así que en el peor caso una sola llamada podía tardar minutos y la interfaz se veía "colgada" resumiendo o traduciendo. Con withTimeoutOrNull la operación SIEMPRE termina: o devuelve el resultado, o falla con un mensaje claro que la UI puede mostrar y reintentar. Se u
`smartReplies(transcript)` asíncrona	Tres respuestas cortas para contestar al último mensaje. Se piden en líneas separadas y no en JSON a propósito: los modelos pequeños que la gente conecta por OpenRouter u Ollama fallan a menudo al cerrar el JSON, y una lista de líneas es trivial de recuperar aunque la respuesta venga sucia.
`summarize(transcript)` asíncrona	Resumen en viñetas de la conversación.
`rewriteTone(draft, tone)` asíncrona	Reescribe draft en el tone indicado, conservando el idioma original.
`translate(text, targetLanguageName)` asíncrona	Traducción con el modelo del usuario. Es notablemente mejor que la API gratuita de memoria de traducción que usa com.Azelmods.App.data.translation.TranslationService como respaldo, sobre todo con jerga, emoji y mensajes cortos sin contexto, que es justamente lo que se escribe en un chat.
`buildTranscript(messages, maxMessages)`	Convierte los últimos mensajes en un guion legible para el modelo. Se etiquetan como "YO" y "OTRO" en vez de con los nombres reales: al modelo no le hace falta saber con quién habla el usuario, y así no se envían nombres de personas a un proveedor externo sin necesidad.

AiKeyStore.kt

`data/ai/AiKeyStore.kt · 210 líneas`

🔐 AI KEY STORE Almacenamiento seguro de la API key de Gemini introducida por el usuario. Usa EncryptedSharedPreferences (androidx.security:security-crypto) de forma que la clave nunca se guarda en texto plano en disco. Características: - Cifrado AES256 (claves y valores) respaldado por el Android Keystore. - Recuperación robusta: si el almacén cifrado se corrompe (p. ej. tras restaurar un backup o rotar el keystore), se limpia y se vuelve a crear sin crashear. - Estado reactivo hasKey para que la UI (cabecera del chat, ajustes) refleje en tiempo real si hay una clave activa. Provisto por Hilt mediante inyección de constructor (@Singleton).

`getApiKey()`	Devuelve la API key del usuario o `null` si no hay ninguna configurada. El valor se devuelve ya recortado (sin espacios sobrantes).
`setApiKey(key)`	Guarda la API key del usuario de forma cifrada. Un valor en blanco equivale a borrarla.
`clearApiKey()`	Borra la API key del usuario del almacén seguro.
`hasApiKey()`	`true` si existe una API key del usuario almacenada y no está en blanco.
`getProvider()`	Proveedor elegido por el usuario; AiProvider.DEFAULT si nunca eligió.
`setProvider(newProvider)`	Cambia el proveedor. La URL base y el modelo se resetean a los del preset: arrastrar la URL de OpenAI a Ollama solo produce errores desconcertantes. La API key NO se borra, porque volver al proveedor anterior es lo normal.

`getBaseUrl()`	URL base efectiva: la que puso el usuario o, si no, la del preset.
`setBaseUrl(url)`	Sin comentario en el código. Por su nombre: set base url.
`getModel()`	Modelo efectivo: el que puso el usuario o, si no, el del preset.
`setModel(model)`	Sin comentario en el código. Por su nombre: set model.
`isProviderUsable()`	`true` si el proveedor activo puede usarse ya: tiene lo que necesita. Los servidores locales no requieren clave, pero sí una URL base.
`readRawKey()` interna	Sin comentario en el código. Por su nombre: read raw key.
`createSecurePrefs()` interna	Crea (o recupera) el almacén cifrado. Si la creación falla por corrupción del archivo cifrado, lo elimina y reintenta una vez para evitar crashes persistentes.
`buildEncryptedPrefs()` interna	Sin comentario en el código. Por su nombre: build encrypted prefs.

AiProvider.kt

`data/ai/AiProvider.kt · 168 líneas`

Proveedores de IA que el usuario puede elegir. Salvo Gemini, todos hablan el mismo dialecto: `POST {baseUrl}/chat/completions` con `Authorization: Bearer <key>`, el estándar de facto que implementan OpenAI, Ollama, OpenRouter, DeepSeek, Mistral, Groq, LM Studio y prácticamente cualquier servidor local. Por eso basta un único cliente para todos ellos: no hace falta una integración por proveedor. Gemini es la excepción porque usa `:generateContent` y un esquema propio (`contents`/`parts` en vez de `messages`), así que mantiene su ruta aparte. La censura no la decide la app: depende del modelo que el usuario elija. Un modelo local vía Ollama no aplica los filtros de un proveedor comercial. La a

`fromId(id)`	Sin comentario en el código. Por su nombre: from id.

GeminiContextManager.kt

`data/ai/GeminiContextManager.kt · 63 líneas`

📦 GEMINI CONTEXT MANAGER Gestiona la ventana de contexto para optimizar el uso de tokens. - Trim del historial a N mensajes (ventana deslizante) - Estimación de tokens - Verificación de seguridad antes de enviar NO modifica prompts existentes, solo limita el historial enviado.

`trimHistory(fullHistory, maxMessages)`	Ventana deslizante: solo últimos N mensajes. Mantiene los prompts intactos, solo limita el historial enviado. Siempre conserva el primer mensaje (system/context) si existe.
`estimateTokens(text)`	Estima tokens de forma simple (4 chars ≈ 1 token)
`isSafeToSend(history)`	Verifica si el historial es seguro para enviar (bajo el límite TPM)

GeminiRateLimiter.kt

`data/ai/GeminiRateLimiter.kt · 45 líneas`

🛡️ GEMINI RATE LIMITER Controla la velocidad de requests a la API de Gemini Free Tier. Free tier: 15 RPM → mínimo 4.5s entre requests para margen seguro.

`executeWithRateLimit(block)` asíncrona	Sin comentario en el código. Por su nombre: execute with rate limit.

GeminiRequestQueue.kt

`data/ai/GeminiRequestQueue.kt · 76 líneas`

🔄 GEMINI REQUEST QUEUE Cola de requests con reintentos automáticos y backoff exponencial. Detecta errores 429/quota y reintenta con espera creciente: 5s → 15s → 30s

`enqueue(maxRetries, peticiones)` asíncrona	Sin comentario en el código. Por su nombre: enqueue.
`isRateLimitError(message)` interna	Sin comentario en el código. Por su nombre: is rate limit error.

AgentCatalog.kt

`data/ai/agents/AgentCatalog.kt · 240 líneas`

Agentes y equipos que trae la app. Las instrucciones describen CÓMO trabaja cada uno; el formato de respuesta lo añade AgentOrchestrator para que sea idéntico en todos.

`agent(id)`	Sin comentario en el código. Por su nombre: agent.

AgentFileExporter.kt

`data/ai/agents/AgentFileExporter.kt · 163 líneas`

Lo que un agente quiere guardar en el teléfono, a la espera de que el usuario lo decida.

`folderLabel()`	Sin comentario en el código. Por su nombre: folder label.
`setFolder(uri)`	Recuerda la carpeta elegida, con permiso persistente para no volver a pedirla.
`answer(allowed)`	Sin comentario en el código. Por su nombre: answer.
`export(fileName, content)` asíncrona	Pide permiso y, si lo dan, guarda. Devuelve el mensaje para el agente.
`write(fileName, mime, bytes)` interna	Sin comentario en el código. Por su nombre: write.
`renderPdf(title, content)` interna	Maqueta el texto en A4, con títulos para las líneas que empiezan por .
`place(text, paint, gapAfter)`	Sin comentario en el código. Por su nombre: place.
`mimeTypeOf(name)` interna	Sin comentario en el código. Por su nombre: mime type of.

AgentModels.kt

`data/ai/agents/AgentModels.kt · 201 líneas`

Herramientas que un agente puede usar. Todas hacen algo real: el texto de usage es lo que lee el modelo, así que describe la entrada exacta que espera cada una.

`fromId(id)`	Sin comentario en el código. Por su nombre: from id.

AgentOrchestrator.kt

`data/ai/agents/AgentOrchestrator.kt · 512 líneas`

Motor de los agentes de IA. Cada agente trabaja en bucle "pensar → usar herramienta → observar" (patrón ReAct) hasta dar su respuesta final. En modo equipo, un coordinador reparte tareas entre especialistas, cada uno recibe el trabajo de los anteriores y un revisor entrega el resultado. Todo lo que ocurre se publica en un MutableStateFlow de AgentRunState: la interfaz lo pinta en vivo (qué agente trabaja, qué herramienta usa, qué encontró), que es lo que permite ver a los agentes trabajar. Usa el proveedor de IA que el usuario tenga configurado (Gemini, OpenAI, OpenRouter, DeepSeek, Ollama…): el protocolo es JSON en texto, no el formato de herramientas propio de cada uno.

`newRunState(runId, goal, agents, isTeam, title)`	Sin comentario en el código. Por su nombre: new run state.
`runSingle(agent, goal, model, state)` asíncrona	Un agente en solitario. Devuelve su respuesta.
`runTeam(team, goal, model, state)` asíncrona	Un equipo: plan del coordinador → especialistas en cadena → revisor.
`makePlan(members, goal, model, rec)` interna asíncrona	Sin comentario en el código. Por su nombre: make plan.

`buildTeamContext(goal, outputs)` interna	Sin comentario en el código. Por su nombre: build team context.
`runAgent(agent, task, context, model, rec)` interna asíncrona	Sin comentario en el código. Por su nombre: run agent.
`completeIfCut(answer, systemPrompt, model, rec)` interna asíncrona	Si el modelo se quedó sin espacio a media respuesta, un bloque de código queda sin cerrar y el programa llegaría cortado: se le pide que siga justo donde se quedó.
`useTool(agent, call, rec)` interna asíncrona	Sin comentario en el código. Por su nombre: use tool.
`toolActivity(tool, input)` interna	Sin comentario en el código. Por su nombre: tool activity.
`buildSystemPrompt(agent, forceFinal)` interna	Sin comentario en el código. Por su nombre: build system prompt.
`buildUserPrompt(task, context, scratch, forceFinal)` interna	Sin comentario en el código. Por su nombre: build user prompt.
`callModel(systemPrompt, userPrompt, model, onText)` interna asíncrona	Pide una respuesta al modelo por streaming y la devuelve entera. Se usa streaming aunque el agente no muestre el texto letra a letra: una petición normal no recibe ni un byte hasta que el modelo termina, y por eso los agentes fallaban por tiempo de espera justo en las tareas largas. El tope es alto porque en los modelos con razonamiento (Gemini 2.5) lo que "piensan" cuenta contra él, y porque un programa completo ocu
`streamReply(systemPrompt, userPrompt, model, onText)` interna asíncrona	Sin comentario en el código. Por su nombre: stream reply.
`isTransient(e)` interna	Sin comentario en el código. Por su nombre: is transient.
`event(build)`	Sin comentario en el código. Por su nombre: event.
`node(agentId, change)`	Sin comentario en el código. Por su nombre: node.
`llmCall()`	Sin comentario en el código. Por su nombre: llm call.
`toolCall()`	Sin comentario en el código. Por su nombre: tool call.
`live(text)`	Lo que va escribiendo el modelo, salvo sus llamadas en JSON, que no interesan.
`clearLive()`	Sin comentario en el código. Por su nombre: clear live.
`savedFile(name)`	Sin comentario en el código. Por su nombre: saved file.
`finish(answer)`	Sin comentario en el código. Por su nombre: finish.

AgentProtocol.kt

`data/ai/agents/AgentProtocol.kt · 254 líneas`

Lo que un agente decide en cada paso. El protocolo es JSON en texto plano y no el "tool calling" nativo de cada proveedor a propósito: así funciona IGUAL con Gemini, OpenAI, OpenRouter, DeepSeek, Groq o un modelo local de Ollama, que no comparten formato de herramientas (y varios modelos locales no lo soportan).

`parseDecision(raw)`	La respuesta final es Markdown normal y solo las llamadas a herramientas van en JSON. Meter la respuesta en una cadena JSON obligaba al modelo a escapar cada comilla y salto de línea del código que escribía, y ahí es donde el código llegaba roto. Se sigue aceptando `{"final": ...}` porque algunos modelos insisten en responder así.
`unwrapWholeFence(text)` interna	Quita la valla solo si toda la respuesta es un único bloque Markdown. Si hay más texto después, esa valla es parte del contenido: quitarla se comía la explicación del código.
`inputValue()` interna	Sin comentario en el código. Por su nombre: input value.
`parsePlan(raw, validIds, maxSteps)`	Plan del coordinador: `{"plan":{"agent":"investigador","task":"..."}}` (también acepta el array suelto). Ignora agentes que no estén en validIds.
`extractThought(raw)`	El campo "thought" de una respuesta JSON, si lo hay (p. ej. el del plan).
`extractJsonObject(text)`	Primer objeto `{...}` equilibrado del texto, respetando cadenas y escapes.
`extractJsonArray(text)` interna	Sin comentario en el código. Por su nombre: extract json array.
`extractBalanced(text, open, close)` interna	Sin comentario en el código. Por su nombre: extract balanced.

`parseObject(candidate)` interna	Sin comentario en el código. Por su nombre: parse object.
`escapeRawNewlinesInStrings(s)` interna	Sin comentario en el código. Por su nombre: escape raw newlines in strings.
`salvageFinal(text)` interna	Sin comentario en el código. Por su nombre: salvage final.
`stripCodeFence(text)`	Sin comentario en el código. Por su nombre: strip code fence.
`string(key)` interna	Sin comentario en el código. Por su nombre: string.

AgentToolbox.kt

`data/ai/agents/AgentToolbox.kt · 919 líneas`

Resultado de una herramienta: lo que verá el agente y si funcionó.

`torActive()` interna	Sin comentario en el código. Por su nombre: tor active.
`run(tool, input)` asíncrona	Sin comentario en el código. Por su nombre: run.
`calculate(input)` interna	Sin comentario en el código. Por su nombre: calculate.
`currentDateTime()`	Sin comentario en el código. Por su nombre: current date time.
`wikipediaLang()` interna	Sin comentario en el código. Por su nombre: wikipedia lang.
`wikipedia(query)` interna	Sin comentario en el código. Por su nombre: wikipedia.
`wikiSearchTitles(lang, query, limit)` interna	Sin comentario en el código. Por su nombre: wiki search titles.
`wikiSummary(lang, title)` interna	Sin comentario en el código. Por su nombre: wiki summary.
`webSearch(query)` interna asíncrona	Pregunta a varios buscadores a la vez y une lo que devuelven. Un solo buscador no basta: ante un nombre poco común, DuckDuckGo devuelve páginas que no tienen nada que ver y el agente concluía que eso "no existe". Por eso se comprueba si cada resultado contiene literalmente lo buscado y se le dice al agente la verdad: dónde buscó y qué no encontró, que no es lo mismo que demostrar que no existe.
`engine(name, search)` interna	null = el buscador falló, que no es lo mismo que no encontrar nada.
`html(url)` interna	Sin comentario en el código. Por su nombre: html.
`newsSearch(query)` interna	Sin comentario en el código. Por su nombre: news search.
`readUrl(input)` interna	Sin comentario en el código. Por su nombre: read url.
`fetchPage(url)` interna	Sin comentario en el código. Por su nombre: fetch page.
`fetchWithReader(url)` interna	Sin comentario en el código. Por su nombre: fetch with reader.
`profileLookup(input)` interna asíncrona	Cada sitio dice de una forma distinta si un perfil existe; aquí solo cuenta lo comprobado.
`unverified(code)` interna	Sin comentario en el código. Por su nombre: unverified.
`status(url, userAgent)` interna	Sin comentario en el código. Por su nombre: status.
`github(input)` interna	Sin comentario en el código. Por su nombre: github.
`githubSearch(query)` interna	Sin comentario en el código. Por su nombre: github search.
`githubRepo(repo)` interna	Sin comentario en el código. Por su nombre: github repo.
`githubFile(path)` interna	Sin comentario en el código. Por su nombre: github file.
`stackOverflow(input)` interna	Sin comentario en el código. Por su nombre: stack overflow.
`packageInfo(input)` interna	Sin comentario en el código. Por su nombre: package info.
`compilerFor(language)` interna	Sin comentario en el código. Por su nombre: compiler for.
`runCode(input)` interna	Sin comentario en el código. Por su nombre: run code.
`filesRef()` interna	Sin comentario en el código. Por su nombre: files ref.
`loadFiles()` interna asíncrona	Sin comentario en el código. Por su nombre: load files.

`saveFile(input)` interna asíncrona	Guarda en el Editor. Un agente lee páginas de internet, que podrían intentar convencerlo de pisar un archivo del usuario: por eso nunca sobrescribe lo que él no creó, sino que guarda una copia con otro nombre.
`exportFile(input)` interna asíncrona	Sin comentario en el código. Por su nombre: export file.
`uniqueName(name, taken)` interna	Sin comentario en el código. Por su nombre: unique name.
`readFile(input)` interna asíncrona	Sin comentario en el código. Por su nombre: read file.
`listFiles()` interna asíncrona	Sin comentario en el código. Por su nombre: list files.
`stripFence(text)` interna	Quita las vallas ``` que rodean a un bloque de código, si las hay.
`weather(input)` interna	Sin comentario en el código. Por su nombre: weather.
`weatherText(code)` interna	Sin comentario en el código. Por su nombre: weather text.
`currency(input)` interna	Sin comentario en el código. Por su nombre: currency.
`get(url)` interna	Sin comentario en el código. Por su nombre: get.
`enc(s)` interna	Sin comentario en el código. Por su nombre: enc.
`check(url, torActive)`	Sin comentario en el código. Por su nombre: check.
`isPrivateIp(host)` interna	Sin comentario en el código. Por su nombre: is private ip.
`toPlainText(html)`	Sin comentario en el código. Por su nombre: to plain text.
`decodeEntities(s)`	Sin comentario en el código. Por su nombre: decode entities.

JsSandbox.kt

`data/ai/agents/JsSandbox.kt · 114 líneas`

Ejecuta JavaScript de los agentes en un WebView aislado. Un agente lee páginas de internet y esas páginas pueden intentar darle órdenes, así que lo que ejecuta no puede tocar nada del usuario: sin red (CSP y bloqueo de cargas), sin archivos, sin almacenamiento y sin ningún puente hacia la app. Cada ejecución usa un WebView nuevo que se destruye al terminar, de modo que tampoco queda estado entre una y otra.

`run(code)` asíncrona	Sin comentario en el código. Por su nombre: run.
`onPageFinished(view, url)`	Sin comentario en el código. Por su nombre: on page finished.
`evaluate(web, script)` interna asíncrona	Sin comentario en el código. Por su nombre: evaluate.
`wrap(code)` interna	Sin comentario en el código. Por su nombre: wrap.

SafeCalculator.kt

`data/ai/agents/SafeCalculator.kt · 175 líneas`

Calculadora de la herramienta `calculator` de los agentes. Los modelos de lenguaje se equivocan con la aritmética; con esta herramienta el agente delega las cuentas en lugar de "estimarlas". Es un analizador descendente recursivo propio: NO evalúa código (nada de motores de script), así que una entrada maliciosa solo puede producir un error de sintaxis. Admite + - * / ^, paréntesis, coma o punto decimal, porcentajes con % ("15% de 80" se escribe 80*15%), constantes pi y e, y las funciones sqrt, abs, round, floor, ln, log, sin, cos, tan (en radianes).

`evaluate(expression)`	Sin comentario en el código. Por su nombre: evaluate.
`format(value)`	Resultado legible: sin ".0" en enteros y como mucho 10 decimales.
`atEnd()`	Sin comentario en el código. Por su nombre: at end.
`skipSpaces()`	Sin comentario en el código. Por su nombre: skip spaces.
`peek()` interna	Sin comentario en el código. Por su nombre: peek.
`eat(c)` interna	Sin comentario en el código. Por su nombre: eat.
`parseExpression()`	Sin comentario en el código. Por su nombre: parse expression.

`parseTerm()` interna	Sin comentario en el código. Por su nombre: parse term.
`parseFactor()` interna	Sin comentario en el código. Por su nombre: parse factor.
`parseUnary()` interna	Sin comentario en el código. Por su nombre: parse unary.
`parsePostfix()` interna	Sin comentario en el código. Por su nombre: parse postfix.
`parsePrimary()` interna	Sin comentario en el código. Por su nombre: parse primary.
`parseNumber()` interna	Sin comentario en el código. Por su nombre: parse number.
`parseIdentifier()` interna	Sin comentario en el código. Por su nombre: parse identifier.

WebSearch.kt

`data/ai/agents/WebSearch.kt · 189 líneas`

Un resultado tal como lo devuelve un buscador.

`duckDuckGo(html)`	Sin comentario en el código. Por su nombre: duck duck go.
`ddgTarget(href)` interna	Los enlaces de DuckDuckGo pasan por un redirector: se extrae el destino.
`bing(html)`	Sin comentario en el código. Por su nombre: bing.
`bingTarget(href)` interna	Bing a veces envuelve el destino en base64 dentro de `u=a1…`.
`yahoo(html)`	Sin comentario en el código. Por su nombre: yahoo.
`yahooTarget(href)` interna	Sin comentario en el código. Por su nombre: yahoo target.
`newsRss(xml)`	Noticias en RSS: titular, medio, fecha y enlace.
`tag(name)`	Sin comentario en el código. Por su nombre: tag.
`exactTerms(query)`	Lo que un resultado tiene que contener para ser de verdad sobre la búsqueda: las frases entre comillas y las palabras "raras" (nombres de usuario, códigos). Sin esto, buscar un nombre poco común devuelve páginas parecidas y el agente concluía que no existe.
`queryWords(query)`	Sin comentario en el código. Por su nombre: query words.
`merge(query, results)`	Sin comentario en el código. Por su nombre: merge.
`normalizeUrl(url)` interna	Sin comentario en el código. Por su nombre: normalize url.

Parte 25

Base de datos local

Caché con Room.

AppDatabase.kt

`data/local/AppDatabase.kt · 52 líneas`

`messageDao()`	Sin comentario en el código. Por su nombre: message dao.
`chatDao()`	Sin comentario en el código. Por su nombre: chat dao.
`userDao()`	Sin comentario en el código. Por su nombre: user dao.
`pendingMessageDao()`	Sin comentario en el código. Por su nombre: pending message dao.
`getInstance(context)`	Sin comentario en el código. Por su nombre: get instance.

CacheManager.kt

`data/local/CacheManager.kt · 254 líneas`

Coordinates all Room cache operations. Provides methods to: - Save/retrieve messages from local cache - Save/retrieve chats from local cache - Save/retrieve user profiles from local cache

`observeMessages(chatId)`	Flow of cached messages for a chat (Room auto-updates).
`getCachedMessages(chatId)` asíncrona	Single-shot: get all cached messages for a chat (for initial load).
`cacheMessages(messages)` asíncrona	Save a batch of messages (replaces existing for upsert).
`cacheMessage(message)` asíncrona	Cache a single message.
`clearChatMessages(chatId)` asíncrona	Remove all messages for a given chat (e.g. on chat deletion).
`observeChats()`	Flow of all cached chats.
`getCachedChats()` asíncrona	Single-shot: get all cached chats.
`cacheChats(chats)` asíncrona	Cache a list of Chat objects (converts to entities internally).
`replaceChats(chats)` asíncrona	Deja en caché exactamente estos chats: los que ya no existen no vuelven a asomar al abrir.
`cacheChat(chat)` asíncrona	Cache a single Chat.
`clearAllChats()` asíncrona	Remove all cached chats.
`deleteChat(chatId)` asíncrona	Borra del caché local una conversación y sus mensajes. Sin esto, borrar un chat lo quitaba del servidor pero no del disco: la pantalla de inicio pinta primero el caché y la conversación reaparecía en el siguiente arranque hasta que Firebase respondía.
`getCachedUser(userId)` asíncrona	Single-shot: get a cached user profile by UID.
`cacheUser(user)` asíncrona	Cache a user profile.
`cacheUsers(users)` asíncrona	Cache a batch of user profiles.
`clearAll()` asíncrona	Sin comentario en el código. Por su nombre: clear all.
`toEntity()` interna	Sin comentario en el código. Por su nombre: to entity.
`toMessage()` interna	Sin comentario en el código. Por su nombre: to message.
`toUser()` interna	Sin comentario en el código. Por su nombre: to user.
`toChat()`	Sin comentario en el código. Por su nombre: to chat.

Converters.kt

`data/local/Converters.kt · 68 líneas`

Helper functions to convert between complex types and JSON strings for Room storage (since Room doesn't support Map/List natively).

`mapToString(map)`	Sin comentario en el código. Por su nombre: map to string.
`stringToMap(json)`	Sin comentario en el código. Por su nombre: string to map.
`booleanMapToString(map)`	Sin comentario en el código. Por su nombre: boolean map to string.
`stringToBooleanMap(json)`	Sin comentario en el código. Por su nombre: string to boolean map.
`listToString(list)`	Sin comentario en el código. Por su nombre: list to string.
`stringToStringList(json)`	Sin comentario en el código. Por su nombre: string to string list.

ChatDao.kt

`data/local/dao/ChatDao.kt · 35 líneas`

`getChats()`	Observe all cached chats, ordered by last message time descending.
`getChatsOnce()` asíncrona	Single shot — get all cached chats.
`getChatById(chatId)` asíncrona	Sin comentario en el código. Por su nombre: get chat by id.
`insertChat(chat)` asíncrona	Sin comentario en el código. Por su nombre: insert chat.
`insertChats(chats)` asíncrona	Sin comentario en el código. Por su nombre: insert chats.
`deleteAllChats()` asíncrona	Sin comentario en el código. Por su nombre: delete all chats.
`deleteChatById(chatId)` asíncrona	Sin comentario en el código. Por su nombre: delete chat by id.

MessageDao.kt

`data/local/dao/MessageDao.kt · 43 líneas`

`getMessages(chatId)`	Observe all messages for a chat, ordered by timestamp ascending.
`getMessagesOnce(chatId)` asíncrona	Single shot — get all cached messages for a chat.
`getMessageById(messageId)` asíncrona	Sin comentario en el código. Por su nombre: get message by id.
`insertMessage(message)` asíncrona	Insert or replace a single message.
`insertMessages(messages)` asíncrona	Insert or replace a batch of messages.
`deleteMessagesByChat(chatId)` asíncrona	Delete all messages belonging to a specific chat.
`deleteAllMessages()` asíncrona	Delete all messages (e.g. on logout).
`getLastMessageTimestamp(chatId)` asíncrona	Sin comentario en el código. Por su nombre: get last message timestamp.

PendingMessageDao.kt

`data/local/dao/PendingMessageDao.kt · 32 líneas`

`insert(pendingMessage)` asíncrona	Sin comentario en el código. Por su nombre: insert.
`getAll()`	Sin comentario en el código. Por su nombre: get all.
`getByChatId(chatId)` asíncrona	Sin comentario en el código. Por su nombre: get by chat id.
`count()` asíncrona	Sin comentario en el código. Por su nombre: count.
`delete(pendingMessage)` asíncrona	Sin comentario en el código. Por su nombre: delete.
`deleteById(id)` asíncrona	Sin comentario en el código. Por su nombre: delete by id.

`deleteAll()` asíncrona	Sin comentario en el código. Por su nombre: delete all.

UserDao.kt

`data/local/dao/UserDao.kt · 35 líneas`

`getAllUsers()`	Observe all cached users.
`getAllUsersOnce()` asíncrona	Single shot — get all cached users.
`getUserById(userId)` asíncrona	Sin comentario en el código. Por su nombre: get user by id.
`insertUser(user)` asíncrona	Sin comentario en el código. Por su nombre: insert user.
`insertUsers(users)` asíncrona	Sin comentario en el código. Por su nombre: insert users.
`deleteAllUsers()` asíncrona	Sin comentario en el código. Por su nombre: delete all users.
`deleteUserById(userId)` asíncrona	Sin comentario en el código. Por su nombre: delete user by id.

CachedChatEntity.kt

`data/local/entity/CachedChatEntity.kt · 51 líneas`

CachedMessageEntity.kt

`data/local/entity/CachedMessageEntity.kt · 65 líneas`

CachedUserEntity.kt

`data/local/entity/CachedUserEntity.kt · 45 líneas`

PendingMessageEntity.kt

`data/local/entity/PendingMessageEntity.kt · 19 líneas`

Parte 26

Copias de seguridad

Copias cifradas.

BackupEncryptor.kt

`data/backup/BackupEncryptor.kt · 320 líneas`

Handles backup encryption and decryption using password-based encryption. Features: - AES-256-GCM authenticated encryption - PBKDF2 key derivation from password - HMAC-SHA256 integrity verification - Salt and IV generation - Compression with GZIP File Format Requirements: 10.1, 10.4

`encryptBackup(inputFile, outputFile, password)`	Encrypts a backup file with a password. Process: 1. Compresses data with GZIP 2. Derives encryption key from password using PBKDF2 3. Encrypts with AES-256-GCM 4. Calculates HMAC for integrity 5. Writes encrypted file with header
`decryptBackup(inputFile, outputFile, password)`	Decrypts a backup file with a password. Process: 1. Reads and validates file header 2. Derives decryption key from password 3. Verifies HMAC integrity 4. Decrypts with AES-256-GCM 5. Decompresses data
`verifyPassword(backupFile, password)`	Verifies if a password is correct for a backup file without full decryption. Only reads the header and verifies HMAC.
`deriveKey(password, salt)` interna	Derives an encryption key from a password using PBKDF2.
`calculateHMAC(data, key)`	Calculates HMAC-SHA256 for integrity verification.
`encrypt(data, key)`	Encrypts raw data with a key.
`decrypt(data, key)`	Decrypts raw data with a key.
`compress(data)` interna	Compresses data using GZIP.
`decompress(data)` interna	Decompresses GZIP data.

BackupManager.kt

`data/backup/BackupManager.kt · 448 líneas`

Manages encrypted backups and restoration of user data. Features: - AES-256-GCM encryption with password-based key derivation - GZIP compression before encryption - Incremental backups (only changed data) - Multiple storage locations (local, Google Drive, Firebase) - HMAC integrity verification - Automatic backup scheduling - Rollback on restore failure Backup Contents - Messages (encrypted) - Contacts - User settings - Signal Protocol keys - Media files (optional) Requirements: 10.1, 10.2, 10.3, 10.4

`createBackup(password, location, includeMedia)`	Creates a full backup of user data. Process: 1. Collect all user data (messages, contacts, settings, keys) 2. Serialize to JSON 3. Compress with GZIP 4. Encrypt with AES-256-GCM 5. Calculate HMAC for integrity 6. Upload to selected storage location
`restoreBackup(backupInfo, password)`	Restores user data from a backup. Process: 1. Download backup file 2. Verify HMAC integrity 3. Decrypt with password 4. Decompress 5. Deserialize JSON 6. Restore data to database 7. Rollback on failure
`listBackups(userId)` asíncrona	Lists all available backups for a user.
`deleteBackup(backupId)` asíncrona	Deletes a backup.
`cleanupOldBackups(userId, keepCount)` asíncrona	Deletes old backups, keeping only the most recent N backups.
`suggestedExportName()`	Nombre de fichero sugerido para exportar (legible y con extensión propia). Ej.: `NexusChat-2026-07-27_1530.azelback`
`localBackupFile(backupId)`	Fichero local de la copia recién creada (el que escribe createBackup).

`exportBackupTo(backupId, destination)` asíncrona	Exporta una copia ya creada al destino que el usuario eligió con el selector del sistema (Storage Access Framework). CAUSA RAÍZ (fix "no exporta nada"): createBackup con destino LOCAL solo escribía el fichero en `Android/data/<pkg>/files/backups`, que en Android 11+ NO es navegable desde la app Archivos. La copia existía, pero el usuario no podía verla en ningún sitio. Ahora, tras crearla, se copia al URI que el usua
`collectUserData(userId, includeMedia)` interna asíncrona	Sin comentario en el código. Por su nombre: collect user data.
`serializeUserData(userData)` interna	Sin comentario en el código. Por su nombre: serialize user data.
`deserializeUserData(json)` interna	Sin comentario en el código. Por su nombre: deserialize user data.
`restoreUserData(userId, userData)` interna asíncrona	Sin comentario en el código. Por su nombre: restore user data.
`toMap()` interna	Sin comentario en el código. Por su nombre: to map.
`deriveAesKey(password)` interna	Derives a 256-bit (32-byte) AES key from the user's backup password/PIN using SHA-256. This guarantees a valid key length regardless of how short the PIN is (fixes "Unsupported key size" crashes with short PINs).
`compressData(data)` interna	Sin comentario en el código. Por su nombre: compress data.
`decompressData(data)` interna	Sin comentario en el código. Por su nombre: decompress data.
`generateBackupId()` interna	Sin comentario en el código. Por su nombre: generate backup id.

BackupResult.kt

`data/backup/BackupResult.kt · 13 líneas`

BackupStorage.kt

`data/backup/BackupStorage.kt · 399 líneas`

BackupStorage – abstracts all I/O between backup files and their storage backends: Firebase Storage (remote) and the app's external/internal files directory (local). All Firebase operations run in the calling coroutine's dispatcher. Use kotlinx.coroutines.Dispatchers.IO at the call site for heavy I/O work.

`uploadToFirebase(backupFile, onProgress)` asíncrona	Uploads backupFile to Firebase Storage under `backups/{uid}/{timestamp}_{filename}`.
`downloadFromFirebase(url, destinationFile, onProgress)` asíncrona	Downloads the backup at url to destinationFile.
`downloadHeaderBytes(url, maxBytes)` asíncrona	Downloads only the first maxBytes bytes of the backup at url into a new temporary file in getLocalBackupsDir. Useful for header-only reads (e.g. password validation) without pulling the full file. NOTE: Because the Firebase Storage Android SDK does not support HTTP Range requests natively, this implementation downloads the full file via a streaming task and stops reading after maxBytes. The network transfer will be c
`saveToLocal(backupFile, destinationUri)` asíncrona	Copies backupFile to a user-chosen location identified by destinationUri, using android.content.ContentResolver to honour Android's scoped storage.
`getLocalBackupsDir()`	Returns (or creates) the directory used for locally stored backups. Prefers `getExternalFilesDir("backups")` (visible in Files app) and falls back to `filesDir/backups` if external storage is unavailable.
`listLocalBackups()`	Lists all `*.azelback` files in getLocalBackupsDir, sorted newest-first.
`listFirebaseBackups()` asíncrona	Lists all backup files stored in Firebase Storage under `backups/{uid}/`. Fetches metadata (size, creation time) and download URL for each item. Items that fail to resolve metadata are silently skipped.
`deleteFirebaseBackup(url)` asíncrona	Deletes the Firebase Storage file identified by url.

`autoDeleteOldBackups(maxBackupsToKeep)` asíncrona	Deletes the oldest Firebase and local backups when the total count for either location exceeds maxBackupsToKeep. Backups are ranked by BackupInfo.createdAt (newest first); the oldest entries beyond maxBackupsToKeep are removed.
`listBackups(userId)` asíncrona	Lists all backups for a user (combines local and Firebase backups)
`deleteBackup(backupId)` asíncrona	Deletes a backup by ID (searches local first, then Firebase)

Parte 27

Datos, varios

Modelos, preferencias y servicios de datos.

AzelAIApiService.kt

`data/api/AzelAIApiService.kt · 814 líneas`

🚀 AZEL AI API SERVICE - GEMINI INTEGRATION Servicio para la API de Gemini con streaming real, cola con backoff y rate limiting. La API key se resuelve en tiempo de ejecución: primero la del usuario (AiKeyStore), con fallback a BuildConfig.GEMINI_API_KEY (leída desde local.properties).

`resolveApiKey()` interna	🔑 Resuelve la API key activa. 1. Clave introducida por el usuario y guardada de forma segura (AiKeyStore). 2. Fallback a BuildConfig.GEMINI_API_KEY (desde local.properties). Devuelve cadena vacía si no hay ninguna configurada.
`hasApiKey()`	`true` si hay una API key disponible (de usuario o de BuildConfig).
`isConfigured()`	`true` si el proveedor activo tiene lo necesario para responder. Lo usa la pantalla de IA para decir la verdad en vez de un "API Key activa" fijo.
`activeProviderSummary(selectedGeminiModel)`	Nombre del proveedor y modelo activos, p. ej. ("OpenRouter", "deepseek/deepseek-chat").
`requireApiKey()` interna	Devuelve la API key activa o lanza una excepción reconocible si falta.
`getSystemPrompt()` interna	🧭 SYSTEM PROMPT NEUTRAL Asistente general capaz y útil. Responde con normalidad a usos legítimos, en el idioma del usuario (español o inglés). Sin técnicas de evasión.
`googleSearch(query)` asíncrona	Búsqueda en Google de verdad, con la función de búsqueda de Gemini. Google no deja consultar sus resultados desde una app sin un servicio de pago, pero la API de Gemini sí permite buscar con la clave del usuario. Devuelve null si el proveedor activo no es Gemini o si falla: quien llama sigue con los demás buscadores.
`chatCompletionStream(model, messages, temperature, maxTokens, topP, frequencyPenalty)`	🚀 CHAT COMPLETION CON STREAMING REAL (GEMINI FORMAT)
`streamGenerateContentOnce(model, messages, temperature, maxTokens, topP, systemPrompt)` interna asíncrona	🔂 UN INTENTO DE STREAMING (llamado por requestQueue.enqueue) Suspende hasta que el stream termina (Done) o falla. Si Gemini responde con un RateLimit (429 / quota / RESOURCE_EXHAUSTED) antes de emitir contenido, lanza una excepción reconocible por GeminiRequestQueue para que ésta reintente con backoff. Si ya se recibió contenido, no se reintenta y el error se propaga al flujo.
`onOpen(eventSource, response)`	Sin comentario en el código. Por su nombre: on open.
`onEvent(eventSource, id, type, data)`	Sin comentario en el código. Por su nombre: on event.
`onFailure(eventSource, t, response)`	Sin comentario en el código. Por su nombre: on failure.
`onClosed(eventSource)`	Sin comentario en el código. Por su nombre: on closed.
`chatCompletion(model, messages, temperature, maxTokens, topP, frequencyPenalty)` asíncrona	💬 CHAT COMPLETION SIN STREAMING (FALLBACK)
`chatCompletionInternal(model, messages, temperature, maxTokens, topP, systemPrompt)` interna asíncrona	💬 IMPLEMENTACIÓN INTERNA DE CHAT COMPLETION (llamada por requestQueue)
`openAiCompatCompletion(provider, messages, temperature, maxTokens, topP, systemPrompt)` interna	Completion contra un proveedor OpenAI-compatible elegido por el usuario. Reutiliza el mismo client que Gemini, así que hereda el resolutor DNS por Tor y los timeouts ya configurados. El modelo y la URL base salen del almacén cifrado, no de constantes: son del usuario.
`openAiCompatStream(provider, messages, temperature, maxTokens, topP, systemPrompt)` interna	Versión en streaming de openAiCompatCompletion.

`extractText(parts)` interna	Une el texto visible de las partes de Gemini, sin el razonamiento interno.
`buildRequestBody(messages, temperature, maxTokens, topP, systemPrompt)` interna	🔧 CONSTRUIR REQUEST BODY PARA GEMINI
`getAvailableModels()`	📊 OBTENER MODELOS DISPONIBLES EN GEMINI
`listProviderModels()` asíncrona	📥 CATÁLOGO REAL DEL PROVEEDOR ACTIVO Consulta `GET {baseUrl}/models` del proveedor OpenAI-compatible configurado y devuelve sus ids. Gracias a esto la app NO necesita una lista de modelos hardcodeada: cuando un proveedor publica una versión nueva, aparece sola. Gemini no expone este endpoint con el mismo contrato, así que ahí se devuelven los modelos conocidos de getAvailableModels.
`checkHealth()` asíncrona	🔍 VERIFICAR SALUD DE LA API GEMINI

OpenAiCompatClient.kt

`data/api/OpenAiCompatClient.kt · 300 líneas`

Cliente para cualquier endpoint que hable el dialecto de OpenAI (`POST {baseUrl}/chat/completions`). Un solo cliente sirve para OpenAI, OpenRouter, DeepSeek, Mistral, Groq, Ollama, LM Studio, vLLM y llama.cpp, porque todos implementan el mismo contrato. Lo único que cambia entre ellos es la URL base, el nombre del modelo y si hace falta clave.

`chatCompletion(client, baseUrl, apiKey, model, systemPrompt, messages)`	rol `system`, que es como lo espera este dialecto (Gemini usa otro campo). debe ser `user` o `assistant`. la cabecera Authorization, porque algunos servidores rechazan un Bearer vacío.
`chatCompletionStream(client, baseUrl, apiKey, model, systemPrompt, messages)`	Igual que chatCompletion pero con `stream: true`: entrega la respuesta a trozos en onDelta según la genera el modelo (Server-Sent Events, el mismo formato en OpenAI, OpenRouter, DeepSeek, Groq, Mistral, Ollama…). Antes los proveedores OpenAI-compatibles se pedían sin streaming y la respuesta aparecía de golpe tras esperar a que terminara entera: con respuestas largas o modelos locales eran decenas de segundos con la
`listModels(client, baseUrl, apiKey)`	Pide al proveedor su catálogo real de modelos (`GET {baseUrl}/models`). Es lo que permite que un modelo recién publicado (una versión nueva de DeepSeek, GLM, Qwen o Kimi) aparezca en la app SIN actualizar la app: la lista no está hardcodeada, la contesta el propio proveedor. Devuelve los ids ordenados. Lanza excepción con el motivo si falla, para que la UI pueda explicar por qué (clave inválida, endpoint mal escrito,
`extractErrorMessage(raw)` interna	Extrae el texto de error del cuerpo. Los proveedores no coinciden en el formato: unos usan `{"error":{"message":…}}`, otros `{"error":"…"}` y algunos texto plano, así que se prueban en orden y se cae al cuerpo tal cual.
`stringOrEmpty(key)` interna	`optString` de Android devuelve el texto "null" cuando el valor es JSON null, y varios proveedores mandan `"content": null` en el primer trozo del stream o en modelos de razonamiento. Sin esto, "null" acababa pegado al principio de la respuesta.

ChannelRepository.kt

`data/channels/ChannelRepository.kt · 198 líneas`

Canales: una persona publica y cualquiera se suscribe. Son públicos a propósito, así que a diferencia de los chats no van cifrados de extremo a extremo.

`observeSubscribed()`	Sin comentario en el código. Por su nombre: observe subscribed.
`observeChannel(id)`	Sin comentario en el código. Por su nombre: observe channel.
`observePosts(id)`	Sin comentario en el código. Por su nombre: observe posts.
`create(name, handle, description)` asíncrona	Devuelve el id del canal, o null si ese @nombre ya está cogido.
`findByHandle(handle)` asíncrona	Sin comentario en el código. Por su nombre: find by handle.
`isSubscribed(id)` asíncrona	Sin comentario en el código. Por su nombre: is subscribed.

`subscribe(id)` asíncrona	Sin comentario en el código. Por su nombre: subscribe.
`unsubscribe(id)` asíncrona	Sin comentario en el código. Por su nombre: unsubscribe.
`publish(id, text, image)` asíncrona	Sin comentario en el código. Por su nombre: publish.
`deletePost(id, postId)` asíncrona	Sin comentario en el código. Por su nombre: delete post.
`parseChannel(snapshot)` interna	Sin comentario en el código. Por su nombre: parse channel.
`topic(id)`	Sin comentario en el código. Por su nombre: topic.
`normalizeHandle(handle)`	"@Mi Canal!" → "micanal": solo minúsculas, números y guion bajo.

ChatCommands.kt

`data/chat/ChatCommands.kt · 153 líneas`

Comandos que se escriben en el chat: el resultado se envía como un mensaje normal, así que lo ven todos los de la conversación. Se ejecutan en el teléfono de quien los escribe.

`suggestions(typed)`	Sugerencias mientras se escribe el nombre del comando.
`run(input, random)`	Texto que se enviará, o null si input no es un comando y debe enviarse tal cual.
`rockPaperScissors(choice, random)` interna	Sin comentario en el código. Por su nombre: rock paper scissors.
`formatNumber(value)` interna	Sin comentario en el código. Por su nombre: format number.
`evaluate(expression)`	Sin comentario en el código. Por su nombre: evaluate.
`parseSum()` interna	Sin comentario en el código. Por su nombre: parse sum.
`parseProduct()` interna	Sin comentario en el código. Por su nombre: parse product.
`parsePower()` interna	Sin comentario en el código. Por su nombre: parse power.
`parseUnary()` interna	Sin comentario en el código. Por su nombre: parse unary.
`parseNumber()` interna	Sin comentario en el código. Por su nombre: parse number.

ChatId.kt

`data/chat/ChatId.kt · 24 líneas`

Utilidad para derivar el identificador determinista de un chat privado. El ChatId se forma ordenando alfabéticamente los UIDs de los participantes y uniéndolos con `_`. Esto garantiza que ambos participantes y todas las pantallas (inicio, NewConversationScreen, Demo Chat) resuelvan el mismo nodo de chat.

`create(uidA, uidB)`	Crea un ChatId determinista e independiente del orden de los UIDs. Propiedad: `create(a, b) == create(b, a)` (conmutativo) y siempre produce el mismo resultado para las mismas entradas (determinista).

ChatOrganizer.kt

`data/chat/ChatOrganizer.kt · 89 líneas`

Carpetas y chats ocultos. Se guardan solo en este teléfono: el servidor no se entera de cómo organiza nadie sus conversaciones ni de cuáles esconde.

`init(context)`	Sin comentario en el código. Por su nombre: init.
`isHidden(chatId)`	Sin comentario en el código. Por su nombre: is hidden.
`setHidden(chatId, hidden)`	Sin comentario en el código. Por su nombre: set hidden.
`createFolder(name, firstChat)`	Sin comentario en el código. Por su nombre: create folder.
`deleteFolder(id)`	Sin comentario en el código. Por su nombre: delete folder.
`toggleInFolder(id, chatId)`	Sin comentario en el código. Por su nombre: toggle in folder.

`save(list)` interna	Sin comentario en el código. Por su nombre: save.
`encodeFolders(list)`	Sin comentario en el código. Por su nombre: encode folders.
`parseFolders(raw)`	Sin comentario en el código. Por su nombre: parse folders.

DemoAccountManager.kt

`data/demo/DemoAccountManager.kt · 255 líneas`

Demo Account Manager Creates and manages the "Azel Assistant" demo account for testing

`initializeDemoAccount(currentUserId)` asíncrona	Initialize demo account for the current user Creates demo user, chat, and messages if not already created
`announceUpdate(currentUserId)` interna asíncrona	Azel Assistant cuenta las novedades una sola vez a quien ya tenía la app.
`createDemoUser()` interna asíncrona	Create demo user in Firebase
`createDemoChat(currentUserId)` interna asíncrona	Create demo chat between current user and demo user
`createDemoMessages(currentUserId)` interna asíncrona	Create demo messages in the chat
`resetDemoAccount()`	Reset demo account (for testing)

DemoAssistant.kt

`data/demo/DemoAssistant.kt · 322 líneas`

🤖 Azel Assistant — guion del chat de bienvenida. El chat demo NO usa un modelo de IA: es un asistente guionizado y determinista cuyo objetivo es dar la bienvenida y explicar **qué es NexusChat, por qué existe y qué hace**, mensaje a mensaje (no un único bloque), para que el usuario pruebe la experiencia de conversación real mientras aprende. Por qué se reescribió el emparejado de palabras La versión anterior decidía el tema con `texto.contains(clave)` sobre claves de dos letras. La entrada `"ia"` casaba dentro de **"histor*ia*s"**, **"grac*ia*s"**, **"famil*ia*"** o **"gu*ía*"**, y como esa rama iba antes que la de historias, preguntar por las historias respondía sobre proveedores de IA. Lo

`loadReleaseNotes(context)`	Lee la primera sección `vX.Y.Z` de `CHANGELOG.md` (la más reciente).
`latestNotes()` interna	Sin comentario en el código. Por su nombre: latest notes.
`repliesFor(userText, turnIndex)`	Respuestas contextuales según el texto del usuario. Devuelve varias líneas cortas a propósito: el ViewModel las envía por separado para simular una conversación real. (0 = primero). Sirve para avanzar el tour sin repetir.
`word(keys)`	Palabra exacta: "ia" casa con "ia", nunca dentro de "historias".
`starts(keys)`	Prefijo de palabra: "cifr" casa con "cifrado" y "cifrar".
`tourStep(turnIndex)` interna	Recorrido guiado para cuando el mensaje no casa con ningún tema. Es finito: al llegar al final se queda en el cierre en lugar de volver al principio, para que el bot no dé la sensación de reiniciarse.
`tokenize(text)` interna	Divide el texto en palabras comparables: minúsculas, sin acentos y sin signos. Sin quitar acentos, "traducción" y "traduccion" serían temas distintos según cómo escriba cada usuario.

FirebaseManager.kt

`data/firebase/FirebaseManager.kt · 20 líneas`

`getCurrentUserId()`	Sin comentario en el código. Por su nombre: get current user id.
`isUserAuthenticated()`	Sin comentario en el código. Por su nombre: is user authenticated.

FirebasePersistenceGuard.kt

`data/firebase/FirebasePersistenceGuard.kt · 97 líneas`

Arranque seguro ("safe boot") para la persistencia offline de Firebase RTDB. Problema que resuelve La persistencia offline de Firebase se guarda en una base SQLite interna (`SqlPersistenceStorageEngine`). Si una escritura pendiente queda "envenenada" o el archivo se corrompe, Firebase la vuelve a reproducir en CADA arranque en su propio hilo de trabajo y lanza una `SQLiteException` que NO se puede capturar con try/catch desde la app (corre en un hilo interno). Resultado: la app crashea al entrar (y al hacer el primer write de login/registro) en un bucle. Cómo funciona el guard - Antes de activar la persistencia se marca un flag KEY_PENDING = true. - Cuando la app arranca de forma sana, markH

`enable(context, database)`	Activa la persistencia offline de forma segura. Debe llamarse UNA sola vez, antes de cualquier otro uso de FirebaseDatabase (por eso se invoca desde el provider de Hilt, que es el primer punto que construye la instancia).
`markHealthy(context)`	Marca el arranque como sano. Se llama desde la UI una vez que la app superó la ventana de arranque sin crashear, de modo que el próximo inicio NO entre en modo seguro.

GifSearchService.kt

`data/gif/GifSearchService.kt · 97 líneas`

Un GIF de la búsqueda: miniatura ligera para la cuadrícula y archivo para enviar.

`apiKey()`	Sin comentario en el código. Por su nombre: api key.
`hasApiKey()`	Sin comentario en el código. Por su nombre: has api key.
`saveApiKey(key)`	Sin comentario en el código. Por su nombre: save api key.
`trending()` asíncrona	Sin comentario en el código. Por su nombre: trending.
`search(query)` asíncrona	Sin comentario en el código. Por su nombre: search.
`request(endpoint, params)` interna asíncrona	Sin comentario en el código. Por su nombre: request.
`parse(body)` interna	Sin comentario en el código. Por su nombre: parse.

AppBackgroundManager.kt

`data/manager/AppBackgroundManager.kt · 143 líneas`

Manages global app background state Features: - Persisted in DataStore - StateFlow for reactive updates - Supports all background types

`saveBackground(config)` asíncrona	Save background configuration
`clearBackground()` asíncrona	Clear background (set to NONE)
`setSolidColor(colorHex, overlayAlpha)` asíncrona	Set solid color background
`setImageBackground(imageUri, overlayAlpha)` asíncrona	Set image background
`setVideoBackground(videoUri, overlayAlpha)` asíncrona	Set video background
`setGradientBackground(colors, angle, overlayAlpha)` asíncrona	Set gradient background

AIMessage.kt

`data/model/AIMessage.kt · 16 líneas`

BackgroundConfig.kt

`data/model/BackgroundConfig.kt · 105 líneas`

Background configuration for app-wide or per-chat backgrounds

CallData.kt

`data/model/CallData.kt · 41 líneas`

Chat.kt

`data/model/Chat.kt · 49 líneas`

Color de perfil elegido por el contacto, o -1 si no eligió ninguno. */ val contactColor: Int = -1, val isOnline: Boolean = false, val isPinned: Boolean = false, val isMuted: Boolean = false, val isArchived: Boolean = false, val isE2EE: Boolean = true, val lastMessageTimestamp: Long = lastMessageTime) : Comparable<Chat> { override fun compareTo(other: Chat): Int { return lastMessageTimestamp.compareTo(other.lastMessageTimestamp) } /** Gets the unread count for a specific user

`compareTo(other)`	Sin comentario en el código. Por su nombre: compare to.
`getUnreadCount(userId)`	Gets the unread count for a specific user
`getTotalUnreadCount()`	Gets the total unread count across all participants

ChatSettings.kt

`data/model/ChatSettings.kt · 31 líneas`

Chat settings model for pin, mute, and archive functionality.

`isCurrentlyMuted()`	Checks if chat is currently muted

GroupSettings.kt

`data/model/GroupSettings.kt · 20 líneas`

Group settings model for welcome messages and other configurations.

`formatWelcomeMessage(userName)`	Formats the welcome message with the user's name

Message.kt

`data/model/Message.kt · 62 líneas`

Story.kt

`data/model/Story.kt · 23 líneas`

User.kt

`data/model/User.kt · 26 líneas`

AiFeaturePreferences.kt

`data/preferences/AiFeaturePreferences.kt · 76 líneas`

Interruptores de las funciones de IA sobre los chats. Todas nacen **apagadas**. Cada una envía texto de la conversación al proveedor de IA que el usuario haya configurado, y eso no puede pasar por defecto en una app que presume de cifrado de extremo a extremo: el usuario tiene que encenderlas a sabiendas. La pantalla de ajustes lo dice con esas palabras.

`setSmartReplies(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set smart replies.
`setAutoTranslate(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set auto translate.
`setConversationSummary(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set conversation summary.
`setToneSuggestions(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set tone suggestions.
`setPhotoEnhance(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set photo enhance.
`setVoiceTranscription(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set voice transcription.
`isSmartRepliesEnabled()` asíncrona	Lectura puntual, para los sitios donde no interesa suscribirse al flujo.
`isAutoTranslateEnabled()` asíncrona	Sin comentario en el código. Por su nombre: is auto translate enabled.
`isPhotoEnhanceEnabled()` asíncrona	Sin comentario en el código. Por su nombre: is photo enhance enabled.
`set(key, value)` interna asíncrona	Sin comentario en el código. Por su nombre: set.

AppLockPreferences.kt

`data/preferences/AppLockPreferences.kt · 103 líneas`

`setLockEnabled(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set lock enabled.
`setAutoLockMinutes(minutes)` asíncrona	Sin comentario en el código. Por su nombre: set auto lock minutes.
`setBiometricEnabled(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set biometric enabled.
`setPin(pin)` asíncrona	Sin comentario en el código. Por su nombre: set pin.
`clearPin()` asíncrona	Sin comentario en el código. Por su nombre: clear pin.
`verifyPin(pin)` asíncrona	Sin comentario en el código. Por su nombre: verify pin.
`hasPin()` asíncrona	Sin comentario en el código. Por su nombre: has pin.
`getTotpSecret()` asíncrona	Secreto TOTP en Base32, o null si aún no se ha configurado el 2FA.
`is2faEnabledNow()` asíncrona	`true` si el segundo factor está activo (y por tanto hay secreto guardado).
`enable2fa(secretBase32)` asíncrona	Guarda el secreto y activa el 2FA en una sola escritura.
`disable2fa()` asíncrona	Desactiva el 2FA y borra el secreto para no dejar rastro reutilizable.
`hashPin(pin)` interna	Sin comentario en el código. Por su nombre: hash pin.

BotPreferences.kt

`data/preferences/BotPreferences.kt · 154 líneas`

Bot preferences for internal bot features Features: - Auto respuesta privado/grupos - Modo Fantasma - Mensajes en masa - Traductor entrada - Quoted personalizado - Mencionar todos

`setAutoReplyPrivate(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set auto reply private.
`setAutoReplyPrivateMessage(message)` asíncrona	Sin comentario en el código. Por su nombre: set auto reply private message.
`setAutoReplyGroups(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set auto reply groups.
`setAutoReplyGroupsMessage(message)` asíncrona	Sin comentario en el código. Por su nombre: set auto reply groups message.
`setGhostMode(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set ghost mode.
`setMassMessages(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set mass messages.
`setTranslator(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set translator.

`setTranslatorLanguage(language)` asíncrona	Sin comentario en el código. Por su nombre: set translator language.
`setCustomQuoted(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set custom quoted.
`setMentionAll(enabled)` asíncrona	Sin comentario en el código. Por su nombre: set mention all.

ThemePreferences.kt

`data/preferences/ThemePreferences.kt · 227 líneas`

Manages theme and color customization preferences. Features: - Custom primary/secondary colors - Predefined theme presets - Chat background customization - Message bubble styles

`getPrimaryColor()`	Gets the primary theme color
`setPrimaryColor(color)`	Sets the primary theme color
`getSecondaryColor()`	Gets the secondary theme color
`setSecondaryColor(color)`	Sets the secondary theme color
`getThemePreset()`	Gets the current theme preset
`setThemePreset(preset)`	Sets a theme preset
`getBackgroundColor()`	Gets background color from current theme preset
`getSurfaceColor()`	Gets surface color from current theme preset
`getChatBackground()`	Gets chat background type
`setChatBackground(background)`	Sets chat background type
`getMessageStyle()`	Gets message bubble style
`setMessageStyle(style)`	Sets message bubble style
`areAvatarRingsEnabled()`	Aro de color alrededor de los avatares de la lista de chats.
`setAvatarRingsEnabled(enabled)`	Sin comentario en el código. Por su nombre: set avatar rings enabled.
`getTabOrder()`	Sin comentario en el código. Por su nombre: get tab order.
`setTabOrder(order)`	Sets the custom tab order
`getVideoWallpaperUri()`	Gets the video wallpaper URI
`setVideoWallpaperUri(uri)`	Sets the video wallpaper URI

TutorialPreferences.kt

`data/preferences/TutorialPreferences.kt · 58 líneas`

Tutorial Preferences Tracks which tutorials the user has seen

`hasSeenTutorial(feature)`	Check if user has seen a specific tutorial
`markTutorialAsSeen(feature)`	Mark tutorial as seen
`resetAllTutorials()`	Reset all tutorials (for testing or user request)
`isFirstTimeUser()`	Check if this is first time opening the app
`markAppAsOpened()`	Mark app as opened

UserPreferences.kt

`data/preferences/UserPreferences.kt · 308 líneas`

User Preferences Manager Manages all app settings and preferences with real-time updates

`setSplashAnimation(enabled)`	Sin comentario en el código. Por su nombre: set splash animation.
`updateDisplayName(name)`	Sin comentario en el código. Por su nombre: update display name.

`updateUsername(username)`	Sin comentario en el código. Por su nombre: update username.
`updateBio(bio)`	Sin comentario en el código. Por su nombre: update bio.
`updatePhoneNumber(phone)`	Sin comentario en el código. Por su nombre: update phone number.
`updateEmail(email)`	Sin comentario en el código. Por su nombre: update email.
`setLastSeenEnabled(enabled)`	Sin comentario en el código. Por su nombre: set last seen enabled.
`setProfilePhotoVisible(visible)`	Sin comentario en el código. Por su nombre: set profile photo visible.
`setReadReceiptsEnabled(enabled)`	Sin comentario en el código. Por su nombre: set read receipts enabled.
`setTwoFactorEnabled(enabled)`	Sin comentario en el código. Por su nombre: set two factor enabled.
`setLowDataMode(enabled)`	Sin comentario en el código. Por su nombre: set low data mode.
`setNotificationsEnabled(enabled)`	Sin comentario en el código. Por su nombre: set notifications enabled.
`setSoundEnabled(enabled)`	Sin comentario en el código. Por su nombre: set sound enabled.
`setVibrationEnabled(enabled)`	Sin comentario en el código. Por su nombre: set vibration enabled.
`setMessagePreview(enabled)`	Sin comentario en el código. Por su nombre: set message preview.
`setGroupNotifications(enabled)`	Sin comentario en el código. Por su nombre: set group notifications.
`setNotificationSound(uri, name)`	Sin comentario en el código. Por su nombre: set notification sound.
`setDarkModeEnabled(enabled)`	Sin comentario en el código. Por su nombre: set dark mode enabled.
`setAccentColor(color)`	Sin comentario en el código. Por su nombre: set accent color.
`setFontSize(size)`	Sin comentario en el código. Por su nombre: set font size.
`setWallpaper(type, value)`	Sin comentario en el código. Por su nombre: set wallpaper.
`setTranslationLanguage(language)`	Sin comentario en el código. Por su nombre: set translation language.
`setAutoDownloadPhotos(enabled)`	Sin comentario en el código. Por su nombre: set auto download photos.
`setAutoDownloadVideos(enabled)`	Sin comentario en el código. Por su nombre: set auto download videos.
`setAutoDownloadFiles(enabled)`	Sin comentario en el código. Por su nombre: set auto download files.
`clearAllData()`	Sin comentario en el código. Por su nombre: clear all data.

MyProfile.kt

`data/profile/MyProfile.kt · 91 líneas`

El perfil del propio usuario, en vivo. El perfil real está en `users/{uid}`. La cuenta de FirebaseAuth casi nunca tiene nombre ni foto, así que quien la lea para pintar un avatar acaba mostrando un "?" en vez de la foto.

`start()`	Se puede llamar varias veces: solo cambia algo si la sesión es de otro usuario.
`text(key)`	Sin comentario en el código. Por su nombre: text.
`stop()` interna	Sin comentario en el código. Por su nombre: stop.
`rememberMyProfile()` pantalla	Dibuja en pantalla: remember my profile.

SessionManager.kt

`data/session/SessionManager.kt · 177 líneas`

SESSION MANAGER — ENTERPRISE GRADE 2026

Gestiona la sesión del usuario con: • Persistencia de sesión entre reinicios • Refresh automático de tokens • Prevención de memory leaks en auth listeners • Validación de expiración de sesión @since 2026 @version 3.0.0 Enterprise Edition @author AzelMods677

`refreshSession()` asíncrona	Refresca el token de sesión Debe llamarse periódicamente para mantener la sesión activa
`saveSession(uid, token)` asíncrona	Guarda la sesión después de login
`clearSession()` asíncrona	Limpia la sesión al hacer logout
`isSessionExpired()` asíncrona	Verifica si la sesión ha expirado
`getCurrentToken()` asíncrona	Obtiene el token actual

StickerRepository.kt

`data/stickers/StickerRepository.kt · 135 líneas`

Paquetes de stickers de cada usuario. Siguen a la cuenta, así que aparecen en otro teléfono. Un paquete se puede publicar con un código para que otras personas lo añadan.

`observePacks()`	Sin comentario en el código. Por su nombre: observe packs.
`parsePack(snapshot)` interna	Sin comentario en el código. Por su nombre: parse pack.
`createPack(name)` asíncrona	Sin comentario en el código. Por su nombre: create pack.
`defaultPack()` asíncrona	El paquete donde van los stickers recibidos y los creados sin elegir otro.
`addImage(packId, image)` asíncrona	Sube la imagen del sticker y la añade al paquete.
`addExisting(packId, sticker)` asíncrona	Guarda en un paquete un sticker que ya existe: por ejemplo, uno recibido en un chat.
`saveReceived(sticker)` asíncrona	Sin comentario en el código. Por su nombre: save received.
`removeSticker(packId, sticker)` asíncrona	Sin comentario en el código. Por su nombre: remove sticker.
`deletePack(packId)` asíncrona	Sin comentario en el código. Por su nombre: delete pack.
`publish(pack)` asíncrona	Publica una copia del paquete y devuelve el código para compartirlo.
`install(code)` asíncrona	Añade a mis paquetes el de otra persona. Devuelve su nombre, o null si el código no existe.

LinuxEnvironment.kt

`data/terminal/LinuxEnvironment.kt · 246 líneas`

Este teléfono no puede ejecutarlo (solo hay binarios de 64 bits).

`initialState()` interna	Sin comentario en el código. Por su nombre: initial state.
`isSupported()` interna	Sin comentario en el código. Por su nombre: is supported.
`install()`	Sin comentario en el código. Por su nombre: install.
`uninstall()`	Sin comentario en el código. Por su nombre: uninstall.
`download()` interna	Sin comentario en el código. Por su nombre: download.
`extract(archive)` interna	Sin comentario en el código. Por su nombre: extract.
`chmod(file, mode)` interna	Sin comentario en el código. Por su nombre: chmod.
`configure()` interna	Sin comentario en el código. Por su nombre: configure.
`shellCommand()`	Programa, argumentos y entorno para abrir una shell de Kali en una sesión de terminal.
`stripRootFolder(name)`	El archivo trae todo dentro de una carpeta como "kali-arm64/": se quita ese primer nivel.

TranslationQuotaTracker.kt

`data/translation/TranslationQuotaTracker.kt · 75 líneas`

📊 Contador LOCAL de cuota de traducción. La API gratuita de MyMemory permite ~1000 palabras/día por IP, pero no expone ningún endpoint para consultar cuánto queda: el usuario solo se enteraba del límite cuando la request ya había fallado. Este tracker estima el consumo contando las palabras enviadas desde este dispositivo y lo persiste en SharedPreferences, con reset automático al cambiar el día. Es una ESTIMACIÓN local (otra app/dispositivo detrás de la misma IP también consume cuota), por eso se usa para avisar preventivamente y no para bloquear.

`recordUsage(text)`	Registra las palabras de un texto enviado a traducir y devuelve las palabras estimadas restantes tras el consumo.
`wordsUsedToday()`	Palabras consumidas hoy según el registro local (0 si cambió el día).
`remainingWordsToday()`	Palabras estimadas restantes hoy.
`countWords(text)` interna	Sin comentario en el código. Por su nombre: count words.
`today()` interna	Sin comentario en el código. Por su nombre: today.

TranslationService.kt

`data/translation/TranslationService.kt · 367 líneas`

Resultado de una traducción exitosa. @property text El texto traducido. @property wasTruncated true si el original superaba TranslationService.MAX_CHARS y solo se tradujo el prefijo. Antes esto pasaba en silencio y el usuario no sabía que le faltaba parte del mensaje. @property remainingWords Palabras estimadas restantes de la cuota diaria local.

`translate(text, targetLang, sourceLang)` asíncrona	Translates text into targetLang. Use "auto" for sourceLang to let the service infer the source language. MULTI-LANGUAGE FIX: Mejora la detección y soporte para todos los idiomas soportados.
`httpGet(urlString)` interna	GET con timeouts EXPLÍCITOS. CAUSA RAÍZ (fix): antes se usaba `URL(url).readText()`, que abre una `URLConnection` con timeout por defecto = 0, es decir INFINITO. Si MyMemory respondía lento o el DNS por Tor no resolvía, la corrutina se quedaba bloqueada para siempre y el indicador de "traduciendo…" no se apagaba nunca. Ese era el "las traducciones cargan infinitamente".
`normalizeLanguageCode(code)` interna	Normaliza códigos de idioma a formato de 2 letras
`detectLanguage(text)` asíncrona	Best-effort language detection. Returns a 2-letter code, defaulting to "en". IMPROVED: Better detection and normalization
`detectLanguageHeuristic(text)` interna	Detección heurística por puntuación. La versión anterior devolvía el PRIMER idioma cuya lista de palabras comunes casara, y el español estaba el primero. Como "de", "que", "la" y "un" son palabras corrientes también en francés, portugués e italiano, cualquier texto en esas lenguas se detectaba como español y se acababa pidiendo a la API un `langpair=es|es`, que devuelve el original o una coincidencia sin sentido. Ésa
`languageName(code)` interna	Nombre del idioma para el prompt de la IA ("es" -> "Español").
`decodeHtmlEntities(raw)` interna	MyMemory devuelve el texto con entidades HTML (', ", &). Sin decodificarlas, una traducción con un apóstrofo llegaba a pantalla como "don't" y parecía un fallo del traductor.

AppTutorials.kt

`data/tutorials/AppTutorials.kt · 525 líneas`

App Tutorials - Complete guide for all features (2026 Edition) Comprehensive tutorials explaining every major feature of Nexus Chat

`getTutorial(feature)`	Sin comentario en el código. Por su nombre: get tutorial.
`getTutorialTitle(feature)`	Sin comentario en el código. Por su nombre: get tutorial title.

AppUpdateManager.kt

`data/update/AppUpdateManager.kt · 229 líneas`

Una versión publicada más nueva que la instalada.

`start()`	Sin comentario en el código. Por su nombre: start.
`listenToServer()` interna	Sin comentario en el código. Por su nombre: listen to server.
`fetchLatestFromGitHub()` interna	Sin comentario en el código. Por su nombre: fetch latest from git hub.
`offer(candidate)` interna	Sin comentario en el código. Por su nombre: offer.
`apkUrlFor(update)`	El APK que le corresponde a este teléfono, si está publicado.
`startDownload(update)`	Sin comentario en el código. Por su nombre: start download.
`install(file)`	Android pide confirmar y comprueba que el APK esté firmado con la misma llave.
`isMandatoryRelease(notes)`	Toda versión es obligatoria salvo que sus notas digan opcional.
`abiOf(fileName)`	"app-arm64-v8a-release.apk" → "arm64-v8a"; lo que no indique arquitectura sirve para todas.
`isNewerVersion(remote, installed)`	Compara "8.1.0" con "8.0.10" número a número: como texto, "8.0.10" saldría menor que "8.0.9".

Parte 28

Casos de uso

Reglas de negocio independientes de la interfaz.

SecurityRepository.kt

`domain/repository/SecurityRepository.kt · 34 líneas`

Repository interface for security operations including Tor integration Esta interfaz usa Orbot para proporcionar conectividad Tor en lugar de un binario Tor embebido.

`startTor()`	Inicia la detección de Orbot y establece conexión Tor
`stopTor()` asíncrona	Detiene el servicio Tor
`getTorState()`	Obtiene el estado actual de Tor
`getTorStatus()`	Obtiene información de conectividad Tor (Orbot disponible/activo)

DecryptMessageUseCase.kt

`domain/usecase/DecryptMessageUseCase.kt · 44 líneas`

Use case for decrypting messages using Signal Protocol E2EE. Handles: - Decrypting received encrypted messages - Establishing sessions from PreKey messages - Returning decrypted plaintext Requirements: 1.1, 1.2

`invoke(peerId, ciphertext, messageType)` asíncrona	Descifra un mensaje intercambiado con peerId. quien envió el mensaje. El secreto ECDH es simétrico, así que la misma clave descifra los mensajes recibidos y los propios; pasar aquí el `senderId` hacía imposible releer lo que uno mismo había escrito.

GoogleLoginUseCase.kt

`domain/usecase/auth/GoogleLoginUseCase.kt · 98 líneas`

`invoke(idToken)` asíncrona	Sin comentario en el código. Por su nombre: invoke.

LoginUseCase.kt

`domain/usecase/auth/LoginUseCase.kt · 17 líneas`

`invoke(email, password)` asíncrona	Sin comentario en el código. Por su nombre: invoke.

PhoneLoginUseCase.kt

`domain/usecase/auth/PhoneLoginUseCase.kt · 135 líneas`

Inicio de sesión con número de teléfono. Recibe la credencial ya construida (la obtiene com.Azelmods.App.ui.screens.auth.PhoneLoginViewModel a partir del SMS), autentica contra Firebase y, si es la primera vez, crea el perfil en `users/{uid}`. Por qué se escribe `email` aunque no haya correo La regla de `users/$uid` en `database.rules.json` valida `newData.hasChildren('displayName', 'email')`. Una cuenta de teléfono no tiene correo, así que se guarda una cadena vacía: el hijo existe —que es lo que la regla comprueba— y el resto de la app ya trata `email` vacío como "sin correo". Sin este campo la escritura del perfil se rechazaría y el usuario quedaría autenticado pero invisible para los dem

`invoke(credential)` asíncrona	Sin comentario en el código. Por su nombre: invoke.

`translateError(e)` interna	Traduce los errores de Firebase a algo accionable. El mensaje crudo de Firebase para un código mal escrito es "The sms verification code used to create the phone auth credential is invalid", que en pantalla no ayuda a nadie.

RegisterUseCase.kt

`domain/usecase/auth/RegisterUseCase.kt · 22 líneas`

`invoke(email, password, displayName)` asíncrona	Sin comentario en el código. Por su nombre: invoke.

Parte 29

Servicios

Notificaciones y servicio de llamada.

ActiveChat.kt

`service/ActiveChat.kt · 15 líneas`

El chat que se está viendo ahora mismo: sus mensajes no necesitan aviso. El identificador lo fija el ViewModel al resolverlo, y la pantalla dice si está a la vista.

`isShowing(id)`	Sin comentario en el código. Por su nombre: is showing.

CallNotificationReceiver.kt

`service/CallNotificationReceiver.kt · 66 líneas`

📞 Botón "Rechazar" de la notificación de llamada entrante. "Aceptar" ya no pasa por aquí: abre directamente la actividad (ver CallNotifications), porque Android 12+ bloquea abrir actividades desde un receptor lanzado por una notificación.

`onReceive(context, intent)`	Sin comentario en el código. Por su nombre: on receive.

CallNotifications.kt

`service/CallNotifications.kt · 130 líneas`

Notificación de llamada entrante: un único sitio que la crea y la retira. Problemas que resuelve respecto a la versión anterior: - **"Aceptar" no hacía nada en Android 12+.** El botón lanzaba un BroadcastReceiver que a su vez abría la actividad; Android 12 prohíbe ese "trampolín" desde notificaciones y la actividad no se abría. Ahora "Aceptar" abre directamente MainActivity. - **Sonaba una sola vez y no se retiraba nunca.** El tono de canal suena una vez salvo que la notificación sea insistente; y nadie la cancelaba al contestar, rechazar o cuando el otro colgaba, así que quedaba "sonando" fija en la barra. Ahora es `FLAG_INSISTENT` (repite el tono como un teléfono), se retira desde cancelIn

`notificationId(callId)`	Sin comentario en el código. Por su nombre: notification id.
`showIncoming(context, callId, callerName, callType, callerPhoto)`	Sin comentario en el código. Por su nombre: show incoming.
`cancelIncoming(context, callId)`	Sin comentario en el código. Por su nombre: cancel incoming.
`manager(context)` interna	Sin comentario en el código. Por su nombre: manager.

MessageNotificationReceiver.kt

`service/MessageNotificationReceiver.kt · 96 líneas`

Botones de la notificación de mensaje: responder y marcar como leído. Responder pasa por el mismo camino que escribir dentro del chat: antes la respuesta se guardaba directamente en el servidor, sin cifrar.

`databaseRepository()`	Sin comentario en el código. Por su nombre: database repository.
`onReceive(context, intent)`	Sin comentario en el código. Por su nombre: on receive.
`handleReply(context, chatId, intent)` interna	Sin comentario en el código. Por su nombre: handle reply.
`handleMarkAsRead(context, chatId)` interna	Sin comentario en el código. Por su nombre: handle mark as read.
`notificationManager(context)` interna	Sin comentario en el código. Por su nombre: notification manager.

NexusFirebaseMessagingService.kt

`service/NexusFirebaseMessagingService.kt · 513 líneas`

🔔 Nexus Chat Notifications — Sistema de notificaciones premium Canales de notificación: - nexus_messages → mensajes de chat (Importance HIGH, con sonido y vibración) - nexus_calls → llamadas entrantes (Importance HIGH, ringtone largo) - nexus_missed_calls → llamadas perdidas (Importance DEFAULT) - nexus_stories → historias nuevas (Importance DEFAULT) - nexus_ai → respuestas de Azel IA (Importance DEFAULT)

`onNewToken(token)`	Sin comentario en el código. Por su nombre: on new token.
`onMessageReceived(message)`	Sin comentario en el código. Por su nombre: on message received.
`withChatNotMuted(data, onAllowed)` interna	Ejecuta onAllowed salvo que el usuario haya silenciado ese chat. Ante cualquier duda —sin sesión, sin chatId, fallo de red al leer el ajuste— se notifica. Perder un mensaje por un error de lectura es peor que una notificación de más en un chat silenciado.
`messageText(data, chatId, senderId)` interna	El texto real del aviso. Un mensaje cifrado llega como texto cifrado: el servidor no puede leerlo, así que se descifra aquí, con las mismas claves que usa el chat.
`showIncomingCallNotification(data)` interna	Sin comentario en el código. Por su nombre: show incoming call notification.
`showMissedCallNotification(data)` interna	Sin comentario en el código. Por su nombre: show missed call notification.
`showMessageNotification(data)` interna	Sin comentario en el código. Por su nombre: show message notification.
`showMessageSummary(manager)` interna	Muestra una notificación de resumen para el grupo de mensajes. Así las notificaciones se agrupan como en WhatsApp.
`showStoryNotification(data)` interna	Sin comentario en el código. Por su nombre: show story notification.
`showAINotification(data)` interna	Sin comentario en el código. Por su nombre: show ainotification.
`showPrivateNotice(title, body)` interna	Aviso simple, que abre la app. Un chat secreto nunca muestra de quién ni qué dice.
`showUpdateNotification(data)` interna	Sin comentario en el código. Por su nombre: show update notification.
`loadContactPhoto(photoUrl)` interna	Carga la foto de perfil del contacto como Bitmap para la notificación. Se ejecuta en segundo plano para no bloquear.

NotificationHelper.kt

`service/NotificationHelper.kt · 144 líneas`

📞 NotificationHelper — Utilidad para mostrar notificaciones desde cualquier parte de la app. También expone un contador de llamadas perdidas para mostrar un badge en el tab de Calls dentro de MainScreen.

`resetMissedCallCount()`	Reincia el contador (llamar desde CallsScreen al abrir).
`showMissedCallNotification(context, callId, callerName, callerPhotoUrl, callerId)`	Muestra una notificación de "Llamada perdida de callerName" con un botón "Call back" que abre la pantalla de llamada al contacto.
`ensureMissedCallsChannelExists(context)` interna	Crea el canal de llamadas perdidas si aún no existe. Llamar desde cualquier lugar antes de notificar.

Parte 30

WebRTC

Conexión de las llamadas.

CallAudioRouter.kt

`webrtc/CallAudioRouter.kt · 116 líneas`

Enrutado del audio de las llamadas. Antes el modo de comunicación solo se activaba al pulsar "Altavoz" y nunca se restauraba. Consecuencias: la llamada sonaba por el canal multimedia (sin cancelación de eco del sistema, con el volumen de la música), el auricular Bluetooth se ignoraba y, después de colgar, el teléfono se quedaba en modo llamada y la música de otras apps salía por el auricular. Ahora: - start pone `MODE_IN_COMMUNICATION` al empezar la llamada y elige la salida: auricular Bluetooth/cable si hay uno conectado; si no, altavoz en vídeo y auricular del teléfono en voz (como cualquier app de llamadas). - setSpeakerOn alterna altavoz y salida "privada". - stop lo deja todo como estab

`start(isVideoCall)`	Sin comentario en el código. Por su nombre: start.
`setSpeakerOn(on)`	Sin comentario en el código. Por su nombre: set speaker on.
`toggleSpeaker()`	Sin comentario en el código. Por su nombre: toggle speaker.
`stop()`	Sin comentario en el código. Por su nombre: stop.
`setDevice(device, speaker)` interna	Sin comentario en el código. Por su nombre: set device.
`findHeadset()` interna	Sin comentario en el código. Por su nombre: find headset.

CallRingback.kt

`webrtc/CallRingback.kt · 44 líneas`

El "tuuu… tuuu…" que oye quien llama mientras suena en el otro teléfono. Sin esto, llamar era un silencio total y parecía que la llamada no funcionaba. Se usa el tono estándar del sistema por el canal de voz, así que sale por el mismo sitio que la llamada: auricular en voz, altavoz en vídeo.

`start()`	Sin comentario en el código. Por su nombre: start.
`stop()`	Sin comentario en el código. Por su nombre: stop.

IceServerProvider.kt

`webrtc/IceServerProvider.kt · 113 líneas`

Servidores STUN/TURN de las llamadas. Antes estaban fijados en el código, con el TURN público y compartido de openrelay.metered.ca. Un TURN gratuito no garantiza nada: cuando se satura o cambia sus credenciales, las llamadas entre redes móviles (CG-NAT) dejan de conectar y la única forma de arreglarlo era publicar otra versión de la app. Ahora se leen de la base de datos, en `config/webrtc/iceServers`, con este formato (lo escribe el administrador desde la consola de Firebase; las reglas no dejan escribirlo a los clientes): `urls` admite una cadena o una lista. Si el nodo no existe, no se puede leer o tarda más de READ_TIMEOUT_MS, se usan DEFAULT_ICE_SERVERS: la llamada sale igual, sin queda

`hasTurn(servers)`	`true` si la lista activa incluye algún TURN (necesario para `relayOnly`).
`getIceServers()` asíncrona	Sin comentario en el código. Por su nombre: get ice servers.
`parse(raw)` interna	Sin comentario en el código. Por su nombre: parse.

WebRTCManager.kt

`webrtc/WebRTCManager.kt · 551 líneas`

Un único contexto gráfico para cámara, codificador, decodificador y pantalla. La imagen viaja entre ellos como textura de OpenGL, y una textura solo existe dentro del contexto que la creó. Con un contexto para cada pieza, nadie podía leer las texturas de los demás: la cámara se abría, la llamada conectaba y el vídeo se veía en negro en los dos teléfonos. / private val rootEglBase: EglBase = EglBase.create() val eglContext: EglBase.Context get() = rootEglBase.eglBaseContext // State flows private val _localVideoTrackFlow = MutableStateFlow<VideoTrack?>(null) val localVideoTrackFlow: StateFlow<VideoTrack?> = _localVideoTrackFlow.asStateFlow() private val _remoteVideoTrackFlow = MutableStateFlo

`initializePeerConnectionFactory()` interna	Sin comentario en el código. Por su nombre: initialize peer connection factory.
`initializePeerConnection(isVideoCall, callId, iceServers, relayOnly)`	Crea la PeerConnection de una llamada. los lee de la configuración remota (`config/webrtc/iceServers`) para poder poner un TURN propio en producción sin publicar otra versión de la app. modo anónimo activo: sin esto, WebRTC anuncia al otro participante las IP reales del dispositivo (candidatos host/srflx), que es justo lo que el modo anónimo pretende ocultar.
`onIceCandidate(candidate)`	Sin comentario en el código. Por su nombre: on ice candidate.
`onDataChannel(dataChannel)`	Sin comentario en el código. Por su nombre: on data channel.
`onIceConnectionChange(state)`	Sin comentario en el código. Por su nombre: on ice connection change.
`onIceConnectionReceivingChange(receiving)`	Sin comentario en el código. Por su nombre: on ice connection receiving change.
`onIceGatheringChange(state)`	Sin comentario en el código. Por su nombre: on ice gathering change.
`onAddStream(stream)`	Sin comentario en el código. Por su nombre: on add stream.
`onSignalingChange(state)`	Sin comentario en el código. Por su nombre: on signaling change.
`onIceCandidatesRemoved(candidates)`	Sin comentario en el código. Por su nombre: on ice candidates removed.
`onRemoveStream(stream)`	Sin comentario en el código. Por su nombre: on remove stream.
`onRenegotiationNeeded()`	Sin comentario en el código. Por su nombre: on renegotiation needed.
`onAddTrack(receiver, streams)`	Sin comentario en el código. Por su nombre: on add track.
`onConnectionChange(newState)`	Sin comentario en el código. Por su nombre: on connection change.
`onCameraError(error)`	Sin comentario en el código. Por su nombre: on camera error.
`onCameraDisconnected()`	Sin comentario en el código. Por su nombre: on camera disconnected.
`onCameraFreezed(error)`	Sin comentario en el código. Por su nombre: on camera freezed.
`onCameraOpening(cameraName)`	Sin comentario en el código. Por su nombre: on camera opening.
`onFirstFrameAvailable()`	Sin comentario en el código. Por su nombre: on first frame available.
`onCameraClosed()`	Sin comentario en el código. Por su nombre: on camera closed.
`createLocalMediaTracks(isVideoCall)` interna	Sin comentario en el código. Por su nombre: create local media tracks.
`createOffer()`	Sin comentario en el código. Por su nombre: create offer.
`onCreateSuccess(sdp)`	Sin comentario en el código. Por su nombre: on create success.
`onSetSuccess()`	Sin comentario en el código. Por su nombre: on set success.
`onCreateFailure(error)`	Sin comentario en el código. Por su nombre: on create failure.
`onSetFailure(error)`	Sin comentario en el código. Por su nombre: on set failure.
`createAnswer()`	Sin comentario en el código. Por su nombre: create answer.
`setRemoteDescription(sdp, type)`	Aplica la descripción remota. Con una OFERTA, la respuesta se crea DENTRO de `onSetSuccess`. Antes el ViewModel llamaba a `createAnswer()` justo después de `setRemoteDescription()`, pero las dos operaciones son asíncronas: si `createAnswer` llegaba primero WebRTC la rechazaba ("no remote description") y el receptor nunca respondía. La llamada se quedaba en "Conectando…" para siempre en dispositivos rápidos.
`flushPendingIceCandidates()` interna	Sin comentario en el código. Por su nombre: flush pending ice candidates.
`addIceCandidate(candidate)`	Sin comentario en el código. Por su nombre: add ice candidate.
`toggleAudio(enabled)`	Sin comentario en el código. Por su nombre: toggle audio.

`toggleMic()`	Invierte el micrófono (lo usa el botón de la notificación de la llamada).
`toggleVideo(enabled)`	Sin comentario en el código. Por su nombre: toggle video.
`switchCamera()`	Sin comentario en el código. Por su nombre: switch camera.
`onCameraSwitchDone(isFrontCamera)`	Sin comentario en el código. Por su nombre: on camera switch done.
`onCameraSwitchError(error)`	Sin comentario en el código. Por su nombre: on camera switch error.
`cleanup()`	Libera todo lo de la llamada actual. El ORDEN importa y no es el que había. Antes se destruían las pistas y el capturador primero y sólo al final se ponían los `StateFlow` a null: entre una cosa y otra la interfaz seguía suscrita a una `VideoTrack` ya destruida, y pintar sobre un objeto nativo liberado es un cierre inmediato de la app, no una excepción que estos `try/catch` puedan recoger. Ahora se avisa primero a la

Parte 31

Inyección de dependencias

Módulos de Hilt.

AppModule.kt

`di/AppModule.kt · 110 líneas`

`provideContext(context)`	Sin comentario en el código. Por su nombre: provide context.
`provideCoroutineScope()`	Sin comentario en el código. Por su nombre: provide coroutine scope.
`provideCredentialManager(context)`	Sin comentario en el código. Por su nombre: provide credential manager.
`provideGoogleClientId(context)`	Sin comentario en el código. Por su nombre: provide google client id.
`provideTutorialPreferences(context)`	Sin comentario en el código. Por su nombre: provide tutorial preferences.
`provideAppBackgroundManager(context)`	Sin comentario en el código. Por su nombre: provide app background manager.
`provideBotPreferences(context)`	Sin comentario en el código. Por su nombre: provide bot preferences.
`provideInternalBotRepository(botPreferences)`	Sin comentario en el código. Por su nombre: provide internal bot repository.
`provideAppDatabase(context)`	Sin comentario en el código. Por su nombre: provide app database.
`provideCacheManager(db)`	Sin comentario en el código. Por su nombre: provide cache manager.
`provideDataStore(context)`	Sin comentario en el código. Por su nombre: provide data store.

FirebaseModule.kt

`di/FirebaseModule.kt · 66 líneas`

`provideFirebaseAuth()`	Sin comentario en el código. Por su nombre: provide firebase auth.
`provideFirebaseDatabase(context)`	Sin comentario en el código. Por su nombre: provide firebase database.
`provideFirebaseStorage()`	Sin comentario en el código. Por su nombre: provide firebase storage.
`provideFirebaseMessaging()`	Sin comentario en el código. Por su nombre: provide firebase messaging.
`provideChatBackgroundRepository(database)`	Sin comentario en el código. Por su nombre: provide chat background repository.
`provideFirebaseManager(auth, database, storage, messaging)`	Sin comentario en el código. Por su nombre: provide firebase manager.

RepositoryModule.kt

`di/RepositoryModule.kt · 37 líneas`

`bindAuthRepository(authRepositoryImpl)`	Sin comentario en el código. Por su nombre: bind auth repository.
`bindChatRepository(chatRepositoryImpl)`	Sin comentario en el código. Por su nombre: bind chat repository.
`bindUserRepository(userRepositoryImpl)`	Sin comentario en el código. Por su nombre: bind user repository.
`bindSecurityRepository(securityRepositoryImpl)`	Sin comentario en el código. Por su nombre: bind security repository.

SecurityModule.kt

`di/SecurityModule.kt · 58 líneas`

Hilt module for security-related dependencies Provides: - TorService for Orbot detection and proxy status - TorDnsResolver for DNS-over-Tor (prevents DNS leaks) - FirebaseProxyConfigurator for Firebase-over-Tor

`provideTorService(context, torDnsResolver, firebaseProxyConfigurator, embeddedTor, preferences)`	Sin comentario en el código. Por su nombre: provide tor service.
`provideTorDnsResolver()`	Sin comentario en el código. Por su nombre: provide tor dns resolver.
`provideFirebaseProxyConfigurator(database, auth, storage)`	Sin comentario en el código. Por su nombre: provide firebase proxy configurator.

Parte 32

Aplicación

Arranque de la app y utilidades generales.

MainActivity.kt

`MainActivity.kt · 359 líneas`

Holds a pending call-screen navigation request coming from a notification full-screen intent (incoming call). Compose observes this and navigates. / private val callNavRequest = mutableStateOf<CallNavRequest?>(null) /** Destino pedido por una notificación; Compose lo consume y navega.

`attachBaseContext(newBase)`	Sin comentario en el código. Por su nombre: attach base context.
`onCreate(savedInstanceState)`	Sin comentario en el código. Por su nombre: on create.
`onNewIntent(intent)`	Sin comentario en el código. Por su nombre: on new intent.
`handleCallIntent(intent)` interna	Traduce los extras de las notificaciones (llamada entrante, contestar desde la notificación, mensaje, llamada perdida…) a una ruta de navegación. Antes solo se atendía "incoming_call": tocar la notificación de un mensaje o de una llamada perdida abría la app en la pantalla de inicio sin más.
`handleDeepLink(intent)` interna	Sin comentario en el código. Por su nombre: handle deep link.
`onResume()`	Sin comentario en el código. Por su nombre: on resume.
`onDestroy()`	Sin comentario en el código. Por su nombre: on destroy.
`onPause()`	Sin comentario en el código. Por su nombre: on pause.

NexusChatApplication.kt

`NexusChatApplication.kt · 277 líneas`

Application-wide coroutine scope backed by Dispatchers.IO. Using IO instead of Main ensures that background work (Firebase calls, demo-account seeding, etc.) never competes with the UI thread. SupervisorJob prevents a failure in one child coroutine from cancelling sibling coroutines. / private val applicationScope = CoroutineScope(SupervisorJob() + Dispatchers.IO) // Lazy: TorService solo se crea si el usuario dejó activado el modo anónimo. @Inject lateinit var torService: dagger.Lazy<TorService> @Inject lateinit var torPreferences: TorPreferences /** Modo anónimo de cierre seguro desde el primer instante: esto corre antes que los ContentProvider de Firebase, así que ninguna conexión Java sa

`attachBaseContext(base)`	Modo anónimo de cierre seguro desde el primer instante: esto corre antes que los ContentProvider de Firebase, así que ninguna conexión Java sale por la red normal mientras Tor arranca, ni siquiera si la app la despierta una notificación.
`onCreate()`	Sin comentario en el código. Por su nombre: on create.
`initializeDemoAccount()` interna	Seeds the demo account once a signed-in Firebase user is available. If the user is already authenticated at launch we call DemoAccountManager immediately. Otherwise we register a one-shot FirebaseAuth.AuthStateListener that removes itself as soon as the user signs in, preventing a memory leak from a permanently-retained listener.
`registerOneTimeDemoListener()` interna	Registers a FirebaseAuth.AuthStateListener that self-removes after the first sign-in event, avoiding a permanent strong reference to the listener.
`newImageLoader(context)`	Provides the application-wide ImageLoader used by Coil. Memory cache : 25 % of available heap (Coil default heuristic). Disk cache : capped at 50 MB — generous enough for chat thumbnails while staying well within typical device storage budgets. DebugLogger : enabled only in debug builds to avoid log spam in release.
`createNotificationChannels()` interna	Crea todos los canales de notificación que usa la app. Se llama desde onCreate y es seguro llamarlo múltiples veces (es idempotente).

I18n.kt

`i18n/I18n.kt · 96 líneas`

Idioma de la interfaz. Con SYSTEM se sigue el del teléfono.

`init(context)`	Debe llamarse antes de pintar nada: Application.attachBaseContext.
`selected(context)`	Sin comentario en el código. Por su nombre: selected.
`select(activity, language)`	Sin comentario en el código. Por su nombre: select.
`wrap(base)`	Para que Android y las fechas usen también el idioma elegido en cada pantalla.
`translate(text)`	Sin comentario en el código. Por su nombre: translate.
`loadEnglish(context)` interna	Sin comentario en el código. Por su nombre: load english.
`tr(text)`	Texto en el idioma de la app.
`tr(format, args)`	Igual, con valores: `tr("Hola %s", nombre)`.

CallService.kt

`services/CallService.kt · 309 líneas`

Servicio en primer plano de la llamada en curso. Mantiene vivo el micrófono (y la cámara) con la app en segundo plano y muestra la notificación "llamada en curso". Correcciones respecto a la versión anterior: - **"Colgar" de la notificación no colgaba.** Solo paraba el servicio: la PeerConnection seguía abierta, el otro seguía escuchando y la llamada nunca se cerraba en la base de datos. Ahora cierra la llamada de verdad. - **"Silenciar" de la notificación no silenciaba.** Solo cambiaba su propio icono. Ahora actúa sobre la pista de audio compartida de WebRTCManager, la misma que usa el botón de la pantalla, y ambos se ven el uno al otro. - **El cronómetro contaba desde que empezaba a sonar.

`onCreate()`	Sin comentario en el código. Por su nombre: on create.
`onStartCommand(intent, flags, startId)`	Sin comentario en el código. Por su nombre: on start command.
`startForegroundService()` interna	Sin comentario en el código. Por su nombre: start foreground service.
`observeCall()` interna	Reconstruye la notificación cuando cambia el estado de la conexión o del micrófono, y arranca el cronómetro la primera vez que el medio conecta.
`statusFor(state)` interna	Sin comentario en el código. Por su nombre: status for.
`startTimer()` interna	Sin comentario en el código. Por su nombre: start timer.
`hangUpFromNotification()` interna	Sin comentario en el código. Por su nombre: hang up from notification.
`stopEverything()` interna	Sin comentario en el código. Por su nombre: stop everything.
`notify(notification)` interna	Sin comentario en el código. Por su nombre: notify.
`formatDuration(elapsedMs)` interna	Formats elapsed milliseconds into HH:MM:SS or MM:SS.
`buildNotification(statusText, muted)` interna	Sin comentario en el código. Por su nombre: build notification.
`createNotificationChannel()` interna	Sin comentario en el código. Por su nombre: create notification channel.
`onBind(intent)`	Sin comentario en el código. Por su nombre: on bind.
`onDestroy()`	Sin comentario en el código. Por su nombre: on destroy.

FirebaseInitializer.kt

`startup/FirebaseInitializer.kt · 64 líneas`

FirebaseInitializer - App Startup Initializer for Firebase. Integrates Firebase initialization into the AndroidX App Startup framework, giving explicit control over initialization order and timing. To enable lazy/manual initialization (disable automatic Firebase ContentProvider), add the following to AndroidManifest.xml inside <application>: <provider android:name="com.google.firebase.provider.FirebaseInitProvider"

android:authorities="${applicationId}.firebaseinitprovider" android:exported="false" tools:node="remove" /> Then register this initializer via App Startup in AndroidManifest.xml: <provider android:name="androidx.startup.InitializationProvider" android:authorities="${applicationId}

`create(context)`	Sin comentario en el código. Por su nombre: create.
`dependencies()`	No dependencies required before Firebase initializes. Add other Initializer classes here if Firebase must run after them.

CrashlyticsLogger.kt

`util/CrashlyticsLogger.kt · 141 líneas`

🛡️ CrashlyticsLogger — Utility for structured error logging Provides consistent crash reporting across the app with: - Custom keys for context (feature, operation, user ID) - Log statements that sync with crash reports - Non-fatal exception recording - Sensitive data filtering (auto-strips tokens, passwords)

`setUserId(uid)`	Set user ID for crash reporting. Call after login/logout.
`clearUserId()`	Clear user ID on logout.
`setCustomKey(key, value)`	Set a custom key for crash context.
`log(message)`	Log a message that will be attached to the next crash report.
`recordException(throwable, feature, operation, message)`	Record a non-fatal exception with context.
`recordRecoverableError(throwable, feature, operation, userMessage)`	Record a recoverable error (non-fatal, with user-friendly message).
`warn(feature, operation, message)`	Log a warning (non-critical, not sent as crash).
`sanitize(input)` interna	Sanitize sensitive data before sending to Crashlytics. Strips tokens, passwords, API keys from log messages.

GlobalErrorHandler.kt

`util/GlobalErrorHandler.kt · 157 líneas`

GLOBAL ERROR HANDLER — ENTERPRISE GRADE 2026

Captura crashes no manejados y guarda logs para diagnóstico Previene pérdida total de la app al crashear @since 2026 @version 3.0.0 Enterprise Edition @author AzelMods677

`install(context)`	Sin comentario en el código. Por su nombre: install.
`uncaughtException(thread, throwable)`	Sin comentario en el código. Por su nombre: uncaught exception.
`buildCrashLog(thread, throwable)` interna	Sin comentario en el código. Por su nombre: build crash log.
`saveCrashLog(log)` interna	Sin comentario en el código. Por su nombre: save crash log.
`logToAndroid(throwable)` interna	Sin comentario en el código. Por su nombre: log to android.
`getLastCrashLog()`	Obtiene el último crash log guardado
`getCrashCount()`	Obtiene el conteo total de crashes
`clearCrashLogs()`	Limpia los logs de crashes

Resource.kt

`util/Resource.kt · 10 líneas`

AudioRecorder.kt

`utils/AudioRecorder.kt · 95 líneas`

`startRecording()`	Sin comentario en el código. Por su nombre: start recording.
`stopRecording()`	Sin comentario en el código. Por su nombre: stop recording.
`cancelRecording()`	Sin comentario en el código. Por su nombre: cancel recording.
`isRecording()`	Sin comentario en el código. Por su nombre: is recording.
`getRecordingDuration()`	Sin comentario en el código. Por su nombre: get recording duration.

FCMTokenManager.kt

`utils/FCMTokenManager.kt · 73 líneas`

Ruta de los tokens de push. Vive FUERA de `/users` a propósito. La lista de contactos necesita leer la colección `/users` entera, y en Firebase el permiso de lectura CASCADEA hacia abajo: una regla más estricta en `users/$uid/fcmTokens` no puede revocar el permiso concedido en el padre. Con los tokens en su propio nodo raíz, cada usuario sigue siendo el único que puede leer y escribir los suyos. Las Cloud Functions los leen con el Admin SDK, que ignora las reglas.

`tokenPath(uid)`	Ruta de los tokens de push. Vive FUERA de `/users` a propósito. La lista de contactos necesita leer la colección `/users` entera, y en Firebase el permiso de lectura CASCADEA hacia abajo: una regla más estricta en `users/$uid/fcmTokens` no puede revocar el permiso concedido en el padre. Con los tokens en su propio nodo raíz, cada usuario sigue siendo el único que puede leer y escribir los suyos. Las Cloud Functions l
`saveFCMToken(userId)` asíncrona	Save FCM token to Firebase for the current user
`deleteFCMToken(userId)` asíncrona	Delete FCM token from Firebase (on logout)

FirebaseErrors.kt

`utils/FirebaseErrors.kt · 20 líneas`

Convierte un fallo de Firebase en una frase clara para quien usa la app. El texto crudo ("Permission denied", "Client is offline") no dice qué hacer. Aquí se distingue sesión caducada, permiso denegado y falta de red, que tienen arreglos distintos.

`friendlyFirebaseError(e)`	Convierte un fallo de Firebase en una frase clara para quien usa la app. El texto crudo ("Permission denied", "Client is offline") no dice qué hacer. Aquí se distingue sesión caducada, permiso denegado y falta de red, que tienen arreglos distintos.

PermissionHelper.kt

`utils/PermissionHelper.kt · 58 líneas`

`hasAudioPermission(context)`	Sin comentario en el código. Por su nombre: has audio permission.
`hasMediaPermissions(context)`	Sin comentario en el código. Por su nombre: has media permissions.
`hasCameraPermission(context)`	Sin comentario en el código. Por su nombre: has camera permission.

PhotoEnhancer.kt

`utils/PhotoEnhancer.kt · 98 líneas`

Realce automático de fotos, **en el dispositivo**. No hay ninguna IA remota detrás ni falta que hace: lo que mejora una foto de chat es subir un poco el contraste, la saturación y el brillo, y eso es una matriz de color que la GPU aplica en milisegundos. Mandar la foto a un servicio externo para esto sería regalar las fotos privadas del usuario a cambio de un

ajuste que se puede calcular localmente. La imagen original nunca se toca: el resultado se escribe en un archivo nuevo dentro de la caché de la app.

`enhance(context, source)`	Devuelve la Uri de una copia realzada de source, o la propia source si algo falla. Nunca lanza: que el realce no salga no puede impedir que se envíe la foto.
`applyMatrix(source)` interna	Contraste +12 %, saturación +15 % y un punto de brillo. Son valores conservadores a propósito: un realce agresivo quema los tonos de piel y satura los cielos, y en una foto de chat eso se nota más que la mejora.

StoryExporter.kt

`utils/StoryExporter.kt · 88 líneas`

Helper that persists a composed Story bitmap (the editor preview with all of its text / sticker / emoji overlays already rendered on top of the media) into a temporary file so it can be uploaded to Storage. The story editor captures its editable area to a Bitmap using a Compose GraphicsLayer (WYSIWYG: the saved pixels match exactly what the user sees in the preview, so no manual coordinate scaling is required). This object only handles writing that bitmap to disk and handing back a Uri.

`saveComposedStory(context, bitmap)` asíncrona	Compress bitmap to a JPEG inside the app cache directory and return a file Uri suitable for com.google.firebase.storage.StorageReference.putFile. Runs on Dispatchers.IO. Throws on failure so the caller can fall back to uploading the original media. IMPORTANT: a Compose `GraphicsLayer.toImageBitmap()` can return a bitmap backed by Bitmap.Config.HARDWARE on many devices. Hardware bitmaps are GPU-only and several pipeli
`toEncodableSoftwareBitmap()` interna	Return a software, JPEG-encodable copy of this when the source is a HARDWARE bitmap (or otherwise not ARGB_8888). The copy is flattened onto an opaque black background so any transparent regions don't turn black-with-artifacts in JPEG. When the bitmap is already a safe software ARGB_8888 buffer it is returned as-is.

StoryVideoComposer.kt

`utils/StoryVideoComposer.kt · 207 líneas`

Burns the story editor overlays (text / stickers / emojis, already rasterized into a single transparent Bitmap) into every frame of a video using Media3 Transformer. Alignment strategy (WYSIWYG): - The editor preview renders the video with `RESIZE_MODE_FIT` (letterbox) inside the editable area, and the overlays are positioned in that same area. - The captured overlayBitmap therefore has the exact pixel size of the editable area, is transparent where the video shows through, and contains the overlays at their on-screen positions. - We force the export resolution to the overlay's dimensions with Presentation.LAYOUT_SCALE_TO_FIT (same letterbox as the preview) and draw the overlay 1:1 on top. T

`composeVideoWithOverlay(context, inputUri, overlayBitmap)` asíncrona	Produce a new MP4 in the app cache with overlayBitmap burned into the video at inputUri. Returns the output Uri. Throws on failure so the caller can fall back to uploading the original clip.
`fitUploadLimit(context, uri)` asíncrona	El servidor rechaza archivos de 100 MB o más. Un vídeo que se acerque a ese tope se convierte a 720p antes de subirlo; si no se puede, se devuelve tal cual y el servidor decide.
`runDownscale(context, inputUri, outputFile)` interna asíncrona	Sin comentario en el código. Por su nombre: run downscale.
`onCompleted(composition, exportResult)`	Sin comentario en el código. Por su nombre: on completed.
`onError(composition, exportResult, exportException)`	Sin comentario en el código. Por su nombre: on error.
`runTransformer(context, inputUri, overlay, outputFile)` interna asíncrona	Sin comentario en el código. Por su nombre: run transformer.

`toEvenSizedArgb()` interna	Return a software ARGB_8888 copy of this with both dimensions rounded down to an even number (a hard requirement for most hardware video encoders). HARDWARE bitmaps are detached from the GPU buffer so they can be uploaded as GL textures.

VideoThumbnailExtractor.kt

`utils/VideoThumbnailExtractor.kt · 56 líneas`

Extract thumbnail from video Uri using MediaMetadataRetriever Returns null if extraction fails

`extractThumbnail(context, videoUri)`	Extract thumbnail from video Uri using MediaMetadataRetriever Returns null if extraction fails
`isVideoUri(context, uri)`	Check if Uri is a video based on MIME type